



Infor Mongoose Guide to the Application Event System

Important Notices

The material contained in this publication (including any supplementary information) constitutes and contains confidential and proprietary information of Infor.

By gaining access to the attached, you acknowledge and agree that the material (including any modification, translation or adaptation of the material) and all copyright, trade secrets and all other right, title and interest therein, are the sole property of Infor and that you shall not gain right, title or interest in the material (including any modification, translation or adaptation of the material) by virtue of your review thereof other than the non-exclusive right to use the material solely in connection with and the furtherance of your license and use of software made available to your company from Infor pursuant to a separate agreement, the terms of which separate agreement shall govern your use of this material and all supplemental related materials ("Purpose").

In addition, by accessing the enclosed material, you acknowledge and agree that you are required to maintain such material in strict confidence and that your use of such material is limited to the Purpose described above. Although Infor has taken due care to ensure that the material included in this publication is accurate and complete, Infor cannot warrant that the information contained in this publication is complete, does not contain typographical or other errors, or will meet your specific requirements.

As such, Infor does not assume and hereby disclaims all liability, consequential or otherwise, for any loss or damage to any person or entity which is caused by or relates to errors or omissions in this publication (including any supplementary information), whether such errors or omissions result from negligence, accident or any other cause.

Without limitation, U.S. export control laws and other applicable export and import laws govern your use of this material and you will neither export or re-export, directly or indirectly, this material nor any related materials or supplemental information in violation of such laws, or use such materials for any purpose prohibited by such laws.

Trademark Acknowledgements

The word and design marks set forth herein are trademarks and/or registered trademarks of Infor and/or related affiliates and subsidiaries. All rights reserved. All other company, product, trade or service names referenced may be registered trademarks or trademarks of their respective owners.

Publication Information

Release: Infor Mongoose 9.01

Publication date: May 2, 2014

Contents

About This Guide	9
Chapter 1: About the Application Event System	11
Basic elements of the Application Event System	11
Advantages of the Application Event System	13
Examples of ways to use the Application Event System	14
Example 1: Sending a notification when a record is added	14
Example 2: Getting approval for a credit limit change	15
Example 3: Complex approval of a purchase order status change	15
Example 4: Automatically shipping a customer order	15
Workflow event handlers	16
Chapter 2: How the Application Event System Works	17
How events are handled	17
When events can be generated	17
Where events can be generated	18
Controlling the sequence of event handlers and actions	18
Restricting which handlers run	18
Using the Session Access As form	19
Using event handler settings	20
Synchronicity	20
Synchronous events	20
Asynchronous events	20
Event handlers	21
Suspension	22
When an event is suspended	22
When an event is not suspended	24
Payload	24
Adjournment and resumption	25
Success, failure, and retries	25

Success	25
Failure	25
Retrying event handlers	27
Transactions	28
Transactions with synchronous events	28
Event handlers marked as transactional	29
Event handlers not marked as transactional	30
Rolling back transactions	30
The Framework Event Service	31
Setting up the Framework Event Service	31
Processing order in the Framework Event Service	31
Administrative details	32
Events and event handler revisions	32
Chapter 3: Designing and Using Events and Handlers	35
Elements of the Application Event System	35
About the Access As identifier	35
About events	36
About event triggers	38
About event handlers	40
About event actions	41
About event action parameters	42
About event action parameter forms	44
About event variables and initial states	45
About event global constants	46
Application Event System design forms	47
Setting up custom events and handlers	49
Designing a custom event	50
About event handling and order	51
Determining names of IDO collections and components	53
Refreshing the metadata cache	54
Chapter 4: Tracking Event System Status	57
Event Status form	57
Event Handler Status form	58
Event Queue form	58
Event Revisions form	59
Event Handler Revisions form	59

Suspended Updates form	59
Chapter 5: Event Messages	61
Event message-related forms	61
The Inbox form	61
The Saved Messages form	63
The Send Message form	64
Send email to external email Inbox for prompts	64
Prompts and responses	65
Voting rules	66
Dealing with indeterminate voting results	69
Quorums	70
Appendix A: Sample Scenarios	71
Sending notifications	72
Requesting approvals	88
Modifying records	113
Voting	115
Localizing message contents	117
More advanced scenarios	119
.	121
Appendix B: Reference Tables	123
Firing events	124
Summary of synchronous functionality	125
Core application events	126
Framework events (non-IDO)	126
Framework IDO events (non-suspendible)	127
Suspendible framework IDO events	129
Non-framework events	130
Framework Event Attributes	132
Event Action Types	139
Event Action Parameters	147
Expression Functions	171
Standard Functions	171
Pre-parser Functions	193

Expression Operators	194
Appendix C: Expression Grammar	197
Restrictions	198
Start Symbol	198
Character Sets	199
Terminals	199
Rules	200
Variable, Constant, and Event Parameter References	200
Expressions	201
Boolean Rules	201
Typeless Rules	204
String Rules	206
Numeric Rules	210
Date Rules	212
Restricted Arguments	220
Keyword Paren Lists	221
Enumerations	221
Appendix D: Sample Stored Procedures	223
Passing Parameters to a Synchronous Event	223
Calling a Synchronous Event within a Transaction and Handling Failure	223
Appendix E: Synchronization of Metadata	225
Overview and rationale	225
The Inherent Hierarchy of Metadata	226
Maintaining Handler IDs Through Metadata Updates	227
Protecting Running Events from Metadata Changes	227
Detailed Examples	228
Using Specific Chronology	228
Using Non-specific Chronology	232
Performing Upgrades	240
Overriding Others' Handlers	244
Non-exclusive Overrides	245
Exclusive Overrides	247
Disabling the Ability to Override	249
Dealing with Obsolete Handlers	249

Appendix F: Event Flow Options	253
Glossary	257
Index	263

About This Guide

This guide describes the application event system, which developers and system administrators can use to customize their systems. With it, developers and system administrators can create events and event handlers to automate many tasks that would otherwise have to be done manually, with less effort and less risk of error.

This version is current with Infor Mongoose 9.01. At this time, the examples in this guide assume the use of Mongoose, though the principles and practices apply to any application (such as SyteLine) built on the Infor Mongoose framework.

Intended audience

The intended audience for this documentation includes:

- Mongoose-based application and form developers
- System administrators
- Anyone else tasked with using the Application Event System to develop or modify forms in Mongoose-based applications

Related documents

In addition to this guide, the online help for your Mongoose-based application includes a significant number of topics designed to help you use this functionality most effectively.

The most current version of all documentation is available on the Infor support web pages (see [Contacting Infor](#)).

Prerequisite knowledge

This publication assumes that you have at least some knowledge in these areas:

- System architecture and function (including the tier structure) for your Infor system
- SQL Server database

If you do not have this prerequisite knowledge, we advise you to obtain it before going any further. For the most up-to-date list of software and hardware requirements for Infor products, see the documentation for your system.

See also the *Guide to Technology* on our support web sites (see Contacting Infor). This document lists typical system administration tasks you should be familiar with before attempting to install and administer Infor products.

Contacting Infor

If you have questions about Infor products, go to the Infor Xtreme Support portal at <http://www.infor.com/inforxtreme>. For questions specific to Mongoose, you can also get additional help through the Mongoose web portal at <https://www.mongoose.infor/portal>.

If we update this document after the product release, we will post the new version on these web sites. We recommend that you check these web sites periodically for updated documentation.

If you have comments about Infor documentation, contact documentation@infor.com.

Chapter 1: About the Application Event System

1

The Application Event System is a powerful and flexible set of tools designed to help extend and customize how the system works. You can use these tools to define and monitor events across the system.

Basic elements of the Application Event System

In its most basic form, the Application Event System consists of three interrelated elements:

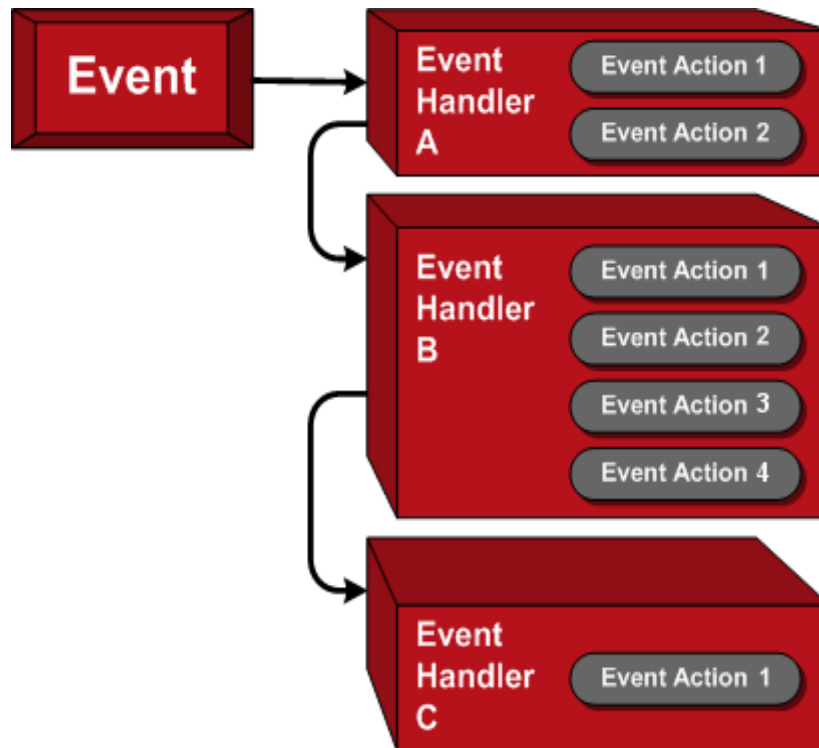
- *Events* are uniquely named incidents that can occur during the use of an application. Events can have multiple triggers and can be generated by user actions, conditions in a database, other events, or other situations.
- *Event handlers* consist of data that specifies:
 - The events to which they are to respond
 - Any conditions, situations, or attributes that determine when and why each handler executes
 - One or more event actions to take place during the handler's execution

Each event can have multiple handlers but each handler can be associated with only one event.

- *Event actions* consist of data that specifies the individual tasks or bits of work that are performed by the event handler. Each event handler must have at least one action and can have multiple actions.

For information about how these basic constituents work together, see Chapter 2, "How the Application Event System Works."

This diagram is an example of one possible set of events, event handlers, and event actions.

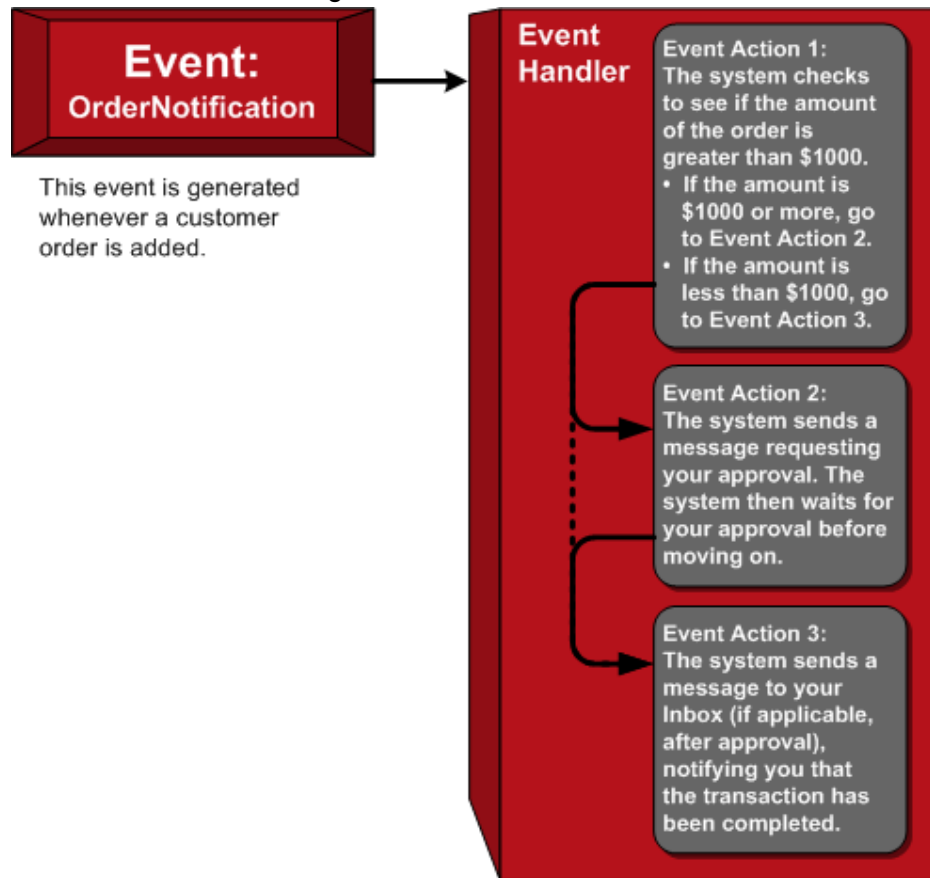


For example, you might want the system to automatically notify you whenever someone adds a customer order to the system and to request your approval if the order is \$1000 or more. In this example:

- The *event* in this case is triggered whenever someone creates a new customer order.
- The event and the situation that tells the *event handler* to run are set up as part of the event handler definition.
- The *event action* that you want to take place is a notification that an order has been placed. You also want that event action to request approval for orders of \$1000 or more.
- Only a single event handler is needed here. But that event handler requires several *actions* to complete all the requirements:
 - The first action is to check the amount of the order and determine, on the basis of the amount, what additional actions are required. If the order is \$1000 or more, then it must direct the flow to another action that requests a manager's approval. If the order is less than \$1000, then it directs the flow to yet another action that simply sends out a notification that the order has been placed.
 - The second action is used only if the order is \$1000 or more. If this action is used, the system generates a prompt message to the manager, requesting approval for the order. The system then waits until the manager responds. If the manager approves the order, the system then proceeds to the next action. If the manager does not approve the order, the event handler fails and no further action is taken. (Though, as good general practice, you should provide for this eventuality as well; but, in the interest of keeping this example simple, we will not provide for the case where the order is not approved.)

-
- The third action is used regardless of the amount of the order. This action sends out a notification to let the manager know that the order has been completed.

Conceptually, the event and associated handlers might look like this:



For more examples, including the processes to create them, see Appendix A, "Sample Scenarios."

Advantages of the Application Event System

Events and event handlers are tagged at the time of their creation with a special identifier (Access As) that prevents them from being modified or deleted by other development organizations. Among other things, this prevents your custom events and handlers from being overwritten when Infor or another developer issues updates to their events and handlers.

The Application Event System provides an infrastructure that allows both the framework and application code to generate events, and for developers and system administrators to create handlers for those events. Event handlers can augment or replace the default framework or application behavior associated with the event. You can design event handlers so that maximal work can be performed without requiring new procedural code.

This is possible because event handlers are defined using metadata. Metadata, in this context, refers to the practice of using uncompiled code and information about data formats that are interpreted during run time, rather than compiled code (called here "procedural code").

For more information about the components and elements of the Application Event System, see Chapter 3, "Designing and Using Events and Handlers."

The Application Event System offers several significant advantages over hard-coded systems:

- The Application Event System provides the power and flexibility to customize the way the system performs *without* having to modify the basic code. Business process are, therefore, "softer" and easier to modify. You can modify application processes and policies without having to directly modify the application code and, in many cases, without having to write any procedural code at all. This means that the amount of procedural code required to implement functionality can be greatly reduced or even eliminated altogether.
- You can use your event handlers with events created by others to gain control of the application flow and take appropriate action, rather than being forced to call into the application using APIs.
- Because event handlers are defined using metadata, there is no upgrade problem, and no collision problem if other developers also have event handlers for the same event.

Examples of ways to use the Application Event System

The subsections below offer examples of how to use the event system, including explanations of how this can benefit you. For more sample scenarios, including the procedures required to implement them, see Appendix A, "Sample Scenarios."

Example 1: Sending a notification when a record is added

Suppose you have a sales manager who wants to be notified whenever someone adds a new customer to the system. You could rely on personnel compliance by anyone who has the ability to add customers to the system and trust them to remember to send the sales manager an email.

You could, however, use the Application Event System instead to automate compliance with this request. You can set up an event that is generated whenever anyone adds a new customer into the system, and use event handlers and actions to automatically generate a notification that is sent to the sales manager.

For a more detailed example, including the procedure to create and use the required handlers, see "Sending notifications" on page 72.

Example 2: Getting approval for a credit limit change

Suppose your company requires that any change to a customer's credit limit be approved by a designated credit manager. You could require any customer representative to contact that credit manager whenever a change to a credit limit is requested.

However, you could instead use the Application Event System to automatically send the credit manager a message requesting approval of the change. To speed up the process, you also send an email to the credit manager with links the manager can click to approve or deny the request.

When the credit manager responds to the message and approves the request, the system can then automatically change the credit limit amount.

For a more detailed example, including the procedure to create and use the required handlers, see "Requesting approvals" on page 88.

Example 3: Complex approval of a purchase order status change

Suppose your company has a business process in place that:

- Requests approval from a purchasing manager when the status of any purchase order (PO) is changed to **Ordered**.
- If the purchasing manager approves the request and the total cost of the PO exceeds a certain amount, requests majority approval from a group of higher level managers on whether the order should be sent to the vendor.
- If those managers also approve and the total cost of the PO exceeds another, higher amount, requests unanimous approval from a group of top-level directors.
- If the request is approved at all the required levels for the total cost of the PO, the transaction is approved and completed.

As with the previous two examples, this process could all be taken care of manually, involving actions by many employees. However, the risk of something getting missed somewhere increases with each approval required.

So, you could use events and handlers to automate this entire process, evaluating the PO at each step of the way, requesting only the necessary approvals, and completing the transaction.

Example 4: Automatically shipping a customer order

Suppose you have a system that, by default, requires someone to manually set the system to ship a customer order line when the status of the line is set to **Filled**. This can cause delays in orders, if/when the responsible employee does not set the order to ship in a timely manner.

You can use the Application Event System to "cut out the middle-man" and automatically set the order to ship as soon as the status is set to **Filled**.

Workflow event handlers

The Application Event System is extremely powerful and flexible. However, for some non-developer users, the system can be somewhat overwhelming at first. To help you with the learning curve, Infor provides a set of predefined workflow event handlers. These handlers are almost ready-to-use; in most cases, all you need to do is define the user names or email addresses of the people who should be notified when a certain event happens, and activate the handler.

For example, if a customer's credit limit is changed, the "Customer Credit Limit Change and Approval" workflow event handler can automatically send an email to the Accounts Receivable manager and request approval of the change. To set this up, just specify the manager's email address and activate the handler on the **Workflow Event Handler Activation** form.

Chapter 2: How the Application Event System Works

2

Events are generated (or fired) in response to an action or condition that occurs in the system. When the event is generated, it can execute one or more event handlers with the event actions associated with each event handler.

How events are handled

Essentially, when an event is generated, it requires an event handler to do something in response. Otherwise, the event serves no practical purpose. That is, if there is no handler for an event, the event actually does nothing when it is generated.

TIP: That does not mean that an event should never exist unless it has an event handler. Infor includes some framework events as part of the system. Infor does not include handlers for these events. These events were created to provide events that other developers can use with their custom event handlers.

When events can be generated

An event can be generated when:

- A system user performs a particular action, perhaps only on a given form and/or when a particular business process is involved.
- A database calculation is performed, perhaps resulting in a certain value.
- Another event results in generating this event.
- A certain amount of time has passed.

These are all examples of situations and conditions that can fire events:

- A sales representative saves a record in the **Customers** form.
- A manager changes the credit hold status of a customer.
- A factory manager adds a new item to the list of those being manufactured in that facility.

-
- The first day of each month arrives. (An event can be used to generate a monthly report, for instance.)
 - The quantity-on-hand of a particular item becomes less than zero.

Where events can be generated

With the Application Event System, events can be generated from any tier.

- In the client tier, the event can be generated by using a form that has a form event handler with a response type of **Generate Application Event**.
- In the middle tier, the event can be generated by invoking the appropriate .NET method.
- In the database tier, the event can be generated by using the appropriate stored procedure.
- In any tier, an event can generate another event by using the GenerateEvent action type.

For details on how to generate an event from any of these locations, see “Firing events” on page 124.

Controlling the sequence of event handlers and actions

Two types of settings control the order in which event handlers and their actions execute:

- Each event handler has a handler sequence number. Handlers execute in the order of their sequence numbers.
To modify the flow and change handler sequence order, you can use the **Keep With** and **Chronology** options on the **Event Handlers** form or the **Event Handler Sequence** form.
For more information, see “About event handling and order” on page 51.
- The event actions associated with each event handler also have their own action sequence numbers. These execute in numeric order, unless:
 - You use the **Initial Action** option on the **Event Handlers** form to designate that a particular action should execute first.
 - You use certain action types to modify the flow.

For more information, see “About event actions” on page 41.

Restricting which handlers run

There might be times when you want or need to disable the event handlers created by one or more development organizations, including yours, at least temporarily. This would occur typically when you

are troubleshooting problems with the event system. The system offers two basic ways to disable event handlers.

Using the **Session Access As** form

When you want to disable (or enable) *all* the event handlers created by a particular developing organization all at once, you can use the **Session Access As** form to accomplish this.

Note: An alternate way to accomplish the same thing for individual handlers with your Access As value is to open the **Event Handler** form and for each event handler that you do not want to include in testing or debugging, clear the **Active** check box.

An example

Suppose, for example, that a customer is having a problem that he suspects is being caused by something he did in the event system, but he is not sure what. He places a call to Infor technical support, and the technical support representative wants to verify that the customer's custom event handlers are not causing the problem.

In this case, the technical support representative might ask the customer to temporarily disable all custom event handlers so that the operation can be tested with only standard functionality in place.

The technical support representative might instruct the customer to use the **Session Access As** form to perform either of these actions:

- In the **Include Access As** field, specify **Core,BaseAppAccessAs** where *BaseAppAccessAs* is the Access As identifier associated with the base application installed on the system.
With this setting, only the Infor core (framework) and base application event handlers will execute. Custom event handlers created by the end-customer do not execute. For more information, see "Session Access As form options" below.
- Leave the **Exclude Access As** field blank, and select the **Exclude Blank Access As** check box.
This option allows all Infor and business partners event handlers to operate. Only the customer's event handlers are ignored. For more information, see "Session Access As form options." below.

Session Access As form options

To disable or enable event handlers using the **Session Access As** form, use any of these options:

Note: You are not obligated to use *both* the **Include Access As** and the **Exclude Access As** fields. You can use any combination of the options available on this form.

- With the **Session Access As** form open, in the **Include Access As** field, specify the Access As identifiers for event handlers that are to be recognized during this session.
To include multiple Access As identifiers, list them separated only by commas (no spaces).
If this field is left blank, the system can recognize and execute all event handlers.

-
- In the **Exclude Access As** field, specify the Access As identifiers for event handlers that are to be ignored during this session.
To exclude multiple Access As identifiers, list them separated only by commas (no spaces).
If this field is left blank, the system can recognize and execute all event handlers. The exception to this occurs only if the **Exclude Blank Access As** check box is selected. In this case, all event handlers are recognized *except* event handlers for which the Access As identifier is null (blank).
 - To exclude those event handlers that have a blank Access As identifier, select the **Exclude Blank Access As** check box.

Using event handler settings

When you want to disable only certain event handlers temporarily, use the **Active** check box on the **Event Handlers** form.

You can only disable (or enable) event handlers that have the same Access As identifier that you have. You cannot, for instance, use this technique to disable Core event handlers.

Synchronicity

Synchronous events

Events are synchronous when they are generated by:

- Core (framework) events
- Calling the FireApplicationEvent() .NET method with the **Synchronous** parameter set to **True**
- Calling the dbo.FireEventSp stored procedure
- Using a form event handler with a response type of **Generate Application Event** and the **Synchronous** option selected
- Using an event action of the type **Generate Event** and a parameter of **Synchronous(True)**

Unless the event is designed to suspend, the expectation is that the event will complete synchronously. Therefore, its synchronous event handlers execute in sequence in the same thread that generated it and block that thread until they have all been executed or until a handler exits with a failure status.

Asynchronous events

Events are asynchronous when they are generated by:

-
- Calling the FireApplicationEvent() .NET method with the **Synchronous** parameter set to **False**
 - Calling the dbo.PostEventSp stored procedure
 - Using a form event handler with a response type of **Generate Application Event** and the **Synchronous** option cleared
 - Using an event action of the type **Generate Event** and a parameter of **Synchronous** with a value of **False**

This means that the event runs in a different thread than the one that posted it, usually on a application server. In this case, the event does not block the generating thread, running independently of it. As soon as an acknowledgment is received that the event was successfully queued, the thread that generated the event continues on.

For more information, see “The Framework Event Service” on page 31.

Event handlers

A *synchronous* event handler is one that must complete before the system continues to the next event handler in a sequence. For the entire event to be handled successfully, all synchronous handlers in the sequence must complete successfully. The exception to this rule is that, if one event handler fails, other than for an illegal operation, and the system is set to ignore failures for that handler, the system can continue to the next synchronous event handler. Otherwise, the system returns a failure error, and no more event handlers in the sequence execute.

By contrast, an *asynchronous* event handler runs independently of other event handlers.

Event handlers can be designated as synchronous or asynchronous at the time they are created, using the **Synchronous** check box on the **Event Handlers** form.

Any event, either synchronous or asynchronous, can execute an asynchronous event handler when execution reaches an event handler designated as an asynchronous event handler; that is, for which the **Synchronous** check box is cleared. At this point:

- The system sends the asynchronous event handler to the event handler queue.
Called queueing, the system does this by means of the PostEventHandlerSp stored procedure, to which it passes the configuration name from the event state.
- If queueing was successful, the event's thread continues on to the next event handler or, if no subsequent handlers are defined, completes the event.
- If queueing was unsuccessful, the event stops with a failure condition.

For a detailed view of synchronicity functionality, see “Summary of synchronous functionality” on page 125.

Suspension

Suspension occurs when a requested operation is sent to the event system for completion at a later time. This occurs when the event handler has the **Suspend** option selected and contains an adjourning action.

Suspension is possible only with certain framework events. You cannot create custom events that can be suspended. Currently, there are only three events that can be suspended:

- `IdoOnItemInsert`
- `IdoOnItemUpdate`
- `IdoOnItemDelete`

Suspension occurs when the system generates an event that:

- Is one of the three framework events that can be suspended.
- Has the **Suspend** check box selected (on the form) for at least one handler that applies to the generated event's object and initiator.

When an event is suspended

When both of the above conditions are met, the system passes control of the requested update (insertion/update/deletion) to the event system. The event system then uses one of the processes in the next two sections to try to make sure the event handlers can all execute successfully before actually committing the system (and the data) to execution of the event.

Suspend-validating mode

When an event is suspended in what is known as *suspend-validating mode*, the system:

- 1 Begins a database transaction.
- 2 Performs an update to the database, to validate pertinent data against any constraints or triggers.
- 3 Validates the data by running all effective event handlers that are synchronous and not suspended in the current transaction.

This involves both application and event system changes.

- At the beginning of this process, in a separate event transaction, the system copies the event handler to a new revision, if necessary (for information about revisions, see “Events and event handler revisions” on page 32.”

The reason for this is that both the *suspend-validating mode* and the *suspend-committing mode* (see the next section) must use the same metadata. However, the validating transaction *will* be rolled back, and the master transaction *might* be rolled back. So, the current event handler revision must be available, in case someone edits the handlers between execution of the two modes.

-
- During this process, a non-output event parameter named **Suspend_ValidatingMode** with a value of **1** is made available to handlers.

TIP: You can test this on any synchronous, non-suspended handlers that you do not want to execute at this time (perhaps because its actions would irreversibly affect something not controlled by our database transaction), using the expression

```
CONDITION( E(Suspend_ValidatingMode)<>1).
```

Note: At this point, the event system treats any failure as an error and exits the execution of the event.

- 4 Rolls back the transaction.
- 5 Places a task on the event queue to run all effective handlers in suspend-committing mode (see the next section).

If the collection update is rolled back due to a later error, this suspension is also rolled back with it, resulting in a state identical to one in which this update was never requested.

When the event service picks up the queued event, it knows that a suspension is in effect by checking the **EventQueue.Suspend** attribute.

- 6 The system marks the record involved in the update/deletion request to have the **InWorkflow** property set to **1**. This prevents any user from attempting another change to the suspended record. Note that suspended insertions do not reach the database at all unless and until the suspended event finishes successfully.
- 7 The thread in which the event was generated continues on as if the change request succeeded.
This occurs even if the thread originated from an event handler executing an `UpdateCollection` or `ExecuteIDORRequest` type event action. In other words, suspension affects only one level of event execution.

Suspend-committing mode

Once the suspended event completes the suspend-validating mode successfully and is placed on the event queue, the system processes the event using these rules. This is known as the *suspend-committing mode*.

- Any failure or error condition prevents further event handlers from executing and prevents the standard processing (that is, the requested insertion, update, or deletion) from occurring. However, the **InWorkflow** attribute is cleared.
- After each event handler finishes successfully, execution proceeds to the next event handler.
- The system queues any asynchronous event handler, and execution proceeds to the next event handler.
- When the last event handler finishes or is queued successfully:
 - The standard processing is committed.
That is, the requested insertion, update, or deletion occurs.
For an update request, this also includes clearing the **InWorkflow** attribute.
 - The event finishes.

When an event is not suspended

When an event that *can* be suspended is *not* suspended (that is, no event handler for that event that applies to that event's object and initiator has the **Suspend** check box selected), the system executes the event using this process:

- 1 The system performs a database update.
- 2 The event system executes all effective event handlers, returning either success or failure.
- 3 The system handles failures as errors and exits the event.

Note: Each insertion/update/deletion request in the process—which could be from an external XML request or from a Save action in WinStudio—participates in the overall transaction. In this case, all complete or nothing completes. So, each handler chain (the same chain executes for each insertion/update/deletion in the group) also participates in that overall transaction. A failure of any of those handler chains results in the entire transaction being rolled back.

Payload

In either of the above cases, an event that can be suspended carries a payload that represents, and that allows handlers to access and modify, the requested change (that is, insertion, update, or deletion).

Each named property value that forms part of the change is passed into and out of the event system as a parameter with a name of the form "Row.*Property*", where *Property* is the name of the IDO property. Modified property values are accompanied by another parameter with a name of the form "Row.*Property*.Modified" and a value of "1".

During notify and prompt actions, these parameters are:

- Temporarily converted to event variables, which overrides any value that might have been set in preceding actions or specified on the **Event Variable Groups** form.
- Available for display (and update, for prompts) on the **Variables** tab of the **Inbox** form (and for display at the end of an external email prompt).
- Then converted back to parameters for subsequent actions.

During other actions such as Set Values, these parameters can be modified to affect the final result of the change when it is applied to the database. This can be done by using this syntax:

- SETPROPVALUES("Property"=*expression*) – If the new value is different than the original framework event parameter received by the event, the PROPERTYMODIFIED(*PropertyName*) framework event parameter is also set to TRUE automatically.
- SETPARMVALUES(Row.Property=*expression*, Row.Property.Modified="1") – Note that you should also indicate to the framework that the property is now modified, as shown.

Adjournment and resumption

Some chains of event actions might need to be temporarily stopped, while waiting for some condition to be met. For instance, a customer order might need a manager's approval before it can be entered into the system. In this case, the system temporarily halts the execution of the event handler chain until the condition is met; for instance, until the manager responds with an approval (or not). This is known as adjournment.

Certain event actions, called *adjourning actions*, must wait for an external stimulus before the event handler can proceed with the next action. An event handler containing such an action must be either:

- Asynchronous
OR
- Part of a framework event that can be suspended and is marked to suspend (see "Suspension" on page 22)

When execution reaches an adjourning action, the event handler state is set to retest or time out at a specified time. At this point, the event becomes an adjourned event.

The event service then processes it at the next opportunity after the **Time Out** and/or **Retest** time. This is called *resumption*.

Success, failure, and retries

Each event handler can exit with a status of **Success** or **Failure**. When an event handler fails, if it is not set to ignore failures, and if it has not attempted an illegal operation, the system can retry the event handler's actions.

Success

An event handler exits with a status of **Success** only when it either:

- Reaches an event action of the type **Finish**.
OR
- Completes all event actions successfully without reaching an event action of type **Fail**.

Failure

An event handler exits with a status of **Failure** when it does any of these:

- Reaches an event action of the type **Fail**.
- Attempts an illegal operation.

-
- Experiences an unexpected failure of an event action or something launched by an event action.

When failures occur

When a failure occurs:

- The **Event Handler Status** form's **Result** field displays an appropriate error message, which includes an error message passed from a failing stored procedure in a message type parameter (@Infobar), where applicable.
- The current event action state's **Times Failed** counter is incremented.
This value displays on the **Actions** tab of the **Event Handler Status** form.
- The event handler state's **Last Activity Date** is set to the date and time of the failure.
This value displays on the **Handlers** tab of the **Event Handler Status** form.
- Any event variables are maintained at their current values for inspection on the **Variables** tab of the **Event Handler Status** form.

Ignoring failures

By default, if any synchronous event handler reports a failure, the system skips any remaining event handlers for that event and exits with a status of **Failure**.

You can override this default behavior by using the **Ignore Failure** option for the event handler (on the **Event Handlers** form). If you select this option, the event handler is always treated as successful unless it deliberately attempts an illegal operation. The **Status** field on the **Event Handler Status** form, however, still shows a status of **Failure**, even though the **Any Handlers Failed** check box remains cleared.

This table shows what conditions can result in failures for various action types and whether the failure can be ignored.

Action type for the current event action	Condition	Ignore Failure option selected?	Result (Event Status)
Set Values	Illegal operation: Any attempt to set a non-output event parameter	N/A	Failure
Some	Illegal operation: Any attempt to use an action type that is not supported on the current tier	N/A	Failure
Any	Illegal operation: Syntax error in parameter	N/A	Failure
Generate Event	Illegal operation: Any attempt to generate a framework event	N/A	Failure

Action type for the current event action	Condition	Ignore Failure option selected?	Result (Event Status)
Any	Unexpected error	No	Failure
		Yes	Success

A Fail action whose condition evaluates to **True** on an event handler for which the **Ignore Failure** option is selected is equivalent to a Finish action.

An asynchronous event handler's failure does not affect any remaining event handlers for that event. However, setting the **Ignore Failure** option in this case avoids affecting the AnyHandlersFailed() attribute of that event, unless the failure was caused by deliberately attempting an illegal operation, as shown in the table above.

Retrying event handlers

In the **Event Status** and **Event Handler Status** forms, you can retry a failed event handler, if it is retryable. Whether a handler can be retried is determined by the cause of the failure and whether the event handler is marked transactional, as shown in this table:

Cause of Failure	Event transactional	Handler transactional	Retryable	Retry point	Event Variables
Illegal operation	N/A	N/A	No	N/A	N/A
Any legal operation	Yes	N/A	No	N/A	N/A
Any legal operation	No	Yes	Yes	InitialAction	Re-initialized
Unconditional Fail Event action	No	No	Yes	InitialAction	Re-initialized
Conditional Fail Event action	No	No	Yes	CurrentAction	Maintained
Unexpected error	No	No	Yes	CurrentAction	Maintained

When retrying a single handler, whether the handler runs synchronously or asynchronously depends on the parameter passed to the API, not the handler's definition. When retrying an event, the handlers are run as shown in this table:

	Failed Synch Handler	Failed Asynch Handler
Restarted synchronously	Synchronous	Asynchronous
Restarted asynchronously	Asynchronous	Asynchronous

An event can have a synchronous handler that is not retryable, but it still retries any asynchronous handlers that can be retried. In this case, any synchronous handlers that would have come after the failed handler do not run.

The Event or EventHandler Result must be cleared when retrying the handler. However, the previous handler result is saved in a handler variable called PreviousResult, and the event results are saved in an event parameter also called PreviousResult.

Transactions

By definition, a *transactional* event is an event or set of event handlers in which all must complete successfully before any data is committed in the system. If any handler fails, then the system rolls back (reverts) all data to its initial state.

If the transactional event contains any adjourning event actions, the system modifies this behavior somewhat: Any actions taken up to the first adjournment, between each resumption of an adjournment and the beginning of the next adjournment, and after the last resumption, are each committed separately.

The system treats any asynchronous event handlers as if they belong to an asynchronously generated non-transactional event. In other words, because they are outside the synchronous flow, these event handlers are not treated as part of the transaction.

For the specifications of synchronously and asynchronously generated events, see the table for "Summary of synchronous functionality" on page 125.

Transactions with synchronous events

An event generated synchronously without the **Transactional** check box selected (on the **Event Handlers** form) either:

- Runs in the current transaction state of the firing thread (if no lower-level transactioning is specified).
- Handles transactions at the event handler or event action level, as explained in these points:
 - In the database tier, this occurs naturally with no special syntax.

When FireEventSp is called within a SQL transaction, any changes to the non-event system portions of the database (using Audit, Call Database Method, and Run Background Task event actions) become a part of that transaction, which can be rolled back after FireEventSp returns.

When FireEventSp is called outside a SQL transaction, each event action that might change non-event system portions of the database is wrapped in its own transaction, which is rolled back in case of failure, except when the containing event handler is marked **Transactional**. Changes to event system state data are performed on a separate connection, so they survive any rollback of the encompassing or wrapping transactions.

- In the middle tier, any execution methods for database-related event actions must enlist in an existing transaction or create a new one.

To include the entire event in a single transaction, call the FireApplicationEvent method from a hand-coded IDO method that is marked **Transactional** in the IDO metadata. In this case, the execution methods for the database-related event action enlist in the transaction created by the IDO Runtime for that transactional IDO method.

When the FireApplicationEvent method is called from a hand-coded IDO method that is *not* marked **Transactional** and whose caller is not in a transaction state, each event action execution method creates a new transaction, which is rolled back in case of failure, except when the containing event handler is marked **Transactional**. Changes to event system state data are performed in a separate connection (except in Suspend-validating mode), so they survive any rollback of the encompassing or wrapping transactions.

- The client is not permitted to control transactions that span form event handler calls. Client-tier event-firing requests are passed directly on to the MGCore.Events.FireEvent IDO method.

A failure of a synchronous event returns an error condition to the firing thread. For framework (Core) events, this causes the entire current transaction to be rolled back. For application events, whatever generated the event (stored procedure/trigger, .NET method, or VB.NET script) is responsible for trapping the error condition, accomplishing a rollback of the current transaction, and throwing execution to an appropriate recovery point in or out of that code.

Event handlers marked as transactional

An event handler marked **Transactional** is wrapped in:

- A new SQL transaction when executed in the database tier.
- A new transaction when executed from the middle tier, if no encompassing transaction is active at the trigger point of the outermost enclosing event.

This event handler cannot contain any *adjourning* event action types (for example, Prompt, Wait, or Sleep), but it can be marked **Asynchronous** itself. If this event handler ends in a failure, the wrapping transaction is rolled back (assuming it was created).

Event handlers not marked as transactional

An event generated asynchronously without the **Transactional** setting does not run in a transaction state. Event handlers that are not marked **Transactional**, for a synchronous event generated from a middle-tier non-transactional custom IDO method, are handled this way:

- An event can be generated asynchronously with a **Transactional** setting that indicates that a new transaction is begun and the entire database-related effects of its synchronous event handlers are committed if they all succeed or rolled back if any fail.
- Any asynchronous event handlers are treated as if they belong to an asynchronously generated non-transactional event.

Rolling back transactions

These conditions cause the system to set the **Transactional** setting on a new event state row:

- FireEventSp or FireApplicationEvent() detects an enclosing transaction.
- FireEventSp, FireApplicationEvent(), PostEventSp, or PostEvent() receives a **Transactional** setting.

This event state setting signals the system administrator that when this event state is finished and not rolled back, everything happened that was expected to happen.

In a synchronous multi-event situation, to encode the proper information in the **Rolled Back** setting, an enclosing transaction that contains one or more FireEventSp or FireApplicationEvent() calls must:

- 1 Once, before calling FireEventSp or FireApplicationEvent(), call EventBeginTransactionSp or EventBeginTransaction().

This uses a separate transaction to clear a storage area to record the row pointers of the event state rows to be created and returns a transaction identifier to be passed to FireEventSp or FireApplicationEvent().

- 2 Pass that transaction identifier to FireEventSp or FireApplicationEvent().

This, in a separate transaction, records the row pointer of the new event state row.

- 3 Upon rolling back or committing the transaction (it does not matter whether before or after), call EventEndTransactionSp or EventEndTransaction() and pass the transaction identifier and whether a commit or rollback will be/was performed.

This uses a separate transaction to set the **Rolled Back** setting on the stored event state rows (if rolling back), and clears the storage area.

If this is neglected, the system administrator cannot determine why database actions that appear to have finished successfully, according to the status forms, are not reflected in the database.

In asynchronous situations, and when a failure is detected by the event system, the **Rolled Back** setting for the failing transactional event is set by the event system.

The Framework Event Service

The Framework Event Service is an independent process that can run on any application server. Each instance of the Framework Event Service monitors the event queue for any number of configurations defined locally.

For load-balancing purposes, this service safely acquires work from the database queue of each configuration's application, so that multiple event services can run on one or multiple servers. The event service monitors the event queue for:

- New events that have been queued by means of asynchronous generation methods or suspending framework events, as found in the Queued Event table.
- New event handlers that have been queued by means of the firing of asynchronous event handlers, as found in the Queued Event Handler table.

Setting up the Framework Event Service

To set up the Framework Event Service to monitor specified configurations, you must specify each configuration individually on the **Event Service** tab of the **Service Configuration Manager** utility.

Note: If you do not do this setup, any event handler you create that requires the event service will not work until you do.

For more information, including the procedure, see the online help for the **Service Configuration Manager** utility.

If you want to use the IDO Runtime Development Server (IDORuntimeHost.exe) in your development work, you must also temporarily remove the dependency that the Event Service has on the IDO Runtime service. For more information, including the procedure to do this, see the online help for WinStudio developers: From the **Help** menu, select **Customizing Forms**; then, when the help window opens, select the topic "Running IDO Runtime Development Server".

Processing order in the Framework Event Service

The Framework Event Service processes any queued events in "first in, first out" (FIFO) order.

Because the event service could receive a request to run something while it is executing a prior request, all requests are queued for execution in the order requested. If the new request happens to be the only one in the queue and the event service is not busy, the request is executed at the next polling.

When the event queue is empty, and immediately after processing each queued event or event handler, the event service checks the Event Handler State and Event Trigger tables for any items whose **Retest At** time setting has arrived or passed.

The event service then checks the Event Handler State table for any items for which the timeout has expired, that is, the **Times Out At** time setting has arrived or passed. The event service processes

these in order (that is, oldest first), using the **Retest At** time or **Times Out At** time to determine the order.

After this, the event service checks the event queue again for any incoming items.

Administrative details

To eliminate the possibility of deadlocks when manipulating event data, a second database connection is used for event data modifications, that is, separate from the one used by the event action execution methods. Also, any event data modifications that do not need to be available to other processes—that is, any tier of any interactive user session or other event service instances—are made in memory and written only when required.

To avoid overtaxing the application server, the Framework Event Service administrator has control over the minimum interval for these attributes:

- **RetestInterval** parameter on Wait event actions
- **Interval** parameter on Sleep event actions
- **RetestInterval** attributes on event triggers

When any of these attributes is referenced, if the designer-specified value is greater than zero but lower than the minimum interval for the Framework Event Service, the minimum value is used.

The Framework Event Service logs various levels of messages to the Infor framework **Log Monitor**.

Events and event handler revisions

The system creates a set of *event handler revisions* the first time:

- The event is generated or any of its handlers executes.
- The event is generated after any of its constituent handlers or their actions has been modified in any way.
- Any of the event's handlers executes after any of its constituent actions has been modified in any way.

Modifications can include additions, changes, or deletions to the handlers or actions associated with the event or handler.

When the revision is created, the system copies all of the event's handlers and actions to a read-only table. The event or handler then uses the data in this read-only table when executing the handlers and actions, until a new revision is created.

Revisions are used to help ensure that the metadata used by an event and its handlers and actions is not altered while the event is executing. In some cases, it can take the system a period of time to finish executing an event's handlers, either because of the processing time involved, or because the system is waiting for some input or response before it can continue. It is possible that someone could

make changes to the event's handlers and actions while the event is processing or waiting, and that these changes could affect the execution of the event that is in progress.

Revisions were developed to address this kind of potential problem. When an event fires, the event and its handlers use the revisions that are in effect at the time they start executing until they are finished.

Example:

Suppose you are using an event to send notices to a manager when a customer order is more than \$10,000. The manager must approve the order before it can be processed.

During a transfer of responsibilities from one manager to another, a new manager is assigned the responsibility of approving such orders. The system administrator makes the change of notice to the appropriate event action.

However, the original manager still has a few orders pending approval. These orders continue to use the metadata for the revision in effect at the time they were created, awaiting that manager's approval. In the meantime, the new manager receives notices for any subsequent orders, because the first new order generates a new revision using the information for the new manager.

Infor has built in to the system a number of events and handlers that are available for immediate use. In addition, if you have an add-on product distributed by one of Infor's business partners, they might have also added their own events and handlers that you can use.

You can also design and create your own custom events and handlers to automate tasks for your particular needs. This chapter provides information about the key elements of the Application Event System and the procedures for creating and using custom events and handlers.

Note: If you are creating and using your own custom events and handlers, we recommend that you refresh the metadata cache periodically. This should be done after doing development work, before testing, and after synchronizing your metadata on your system. For more information, see "Refreshing the metadata cache" on page 54.

Elements of the Application Event System

This section identifies and describes the key elements of the Application Event System. Where applicable, procedures for creating and using various elements are also provided.

About the Access As identifier

One key element of the Application Event System is the Access As identifier. The Access As identifier is used to:

- Indicate who (what organization) created and "owns" an event or related event system object.
- Prevent unauthorized developers from modifying or deleting event system objects that they do not own.

The Access As identifier is also used to indicate ownership and modification rights for certain IDO metadata. It might also be used in the future for other, similar metadata.

Generally, the Access As identifier falls into one of three classifications:

-
- **Core** – Indicates that the object is one that Infor created and owns. These event system objects are used for the framework forms and operations.
 - Any other name – Indicates that the object was created by and belongs to an application created either by Infor or by one of Infor's business partners or other authorized vendors.
 - Blank – Indicates that the object was created by and belongs to the end customer.

Within WinStudio, several forms include an **Access As** field. On the **Access As** form, this field indicates the "current Access As" value. This is the value assigned to any new event system objects you might create. On all other forms, this field indicates who has ownership of the pertinent object metadata, in other words, who created and owns it.

You can only modify or delete metadata for event system objects that have the same value as on the **Access As** form, in other words, event system objects that your organization has created and owns.

Note: If you are an Infor business partner or authorized vendor developer, and you need to change the Access As identifier, contact your Infor Consulting Services (ICS) representative for help.

With a few exceptions (noted where applicable), you can attach your own event triggers and handlers to event system objects owned by other organizations (that is, with a different Access As identifier than yours), but you cannot directly modify or delete those event system objects.

About events

An *event* is defined as a uniquely named situation that can be triggered by:

- An action performed by somebody working in the system.
- A particular condition that occurs while the system is running.
- A certain value that is exceeded in a database record.
- Another event's handler.
- Other, similar occurrences.

Event types

Events can be one of three general types:

- Core, or framework, events – These are events that Infor has defined and built in to the system. They are tagged with an Access As identifier of **Core**.

These events generally fall into one of two categories:

- IDO (business process-related) events that are generated when certain IDOs are invoked. These IDOs include `IdoOnItemInsert`, `IdoOnLoadCollection`, `IdoOnInvoke`, and others. You can identify these events easily by their names, which begin with the letters **Ido**.
- Session events that are generated when certain session activities take place. These include `SessionOnLogin`, `SessionOnLogout`, and `SessionOnVarChanged`.

These events are always synchronous (see “Synchronicity” on page 20) and transactional (see “Transactions” on page 28). Some can be optionally suspended to await user responses (see “Suspension” on page 22).

- Application-specific events – These are events that typically have been created by Infor, its business partners, and authorized vendors. They are tagged with an Access As identifier that indicates what application or development organization they belong to.
- Customer-defined events – These are events that a developer in an end-customer organization has created. They are tagged with a blank Access As identifier, which indicates that they were created by and belong to the customer.

For more information about Access As identifiers, see “About the Access As identifier” on page 35.

Defining events

Events can be defined (named) on these forms:

- **Events**
- **Event Triggers**
- **Event Handlers**

To define an event, specify the name of the event in the **Event Name** field of one of these forms.

Note: If you do not name the event on the **Events** form, it is still available to the drop-down lists on other forms. Events named on those forms, however, do not display on the **Events** form. So, if you want the event to display on the **Events** form, you should name the event on that form.

When an event is defined, or named, it is really just that: a name. Until you define a way for it to be triggered or initiated, the event remains just a name. This can be done using either the **Event Triggers** form, the **Event Handlers** form, or both. For the means of defining ways to trigger custom events, see “About event triggers” on page 38.

For more information about any of these forms, see the online help for that form.

Modifying events

Once an event has been created and saved, the only thing you can modify is the event's description. The event name and other attributes are locked.

Note: You can modify an event's description only if the **Access As** field has the value of the current Access As value, as displayed on the **Access As** form.

To modify an event's description:

- 1 Open the **Events** form and select the event you want to modify.
- 2 In the **Description** field, modify the description text as desired.
- 3 Save.

Deleting events

If you are certain you no longer need an event and you want to delete it, you can.

You can delete an event only if the **Access As** field has the value of the current Access As value, as displayed on the **Access As** form.

To delete an event:

- 1 Open the **Events** form and select the event you want to delete.
- 2 From the **Actions** menu, select **Delete**.
- 3 Save.

About event triggers

An *event trigger* is defined as a condition that causes a particular event to fire, more-or-less independent of anything that might be happening in the user interface. The event trigger carries a set of event trigger parameters for use when the event fires.

An event trigger can be set to fire the event only once, or it can be set to retest for its condition after waiting a certain amount of time since either of these situations was true:

- The trigger last successfully fired the event.
- The trigger last tested unsuccessfully for its condition.

In both cases, the event trigger's designer can set the interval to wait, both for the successful firing of the trigger and for the unsuccessful test for the trigger conditions (using separate settings). Testing and retesting is accomplished by means of polling; this is not a true interruptive trigger.

An event trigger carries with it the user name and configuration in effect at the time it was defined. This data is passed on to the event state when the trigger fires the event.

Defining event triggers

Event triggers are defined using the **Event Triggers** form. Use this form to determine the condition that will fire the event, set the parameters to be passed to the event when it fires, and specify retest intervals.

To create an event trigger:

- 1 Open the **Event Triggers** form.
- 2 Press F3.
- 3 From the **Actions** menu, select **New**.
- 4 From the **Event Name** drop-down list, select the event for which you want to define a trigger.

Note: You cannot define a trigger for a framework (**Core**) event.

- 5 On the **Trigger** tab, in the **Condition** field, specify the condition at which the event is to fire.

For more information about defining conditions, see “Triggers and conditions” on page 39.

- 6 On the **Parameters** tab, specify the name and values for any event parameters for which you need to pass values to the event handlers when the event fires.
- 7 Set other options on this form as desired.

For more information on other form options see the online help for each field.

- 8 Save the form.

Triggers and conditions

Each event trigger must contain a condition consisting of one of these:

- A Boolean expression
- Two non-Boolean expressions separated by a comparison operator

Examples:

- This example causes the event to fire when seven days have elapsed since the current result of the database function `dbo.LastEntryDate()`:

```
DATEDIFF(day, DBFUNCTION("LastEntryDate"), CURDATETIME()) > 7
```

- This example causes the event to fire when the balance on a certain customer's order is greater than \$10,000:

```
DBFUNCTION("OrderBalance", GC(BigCustNum)) > 10000
```

- This example causes the event to fire on the first day of each month:

```
DATEPART(day, CURDATETIME()) = 1
```

Note: The condition should generally involve either a time operation or a database calculation, or both. This is because time and the database are the only known factors that can undergo change from external stimuli (that is, by the forward movement of time or by the actions of other application users, respectively).

Retesting triggers

When the event trigger's **Retest At Date** setting becomes older than the current system clock time, the event trigger is available to be processed by the event service. This happens when the event service is free from processing any waiting queued events and handlers and any already-waiting triggers that need to be tested or retested.

To process the event trigger, the system parses the condition, evaluates it, and, if the condition evaluates to True, then the event fires. At that point, the **Retest At Date** setting is set to the current time plus the amount of time set for the **Trigger Reset Interval**. If the **Trigger Reset Interval** is set to 0 (zero), then the **Active** indicator is cleared, which indicates that the trigger is not to be retested.

If the condition evaluates to False, then the **Retest At Date** setting is set to the current time plus the amount of time set for the **Condition Retest Interval**. If the **Condition Retest Interval** is set to 0 (zero), then the **Active** indicator is cleared, which indicates that the trigger is not to be retested.

About event handlers

An *event handler* defines the actions to be taken upon the firing of a particular event. Each event handler is comprised of one or more event actions and, optionally, an initial state.

Each event can have multiple event handlers that execute when the event fires. In such cases, the handler sequence number and other factors determine the order in which event handlers are actually processed.

For more information about event actions, see “About event actions” on page 41.

For more information about initial states, see “About event variables and initial states” on page 45.

Defining event handlers

Each event handler is uniquely defined in the system by the combination of an event name and a handler sequence number. Both of these are set on the **Event Handlers** form. Event handlers also must have one or more associated event actions. For more information, see “About event actions” on page 41.

To create an event handler:

- 1 Open the **Event Handlers** form.
- 2 Press F3.
- 3 From the **Actions** menu, select **New**.
- 4 In the **Event Name** field:
 - Select the event for which you want to create a handler.
This sets up the event handler for an already-defined event.
 - Specify the name for an event which has not been previously defined.
This effectively defines a new event as well.
- 5 (Optional) To control the order in which the event handler executes (especially with respect to existing event handlers), use the **Keep With** and **Chronology** fields, or the **Event Handler Sequence** form. For more information, see “About event handling and order” on page 51.
- 6 Save the new event handler.
- 7 Use the **Event Actions** button to open the **Event Actions** form and create the actions to be performed when this event handler is executed.

For more information, including the procedure, see “About event actions” on page 41.

- 8 After closing the **Event Actions** form, set the other options on the **Event Handlers** form as desired.

An event handler can be restricted to execute only in relation to a specific set of conditions. For example, a particular event handler might be defined to execute only when the associated event is triggered by an action on a particular form or when a particular IDO is involved.

An event handler can also be set to execute synchronously or asynchronously, or as part of a transactional event or set of event handlers. For more information, see “Synchronicity” on

page 20 or “Transactions” on page 28.

For information about the other options, see the online help for each option.

- 9 Save the event handler.

About event actions

An *event action* is defined as a unit of work to be performed during the execution of an event handler.

A single event handler can have multiple event actions, but each action is assigned to a single event handler.

Depending on its action type, an event action can do such things as:

- Evaluate and compare expressions, using the results to select which event action of its event handler to perform next.
- Affect the event's visual state.
- Complete the event handler.
- Set event variables.
- Call methods or web services.
- Perform other predefined tasks.

Defining event actions

To define new event actions, open the **Event Actions** form using the **Event Actions** button on the **Event Handlers** form. If you open the **Event Actions** form from the Explorer or the **Select Form** dialog box, you can only view and modify existing actions.

To create an event action:

- 1 Open the **Event Handlers** form.
- 2 Create a new event handler; or, in the grid view, select the event handler for which you want to create the action.

Note: If you are creating a new handler, you must save it before you can do the next step.

- 3 Click the **Event Actions** button.
- 4 (If there are no existing event actions) From the **Actions** menu, select **New**.
- 5 In the **Action Sequence** field, specify a number.

This number determines the order in which actions for a particular handler are processed.

- 6 In the **Event Actions** form, **Action Type** field, specify the type of action to be performed.
- For a complete list of action types and what they do, see “Event Action Types” on page 139.

-
- 7 To define the parameters for the action, click **Edit Parameters** and use the associated event actions parameter form.

For more information about defining parameters, see “About event action parameters” on page 42.

- 8 To verify that the syntax is correct, click **Check Syntax**.

If you have any syntax errors, fix them before proceeding.

- 9 If the action involves a variable to be used in event messages and you want to restrict the variable's accessibility on the target form, set those restrictions on the **Variable Access** tab.

For more information about setting variable access, see the online help for the **Variable Access** tab.

- 10 Save your changes and close the form.

About event action parameters

Depending on the action type, you also specify optional parameters for each event action type. You can specify parameters in any order for a particular action. To list multiple parameters, specify them one after another, specifying either a space or nothing between them. Event action parameters are defined on the **Parameters** tab of the **Event Actions** form.

TIP: You can use the event action parameter forms to define the parameters. When you do, the parameters are returned to the **Event Actions** form properly formatted and free of syntax errors. For more information, see “About event action parameter forms” on page 44.

The syntax for each event action parameter is as follows:

FUNCTION(*value*)

TIP: Although it is not a requirement that function names be specified using all uppercase letters, we recommend the practice, as it leads to greater ease of recognition and readability.

The value enclosed in parentheses can consist of:

- A constant number.
- A literal string enclosed in quotation marks.
- A Boolean value: TRUE or FALSE.
- An event function call.

These can be nested. For more information, see “Nesting function calls” on page 43.

- An expression consisting of a number of these elements combined using operators.

You can also use the parentheses following the function to wrap expressions that signify operations to be performed on the results of the expression.

For example, the function `√` takes as a parameter the name of a variable. This function can be placed in the parameters for other functions, as in this example:

`METHOD ( V ( FuncNameVar ) )`

For more information about event action parameters, including a list of all acceptable parameters, their meanings, and examples, see “Event Action Parameters” on page 147.

For the complete expression grammar for constructing event action parameters, see Appendix C, “Expression Grammar.”

Function types

Functions can be any of three basic types:

- *Parameter* functions — These are functions whose parentheses wrap a "parameter" to the event action. For example, these are all typical parameter functions:
 - SETVARVALUES
 - METHOD
 - INTERVAL
 - EVENTNAME

These functions must always appear at the "root" level and can never be nested inside any other type of function.

These functions are identified in this documentation generically as PARAMS(...).

- *Value* functions — These are functions that call event values such as:
 - SUBSTITUTE
 - DATE
 - ABS
 - CEILING

These types of functions can never appear at the "root" level but must be nested within another function construct (either a parameter or another function).

These functions are identified in this documentation generically as FUNCTION(...).

- *Word* functions — These are verbatim words used inside certain event function calls, such as:
 - AS
 - STRING
 - NUMBER
 - DAY
 - DATE

Note that some of these can also be used as event function calls. These functions always appear within an event function call, however.

These functions are identified in this documentation generically as ...WORD...

Nesting function calls

When defining event action parameters, keep these rules (as specified in the previous section) in mind:

- PARAMS(...) -type functions can never be nested.
- FUNCTION(...) -type functions *must be* nested and can be nested either within PARAMS(...) functions or within other FUNCTION(...) functions.
- ...WORD... -type functions must be nested within FUNCTION(...) -type functions.

This is an example of an event action parameter that uses nested functions:

```
PARAMS(...FUNCTION1(...FUNCTION2(...WORD1...)...WORD2...)...)
```

Passing parameters from actions

Parameter lists to methods, scripts, Web services, and generated events are always enclosed in a PARAMS(...) function and delimited by commas. Each parameter is specified in one of these ways:

Syntax	Direction	Meaning
V(<i>var</i>)	Input	Pass in the value of the variable <i>var</i> .
<i>expression</i>	Input	Pass in the value obtained by evaluating the expression.
RV(<i>var</i>)	Input and output	Pass in the value of the variable <i>var</i> , and place the output value into the same variable.

Setting variable and parameter values

You can set values for event variables and parameters using these syntaxes:

Event action type	Type of storage	Syntax
Call Database Method	Variable	PARMS(RV(<i>var</i>))
Call IDO Method	Parameter	PARMS(RE(<i>param</i>))
Generate Event	Variable	SET(RV(<i>var</i>)= <i>name</i>)
Load IDO Row	Parameter	SET(RE(<i>param</i>)= <i>name</i>)
Set Values	Variable	SETVARVALUES(<i>var=expr</i>)
	Parameter	SETPARMVALUES(<i>param=expr</i>)

About event action parameter forms

To make it easier to create event action parameters that are properly defined and formatted, each event action type has a corresponding event action parameter form. These forms allow you to build appropriate sets of parameters for each action type by providing options that pertain to that action type.

For example, if you are creating an event action of the type Notify, when you open the associated event action parameter form, you see something like this:

The screenshot shows a modal window titled "EVENT ACTION NOTIFY (MODAL)". On the left, there is a vertical list of buttons: "Condition", "To", "Cc", "Subject", "Category", "Body", "HTML Body", and "Attach". Each button is positioned to the left of a corresponding text input field. The "Body" and "HTML Body" fields are significantly larger than the others and include vertical scrollbars. To the right of the main input area, there are two more input fields labeled "Filter Form" and "Filter Spec". Below these, there are three rows of controls: "Payload Access: Default", "Modified Payload Access: Default", and "Unmodified Payload Access: Default", where "Default" appears to be a button or a selectable value. At the bottom left, there is a checkbox labeled "Save In Sent Items". At the bottom right, there are "OK" and "Cancel" buttons.

Notice that each editable text field is labeled with a button on the left. This is a fairly typical pattern for event action parameter forms. With each button/field pair, you have the option to either specify the value for that parameter in the field or click the button. Clicking the button opens another form that pertains to the parameter associated with that button and that allows you to build just that parameter.

Some options are presented using combo boxes that also have drop-down lists from which you can select appropriate values for that field. Some options are not accompanied by buttons at all. Other options are selected (or not) using check boxes, as with the **Save in Sent Items** in the foregoing example.

Very few options on any given form are considered required. As a rule, only those fields that you populate return parameter values to the **Event Actions** form. Where strings require quotation marks, the system inserts them automatically. Where expressions are nested inside other expressions or parameter statements, the system automatically places parentheses where they are required.

The result is that, by using the event action parameter forms, you can eliminate most, or more likely all, syntax errors in your parameter statements.

About event variables and initial states

If an event handler has variables associated with it, those variables can be assigned an *initial state*, that is, a set of values they are assigned when the event handler starts to execute. Each initial state consists of:

- A name that identifies it in the system
- Any number of event variables with the initial values they are to have when the event handler starts to execute

For example, you might want to use with your event handlers variables with these initial values:

Name of variable	Initial value	Comments
Increment	250	—
ItemTypes	AB	—
TimesRemaining	GC(MaxTimes)	In this case, the value is determined by the value of the MaxTimes global constant.

Initial states are defined using the **Event Variable Groups** form. Once created, you can use a defined initial state with any other event handler by selecting it in the **Initial State** field on the **Event Handlers** form.

About event global constants

An event *global constant* is defined as a named static value that event expressions can reference during processing of the associated event handlers.

An example

For example, you could use a global constant to create a list of managers who are authorized to control customer credit limits. You could then reference this global constant whenever you create an event handler that sends a notification (Notify event action) regarding a customer's credit limit. This practice allows you to use the list for multiple event handlers without having to hard-code the list of names in each event handler's actions. It also allows you to change the list in one place and have that one change affect every event handler that uses it.

For more examples of how global constants can be used, see Appendix A, "Sample Scenarios."

Defining and using event global constants

Event global constants are defined using the **Event Global Constants** form.

The system references a global constant using a function mechanism that allows dynamic evaluation at each reference.

Event global constants can be especially useful when defining a set of choices to offer the recipients of a prompt message. For example, a global constant named **PromptChoicesYesNo** can be defined with the value **1,sYes,0,sNo**. You can then reference this constant for any prompt event action, using the expression:

Application Event System design forms

The system includes a set of specialized forms created to enable you to create and use your own custom application events. With the exception of the **Access As** form and the **System Configuration Parameters** form in this table, these forms are located in the Explorer under **Master Explorer > System > Event System**. The **Access As** and **System Configuration Parameters** forms are located one level up, at **Master Explorer > System**.

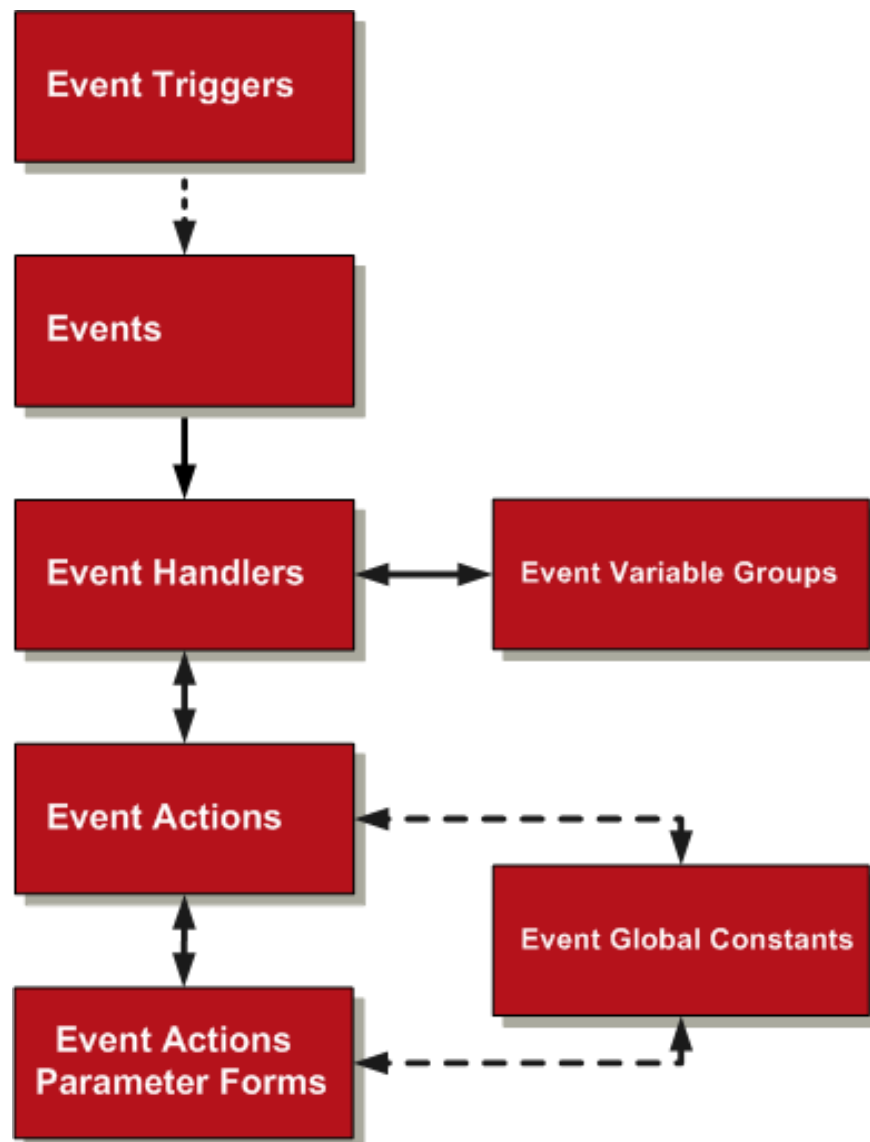
Access to these forms is controlled by the System Administration authorization group.

This table lists and briefly describes the use of the event system design forms.

Form name	Description/Use
Access As	Although not directly used in the creation and customization of application events and handlers, this form displays the current Access As setting. This, in turn, is an indicator of which event system elements you are authorized to modify and/or delete. For more information about the Access As setting, see "About the Access As identifier" on page 35.
Events	Used to name events. Once named here, events are available on other forms as well, particularly the Event Triggers and Event Handlers forms. For more information, see "About events" on page 36.
Event Triggers	Used to define event triggers, which set conditions that cause a named event to fire. For more information, see "About event triggers" on page 38.
Event Handlers	Used to display and define event handlers, which determine the work to be done when an event fires. For more information, see "About event handlers" on page 40.
Event Handler Diagram	Used to present a graphical representation of an event handler flow. You can also use this form to access the Event Actions form for selected event actions, to view or modify them. Finally, you can also add event actions to an event handler flow using this form. For more information, see the online help topic "Using the Event Handler Diagram Form."
Event Handler Sequence	Used to change the order of any handlers that have the current Access As identifier (as indicated on the Access As form). For more information, see the online help for this form.

Form name	Description/Use
Event Actions	Used to display and define the actions to be performed by a particular event handler during its execution. These are the individual tasks accomplished by the event handlers. For more information, see “About event actions” on page 41.
Event action parameter forms	Used to define the event action parameters for each action type. For more information, see “About event action parameter forms” on page 44.
Event Variable Groups	Used to display and define initial states, which are sets of event variables and the initial values they are to pass to the event handler when it starts to run. For more information, see “About event variables and initial states” on page 45.
Event Global Constants	Used to display and define event global constants, which are static values that can be accessed and used by expressions during the running of an event handler. For more information, see “About event global constants” on page 46.
System Configuration Parameters	Used to define certain parameters that affect the system configuration. Although not directly used in the creation and customization of application events and handlers, you can use this form to control some aspects of how events and handlers behave on the system. These aspects are concerned mostly with retest and reset intervals that globally govern when and how often various conditions can be retested or events can retry. For more information, see the online help for this form.
Workflow Event Handler Activation	Used to activate predefined event handlers that represent common workflows. You must specify some information, such as users or email addresses that will be notified when an event occurs. You can also copy and modify these event handlers. For more information, see the online help for this form.

This diagram shows the functional relationship between the design forms and elements for the Application Event System.



Setting up custom events and handlers

To create and use your own custom application events, there are several important steps and considerations to keep in mind. You must:

- Design and define your custom event (see “Designing a custom event” on page 50”).
- Carefully plan and set up the order in which event handlers and actions execute (see “About event handling and order” on page 51).

-
- Periodically discard the metadata cache, to ensure that you are working with the most current version of the event metadata (see “Refreshing the metadata cache” on page 54).

Designing a custom event

When you create a custom event, you must also define what will fire the event. In the current WinStudio system, there are four ways to generate a custom event:

- Create an event trigger using the **Event Trigger** form. This is probably the easiest and most common way to generate a custom event.
- Use an action of type Generate Event in another event handler.
- Use a form (WinStudio) event handler with a Response Type of Generate Application Event. (Do not confuse the form, or WinStudio, event handlers with the Application Event System handlers described in this guide.)
- Use the WinStudio API to write a custom script that generates an event.

These steps represent a typical process for creating custom application events:

- 1 (Optional) If you want to name the event before defining how it should be triggered and/or handled, use the **Events** form.

If you do not name the event on the **Events** form, it will still be available to the drop-down lists on other forms (specifically, the **Event Triggers** and **Event Handlers** forms). Events named on those forms, however, do not display on the **Events** form. So, if you want the event to display on the **Events** form, you should name the event on that form.

For the procedure to create an event, see “Defining events” on page 37.

- 2 (Required only if you want to fire the event using an event trigger) To define one or more triggers that will fire the event, use the **Event Triggers** form.

For more information about event triggers, including the procedure to create them, see “About event triggers” on page 38.

- 3 To define one or more event handlers that will execute when the event fires, use the **Event Handlers** form.

For more information about event handlers, including the procedure to create them, see “About event handlers” on page 40.

Each event can have multiple handlers that execute when the event fires. The order in which multiple handlers execute is controlled by a number of factors. For more information and examples, see “About event handling and order” on page 51.

- 4 To define one or more event actions for each event handler, use the **Event Actions** form.

For more information about event actions, including the procedure to create them, see “About event actions” on page 41.

- 5 If required, to name and define an initial state for the event handler to use, use the **Event Variable Groups** form.

For more information about event variables and initial states, see “About event variables and initial states” on page 45.

- 6 If required, to name and define any global constants for the event handler to use, use the **Event Global Constants** form.

For more information about event global constants, see “About event global constants” on page 46.

- 7 Test the event and its triggers and handlers on a test system before implementing them on your live system.

For specific examples of how to create and use custom application events, see Appendix A, “Sample Scenarios.”

About event handling and order

There are two basic ways to arrange the order in which events that have multiple handlers and actions execute those handlers and actions.

Ordering event handlers

When an event handler is first defined and saved, the system automatically assigns it a handler sequence number. When the event handler is first saved, the system checks to see if there are already any other handlers associated with the named event. Depending on the results of that check, the system then assigns **1** to the handler (if there are no other handlers associated with the event) or the next available integer (if there *are* other handlers associated with the event).

In general, then, if an event uses multiple event handlers, by default, the system uses the handler sequence numbers to determine the order in which the handlers execute.

It is possible, however, to indirectly alter this default order. This is done either by:

- Using the **Keep With** and **Chronology** fields on the **Event Handlers** form.
- Moving your own adjacent event handlers up or down in the sequence, using the **Event Handler Sequence** form.

To alter the default order in which event handlers execute using the **Keep With** and **Chronology** fields on the **Event Handlers** form:

- 1 (Optional) From the **Keep With** drop-down list, select the event handler you want to use as a reference point and anchor for the current handler.

This step is not required if you are using the **First** or **Last** option in the **Chronology** field.

- 2 From the **Chronology** drop-down list, select the option you want the current handler to use with respect to the handler you selected in Step 1. Options include:
 - **First** – Executes the current handler before any other handlers.
 - **Before** – Executes the current handler just *before* the referenced handler.

-
- **Instead** – Executes the current handler *in place of* the referenced handler. In this case, the referenced (original) handler does not execute at all.
For exceptions to this rule, see “Overriding Others’ Handlers” on page 244.
 - **Exclusively Instead** – Executes the current handler instead of the referenced handler *and any other handler* that may be referenced to execute instead of (**Instead**) that same handler.
 - **After** – Executes the current handler just *after* the referenced handler.
 - **Last** – Executes the current handler after all other handlers.

For more information about reordering event handlers, including some detailed examples, see Appendix E, “Synchronization of Metadata.”

Resequencing event handlers

Use the **Event Handler Sequence** form to change the sequence in which your adjacent event handlers execute for a specified event.

Note:

- This form is intended to be accessed as a linked form, from the **Event Handlers** form only, using the **Resequence** button. If you open it in standalone mode, the results can be unpredictable.
- You can change the order only of event handlers that have the same Access As value as displayed on the **Access As** form, and *then* only if they are grouped together (that is, adjacent to one another in the sequence), and within the group. You cannot use this form to change the sequence of event handlers with other Access As values.
- To change the order of your event handlers with respect to those of others (that is, with different Access As values), use the **Keep With** and **Chronology** fields on the **Event Handlers** form.

To change the sequence of an event handler:

- 1 In the grid view, select an event handler that has your Access As value.
- 2 Click the **Up** or **Down** button to move the selected handler up or down in the sequence, keeping in mind the restrictions mentioned above.

If you try to violate those restrictions, the system generates an error and you cannot complete the move.

- 3 Save.

Ordering event actions

A single event handler can contain multiple event actions. When you define an event action, you must assign an action sequence number to it (in the **Action Sequence** field of the **Event Actions** form). You can assign any number you want in that field, and the system automatically sorts the actions in the correct sequence on the **Event Handlers** form. When the event handler executes, the system uses this **Action Sequence** number to determine the order in which the actions execute.

The only exception to this rule is that, if you select a particular action (other than **1**) in the **Initial Action** field of the **Event Handlers** form, then processing of the event actions begins with the designated action and proceeds from there. So, for example, if you have four actions associated with

a handler, and you later decide you want action number 3 to be the starting point, you would select **3** in the **Initial Action** field. In this case, only actions 3 and 4 would execute. The system would skip over actions 1 and 2. (You could later execute these actions by using an action type of Branch or Goto, with one of these actions as the destination.)

Determining names of IDO collections and components

To create custom event handlers and actions, you are often required to know the internal names of the collections (known internally as "IDO collections") and the components you want to refer to. For example, to set up a handler, you often need the name of the IDO collection associated with a particular form. To include dynamic content in the subject or body of a message, you often must know the internal name of a column or component within that IDO table.

But what if you do not know those names? How can you find what you need?

Determining the name of an IDO object

To determine the name of an IDO object for an event handler definition:

- 1 Open the form that uses the IDO collection you require.

For example, if you are setting up an event handler to work with customer records, you would open the **Customers** form.

- 2 Specify Design mode for that form.

- 3 Verify that the **Form** properties sheet is showing.

If it is not, from the **View** menu, select the **Form Properties** option.

- 4 Select the **Collections** tab.

The names of all IDO collections associated with that form are displayed in the Collections list at the top of the form. Usually, there is only collection, which makes it easy to figure out. If more than one collection is listed, you might have to take further steps to determine which is the one you need.

The internal name of the IDO is that part of the IDO name that displays after the period. For example, the **Customers** form uses the SL.SLCustomers IDO. The name of the IDO collection as you need it to create an event handler, is **SLCustomers**.

Determining the name of a form component

To determine the name of a form component:

- 1 Open the form that has the field or other component that displays the data to which you want to refer.

For example, to create a subject line that includes a customer's ID number and name, you would open the **Customers** form.

-
- 2 Specify Design mode for that form.
 - 3 Verify that the **Component** properties sheet is showing.
If it is not, from the **View** menu, select the **Component Properties** option.
 - 4 On the form, select the component for which you require the name.
For example, to determine the customer ID number, select the drop-down list box to the right of the **Customer** field label.
 - 5 On the **Component** properties sheet, locate the **Data Source** section, **Binding** field.
The component's internal name displays in that field, after the period. For example, for the customer ID number, the **Binding** field displays **object.CustNum**. Thus, the customer ID field name is **CustNum**.

Refreshing the metadata cache

Because certain summary event metadata is cached in memory for faster performance, the IDO metadata cache should be discarded periodically, after changes to event metadata (that is, after making changes to events, handlers, actions, triggers, and global constants). This should be done, at a minimum, after doing development work, before testing, and after synchronizing the metadata on your system.

Note:

- If you have multiple utility servers in your system, you must discard cached metadata for each application server on which metadata might have been cached. The best way to do this is to use the second option described below.
- Any event metadata that is not referenced within two minutes is automatically discarded from the cache. That is why you might notice that things work the way you expect, even without manually discarding the cached metadata. We still recommend that, as a precaution, you manually discard the cached metadata to be sure.

There are four ways you can discard the cached metadata:

- By using the **Discard Cache** button on the **Utilities** tab of the Configuration Manager. For more information, see the Configuration Manager online help.
- By unloading global objects from your system (requires that certain settings be made; see “Refreshing cached metadata by unloading global objects” on page 54).
- By restarting the IDO Runtime Service on the application server (see “Refreshing cached metadata by restarting the IDO Runtime Service” on page 55).
- In the IDO Runtime Development Server (requires that you be running a copy of this software locally; see “Refreshing cached metadata in the IDO Runtime Development Server” on page 55).

Refreshing cached metadata by unloading global objects

With the correct system settings, you can have the system discard the cached metadata automatically every time you unload all global objects.

To discard cached metadata by unloading global objects:

- 1 In WinStudio, from the **View** menu, select **Settings**.
- 2 Select the **Runtime** tab.
- 3 Select the check box labeled **Unload IDO metadata along with global objects**.

While working on event metadata, periodically unload global objects, and the cached metadata is automatically discarded with the global objects.

Refreshing cached metadata by restarting the IDO Runtime Service

If you do not want to use the first option and you are not using the IDO Runtime Development Server on your local machine, you can discard the cached metadata by manually stopping and restarting the IDO Runtime Service on the application server. This works because the metadata cache is not saved when the service is stopped.

To discard cached metadata by restarting the IDO Runtime Service:

- 1 On the application server, open **Control Panel**.
- 2 Select **Administrative Tools > Services**.
- 3 From the list double click **Infor Framework IDO Runtime Service**.
- 4 In the **Infor Framework IDO Runtime Services Properties** dialog box, click **Stop**.
- 5 When the **IDO Runtime Services Properties** dialog box indicates that the service has stopped, click **Start**.

Refreshing cached metadata in the IDO Runtime Development Server

If you are using a local installation of the IDO Runtime Development Server, you can discard the cached metadata manually.

To manually discard cached metadata in the IDO Runtime Development Server:

- 1 In the IDO Runtime Development Server, select the configuration for which you want to discard the cached IDO and event metadata.
- 2 From the **Configuration** menu, select **Discard IDO Metadata Cache**.

Some application events can take a considerable amount of time to process, especially if they involve event messages that require responses from the recipients. The system provides a number of tools and forms that allow you to track the status of application events as they execute and after they have finished executing. Some of these forms also allow you to temporarily adjust the behavior of handler execution.

These forms are located in the Explorer under **Master Explorer > System > Event System**.

Initially, these forms can be accessed only by members of the System Administration authorization group.

These forms can all be used to track various aspects of event system status:

- Event Status form
- Event Handler Status form
- Event Queue form
- Event Revisions form
- Event Handler Revisions form
- Suspended Updates form

Event Status form

Use the **Event Status** form to view:

- The status of events that are currently running
- The history of events that have finished running

This form has five tabs:

- **Event** tab – Provides options to filter for events and handlers in various states and make it easier to locate specific ones.
For example, suppose you want to find data about all events that are currently suspended. With Filter-in-Place, you would select the **Suspended** check box and then execute Filter-in-Place.
- **Handlers** tab – Shows data about handlers that are currently processing or have finished processing.

-
- **Parameters** tab – Shows data about the parameters associated with an event that is currently running. These include input parameters passed into the event, as well as output parameters created by handlers' actions.
 - **Output Parameters** tab – Shows data about the output parameters of an event that has finished running.
 - **Voting** tab – Shows data about the voting status of a Prompt action.

For more information about this form and its options, see the online help.

Event Handler Status form

Use the **Event Handler Status** form to view:

- The status of event handlers that are currently running
- The history of event handlers that have finished running

This form has four tabs:

- **Handler** tab – Shows data about the event handler itself.
- **Actions** tab – Shows only data about actions of the handler that have started.
- **Variables** tab – Shows information about variables associated with the event handler.
- **Voting** tab – Shows data about the voting status of a Prompt action.

For more information about this form and its options, see the online help.

Event Queue form

The **Event Queue** form displays a list of all asynchronous events and event handlers that the system has queued for processing. All information on this form is display-only. For more information about synchronous and asynchronous events and handlers, see “Synchronicity” on page 20.

Events on the queue are processed in FIFO (first in, first out) order. The **ID** number on this form indicates the order in which events are queued for processing.

This form also displays other information about events and event handlers that have been queued for processing. Each event or event handler that has been queued is displayed as a separate record in the grid view.

For more information about this form and its options, see the online help.

Event Revisions form

The **Event Revisions** form displays information about the event revisions associated with events.

For more information about event revisions and how they work, see “Events and event handler revisions” on page 32.

Event Handler Revisions form

The **Event Handler Revisions** form shows information about event handler revisions associated with event handlers. All information on this form is display-only. The reason for this is that some running or finished handlers are using the metadata in this revision to complete their processing. To change the data for this event handler, you must use the **Event Handlers** form.

For more information about event handler revisions and how they work, see “Events and event handler revisions” on page 32.

Suspended Updates form

The **Suspended Updates** form displays a list of all the update actions that are currently in a suspended state. This form also allows you to take selected updates out of suspension manually.

For more information about this form, see the online help for the form.

Event messages in the Application Event System can be generated by:

- The system, as part of an event handler's actions
Only Notify and Prompt action types can generate messages.
- Other users on the system, much like email

Each message is visible only by the recipients and, optionally, the sender of that message.

Event message-related forms

Event message-related forms are used to view, sort, file, respond to, and send messages generated within the Application Event System. These message forms reside in the **Master Explorer > System > Messages** folder.

The Application Event System uses these message-related forms:

- The Inbox form
- The Saved Messages form
- The Send Message form

The **Inbox** form

The **Inbox** form can be accessed in a number of different ways:

- From the **View** menu, by selecting the **Inbox Form** option.
- In the Windows taskbar (notifications area), by double clicking the **Inbox** notification icon ().
- In the same ways that any other forms are accessed.

The **Inbox** form displays system messages that can come from two possible sources:

- Application events that employ a Notify or Prompt action type.
- Communications initiated and sent by individual system users to other users on the system.

This form displays only those messages which are still in the recipient's **Inbox** "folder" and not messages that have been moved to other folders. To view those messages, recipients must use the **Saved Messages** form. The **Inbox** form also does not allow recipients to move messages to other "folders." To do that, recipients must use the **Saved Messages** form. For more information, see "The Saved Messages form" on page 63.

When a message is received, if the recipient has set options to be notified, the system alerts the recipient with the selected notifications. For more information, see "Notifications Settings" in the online help.

Recipients can mark messages as read and, depending on how and why the message was sent, make other responses.

Responding to system-generated messages

If the message is the result of a system-generated prompt, each recipient can respond to the prompt, usually by means of a set of voting buttons. For more information about these buttons, see "Prompts and responses" on page 65.

If the message was also sent to the recipient's external email inbox, and the recipient responded to the email message, the message in the refreshed **Inbox** is marked as expired and the buttons are inactive, so the recipient cannot respond twice.

If the message is system-generated and involves variables, for each variable, depending on the **Variable Access** setting for the event variable (on the **Event Actions** form) or initial state (on the **Event Variable Groups** form) or payload status (on the **Event Action Notify** or **Event Action Prompt** form), recipients might be able to:

- Provide an optional response
- Provide a mandatory response
- Only read the variable value
- Not see the variable value at all

Setting variable access

The effective visibility and writability of each variable displayed on the **Inbox** form is determined by two things:

- What type of action generated the message (Notify or Prompt)
- The (optional) variable access level as specified on the **Event Action** form for the action itself and/or on the **Event Variable Groups** form for the event handler's initial state and/or on the **Event Action Notify** or **Event Action Prompt** form for the action with regard to the payload status of the property that corresponds to the variable, when the handler is associated with an IDO event.

For Notify type messages

The default for variable access is **Read-Only**. You can use the variable access options to override this to **Hidden** for each variable.

For Prompt type messages

The default for variable access is **Writable**. You can use the variable access options to override this to **Hidden**, **Read-Only**, or **Mandatory** for each variable.

To change the value of a variable that might appear with a message in the **Inbox** form, you can do any of these actions:

- Specify the data using the **Variables** tab on the **Inbox** form as part of a response to a Prompt action.
- Use a Set Values event action with the function syntax: `SETVARVALUES (PropertyName=name)`
- Use any of various event actions that can set a variable on output using the function syntax:
`RV (PropertyName)`

Setting translatable captions for variables

In the **Inbox** form's **Variable** tab, the **Caption** column displays the contents of the notify or prompt message's variable captions in the current user's language. This assumes that:

- The **Caption** component attributes are set to interpret the bound contents.
- The caption contains a translatable string name.

For payload variables resulting from an IDO event, this string name might come from an IDO property's label string ID. For non-payload variables that are created by an event action, you define this string name using the **Event Actions** form, **Variable Access** tab.

The **Saved Messages** form

The **Saved Messages** form displays messages that you have saved or at least have not yet deleted for a selected "folder." Within this folder, you can sort messages by any one of a number of criteria. For more information, see the online help for this form.

You can also use this form to:

- Create your own "folders."
- Move messages from one folder to another.

For more information, including the procedure to move messages, see the online help for the **Folder Name** field on this form.

Folders, in this system, are not represented visually as they typically are in email programs such as Outlook. The only place you can actually view your personal folders is in the **Folder Name** field on this form. For more information, including the procedure to create your own folders, see the online help for the **Folder Name** field.

You can also view messages on this form for information about any responses you might have made to messages generated by the system.

Moving messages between folders

You can use the **Saved Messages** form to move messages from one folder to another. If the folder does not already exist, you can create the folder at the same time.

To move a message from one folder to another folder:

- 1 Open the **Saved Messages** form.
- 2 From the **Folder Name** drop-down list, select the folder in which the message you want to move is currently located.

The system displays in the grid view all messages in that folder.

- 3 In the grid view, select the message you want to move.
- 4 In the **Folder Name** field, do one of these actions:
 - For an existing folder, specify the name of the folder or select the folder from the drop-down list.
 - To create a new folder and move the message to that folder, specify the name of the new folder.
- 5 Save the message.

The **Send Message** form

The **Send Message** form is used to send system messages, similar to email messages, to other users in the system. You can designate multiple recipients, "carbon-copy" recipients, and instruct the system whether to save a copy in your **Sent Items** "folder."

For more information, including specific procedures and instructions for using this form, see the online help for this form.

Send email to external email Inbox for prompts

If a user is set up in the **Users** form to allow the system to "send external prompts," then a prompt action sends the message both to the Mongoose **Inbox** form and to the recipient's inbox in an external email client such as Microsoft Outlook. The message that is sent to the external email system is an HTML-formatted email that consists of these parts:

- Original subject in **Subject** line
- **Category**
- List of internal recipients
- List of internal **Cc** recipients
- Original **Body** (either in plain text or HTML format)
- Original **Question**

-
- **Choices**, as individual hyperlinks to a .NET active server page (ASP) that records the vote
 - Payload, which is the contents of the Variables grid from the **Inbox**

Text in the **Subject**, **Category**, **Body**, **Question**, and **Choices**, as well as payload captions, can be translated and formatted based on the default language specified for the recipient user in the **Users** form.

When the recipients vote by clicking the link in the email, their Windows-default Web browser opens, or opens a new tab if applicable, and sends the information from the link they clicked to the ASP URL that was included (and hidden) in the email. This URL is built by the system based on the web server list.

The ASP then registers the vote programmatically, as if the recipient had logged into the system, displayed the same message in the **Inbox**, and selected the corresponding **Choice** button on the **Response** tab.

The web page then displays a success or failure message.

The message displays both in email and in the **Inbox** form, but only one response is allowed:

- If users respond from email first, when they display the **Inbox** (properly refreshed), the message is expired and the buttons are inactive.
- If users respond from the **Inbox** first, then clicking a link in the external email brings up the web page with a message that the message has expired and they already voted, using the previously selected choice.

Prompts and responses

If a message is the result of a Prompt action, the sender of that message can request a specific response from each recipient, usually in the form of a button-vote mechanism. In such cases, the system must be set first to wait for responses, then to know how to handle responses as they are received, and finally, be instructed what to do when responses are not received within a specified timeout period.

Incoming prompts request a response from recipients (using the **Question** field on the **Inbox** form or the external email) and display a set of choices. The choices are displayed in the form of voting buttons in the **Response** tab area of the **Inbox** form, or in the form of links in an external email. For example, a prompt might include buttons or links labeled:

- Approve / Disapprove (default option)
- Yes / No
- OK / Send More Info / Cancel

To customize the choices for a prompt, you must include a Prompt action with a **Choices** parameter as part of the event action definition. This **Choices** parameters consists of the CHOICES function followed by a string expression that evaluates to a comma-separated list that contains an even number of elements (value/label pairs). For example, if you want the voting buttons to be labeled **Yes** and **No**, with corresponding values returned to the action to be **1** and **0**, you could include this parameter:

```
CHOICES ("1, sYes, 0, sNo")
```

In this example, the strings "sYes" and "sNo" are WinStudio form strings. These have already been defined for the system as **Yes** and **No**, respectively.

If you want your button labels to be localized, for the recipient, you must:

- Use names of existing form strings (as found in the Strings table).
- OR
- Add your own form strings, using the **Strings** form, and provide the necessary translations. (To open the **Strings** form, you must be in Design Mode and, from the **Edit** menu, select **Strings**.)

If localization is not an issue, you can also use a literal value that displays on the button verbatim. To specify the string as a literal value here, simply specify it as a list value. If the system does not find the string in the Strings table, the system automatically treats it as a literal value.

For more information about the **Choices** parameter, see “Event Action Parameters” on page 147.

Voting rules

When a prompt is sent to a single recipient, the result of the prompt is the return value from that recipient's choice. However, when a prompt is sent to multiple recipients, you must select a vote-counting method to determine the result of the prompt and include a Voting Rule parameter in your event action definition.

This table lists and describes the available voting rules.

Rule	Description
Majority	A choice must receive more than 50% of the vote to win. As soon as more than 50% of the recipients respond with a particular choice, that choice wins. If you use this voting rule, you should use a Voting Tie parameter (VOTINGTIE) to tell the system how to handle a tied vote. For more information, see “Dealing with indeterminate voting results” on page 69.
Plurality	The choice with the highest number of votes wins, even if it does not receive more than 50% of the vote. For example, if three choices are offered, and: The first choice receives 24% of the vote; The second choice receives 43% of the vote; and The third choice receives the remaining 33% of the vote; the second choice wins, even though it received less than 50% of the total vote. If you use this voting rule, you should use a Voting Tie parameter (VOTINGTIE) to tell the system how to handle a tied winning vote. For more information, see “Dealing with indeterminate voting results” on page 69.
ConditionalPlurality	The choice with the highest number of votes wins, but only if a specified minimum percentage of votes is reached. If you use this rule, you must also include a Minimum Percentage (MINIMUM) parameter. For example, if three choices are offered to 19 recipients, and you specify a minimum of 40% to win, then: In an 8-7-4 split, the choice with 8 votes would win because it meets the minimum percentage. In a 7-6-6 split, there would be no winner, because no choice meets the minimum percentage. In this case, the system must deal with the vote as an indeterminate result. For more information on indeterminate results, see “Dealing with indeterminate voting results” on page 69. (With a simple Plurality vote, the choice that reaches 7 votes in a 7-6-6 split would win.) If you use this voting rule, you should use a Voting Tie parameter (VOTINGTIE) or Voting Disparity parameter (VOTINGDISPARITY) to tell the system how to handle the indeterminate vote. For more information, see “Dealing with indeterminate voting results” on page 69.

Rule	Description
MinimumCount	The first choice to reach a specified minimum number of votes wins. If you use this rule, you must also include a Minimum Count (MINIMUM) parameter. For example, if three choices are offered to 13 recipients, and you specify a minimum of 5 votes to win, the first choice to receive 5 votes automatically wins. Note that, as soon as the minimum count is reached, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place.
MinimumPercentage	The first choice to receive a specified percentage of the vote wins. The percentage is based on the number of recipients of the prompt, not the number of respondents. If you use this rule, you must also include a Minimum Percentage (MINIMUM) parameter. Note that, as soon as the minimum percentage is reached for a choice, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place.
EarliestResponse	The first response to the prompt wins, regardless of the choice. Note that, as soon as the first response is received, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place.
PreferredChoice	If any one respondent votes for the preferred choice, that choice wins. In a case where none of the respondents select the preferred choice, then this rule behaves like the Plurality rule for the remaining choices. If you use this rule, you must also include a Preferred Choice (PREFCHOICE) parameter to specify which choice is the preferred choice. For example, if you have three choices, and you specify the first choice as the preferred choice, then: If anyone votes for the first choice, that choice wins. If the end vote is a 0-6-5 split, the second choice wins. Note that, as soon as the preferred choice receives a vote, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place.

Rule	Description
MinimumCountPreferredChoice	If a specified number of votes for a specified choice is cast, that choice wins. If you use this rule, you must also include a Minimum (MINIMUM) parameter to specify the minimum count, and a Preferred Choice (PREFCHOICE) parameter to specify which choice is the preferred choice. For example, if you set the Minimum to 3 for a Preferred Choice of "Approve," and 3 recipients respond with "Approve," the preferred choice wins. If less than that number of votes are cast for that choice after all recipients have responded, the vote reverts to Plurality. (In that case, the preferred choice may still win.) Note that when you set the Minimum to 1, this rule behaves exactly like Preferred Choice.
MinimumPercentagePreferredChoice	If a specified percentage of votes for a specified choice is cast, that choice wins. If you use this rule, you must also include a Minimum (MINIMUM) parameter to specify the minimum percentage, and a Preferred Choice (PREFCHOICE) parameter to specify which choice is the preferred choice. For example, if you set the Minimum to 25% for a Preferred Choice of "Approve," and 2 of 8 of recipients respond with "Approve," the preferred choice wins. If less than that percentage of votes are cast for that choice after all recipients have responded, the vote reverts to Plurality. (In that case, the preferred choice may still win.)

Dealing with indeterminate voting results

These example situations create "indeterminate" voting results and set action attributes that are exposed as event functions that can be evaluated by subsequent event actions:

- Any disagreement among multiple recipients, registered as soon as a disagreement is detected. This can include a vote like the example offered in the Plurality description above.
The associated event function is the VOTINGDISPARITY() event function, which is a Boolean function that indicates only that there was a disagreement.
- A tie in the case of a Plurality or Majority vote, registered at the point when all responses have been received or when the timeout period has expired.
The associated event function is the VOTINGTIE() event function, which is a Boolean function that indicates only that there was a tie.

You can use the returns from these functions, along with the functions RECIPIENTS(), RESPONDERS(), RECIPIENTLIST(), RESPONDERLIST(), and NONRESPONDERLIST() to take further actions, such as:

- Reprompting all the recipients and try to get a consensus.
- Reprompting only a select group of the respondents and urging them to adopt a different choice.
- Reprompting only recipients who have not yet responded.
- Taking some other predetermined action.

Quorums

On a Prompt action, a quorum is automatically calculated based on the number of recipients, the voting rule, and voting parameters such as Minimum. If there is a number of votes by whose tally a voting result can be determined unambiguously, that number is the quorum. Otherwise, the quorum is the number of recipients, that is, everyone has a chance to vote unless a timeout expires. As soon as the quorum is reached, voting is closed, any remaining unvoted messages are expired, and the event continues to the subsequent event action.

However, if you specify a **Quorum** value, that overrides the automatic calculation. For example, if a message requiring a response is sent to 10 people, but you want a quorum to be reached when only 4 have voted, then specify 4 as the Quorum value.

By default, if **Quorum** is not specified or is specified with a positive value, **Wait for Quorum** is true; that is, the event waits until the quorum is reached before it continues with the next event action. If **Quorum** is specified with a non-positive value, the **Wait for Quorum** default value is False. If these settings conflict, for example **Quorum = 3** and **Wait for Quorum** is False, the system displays an error message.

If **Wait for Quorum** is false, the event does not wait for a quorum to be reached. As soon as the messages are sent, execution continues with the next event action. If the system is not waiting for a quorum, the event designer needs to determine when a quorum is reached and what further actions to take. This can be done using VOTINGRESULT(), RESPONDERS(), RECIPIENTS(), etc., in combination with the Wait or Sleep actions.

Appendix A: Sample Scenarios

A

This appendix presents a number of typical scenarios in which you might want to use the Application Event System to automate various tasks in response to various situations. In each case, the situation is described and then a proposed solution involving events, handlers, and/or triggers. These solutions are presented in a step-by-step format, as examples that you can learn from and possibly modify for your own use.

Note:

- This appendix is still under development, and more scenarios will be provided as they become available. We recommend that you frequently check the Infor global support web site, for updates to this guide.
- At this time, all sample scenarios are built on SyteLine forms. The principles used here, however, can be applied to any application built on the Mongoose framework.
- To a certain extent, each scenario builds on the concepts and practices of previous scenarios, so the most effective way to use them is to work through them sequentially. However, each scenario is also more-or-less "self-contained" and can be used independently of the others.
- To see a graphical representation for each flow as you work on it, you can use the **Diagram** button on the **Event Handlers** form. This button opens the **Event Handler Diagram** form, which you can use to view the flow of the event handler as well as access the **Event Actions** form to edit individual actions. For more information, see the online help for the **Event Handler Diagram** form.

This list is a list of the scenarios included in this appendix:

Sending notifications

- Scenario 1: Notification of a new record—adding a customer on page 73 – A simple notification is sent to a credit manager when a new customer is added to the database.
- Scenario 2: Notification of changes to an existing record—changing the credit limit on page 78 – The credit manager is notified by email that a customer's credit limit has been changed and is told what the new credit limit is.
- Scenario 3: Notification that includes an "old" value on page 82 – A group of inventory stockers are automatically notified whenever an item's lot size changes. In the message that is sent, both the previous lot size and the new lot size are included.

Requesting approvals

- Scenario 4: Approval for a new record on page 88 – A purchasing manager is prompted for approval whenever a new purchase order is requested.
- Scenario 5: Requesting approval by external email for changes to an existing record on page 94 – A credit manager is prompted through an external email for approval of a change to a customer's credit limit.
- Scenario 6: Requesting multiple and complex approvals on page 101 – A purchasing manager is prompted for approval on a purchase order (PO) both of a change in status to **Ordered** and for the amount of the PO. If the PO is for an amount greater than \$100,000, a supervisor is also prompted for approval. If the PO is for an amount greater than \$1,000,000, two senior-level executives must also approve it.

Modifying records

- Scenario 7: Adding information to a record on page 113 – A credit manager is prompted to provide a credit limit for a new customer, by means of a response to a message.

Voting

- Scenario 8: Voting for various choices on page 115– Several managers are prompted to approve an engineering change, by means of a response to a message.

Localizing message contents

- Scenario 9: Translating captions in a purchase request on page 117 – A message containing localizable strings is created.

More advanced scenarios

- Scenario 10: Opening a session in a remote environment on page 119 – A remote site or Mongoose environment is accessed to retrieve data. The details of and procedure for this scenario are in the *Integrating IDOs with External Applications* guide.
- Scenario 11: Cross-site event firing - Adding a message to another site's Inbox on page 120 – A message is sent to another site's **Inbox** form by using a GenericNotify event.

Sending notifications

One of the simplest uses of the event system is to set up situations in which message notifications are sent out automatically to specified individuals whenever a situation occurs or a condition is met. The scenarios in this section illustrate this kind of situation.

Scenario 1: Notification of a new record—adding a customer

Suppose you have a credit manager who wants to be notified whenever a new customer is added to the system, regardless of who adds the customer. Of course, you can simply require each employee who adds customers to the system to manually send a notice whenever a customer is added. But that places an additional burden on the employee and is prone to possible oversight.

A simple event and handler

You can use the Application Event System to automatically create a notice whenever a new customer is added. In this example, you do not need to create an event, because Infor provides an event named `IdoPostItemInsert` that you can use. All you need to do is create an event handler for that event and assign an action to it that generates and sends the message to the credit manager.

We could use the `IdoOnItemInsert` framework event instead of the `IdoPostItemInsert` event. This will be significant when we get to the section on Refining the message on page 75. The advantage of using the `IdoPostItemInsert` event is that, if you allow the **Customers** form to auto-assign the customer number (instead of specifying the customer ID number yourself), the system waits until the ID number has been assigned before filling in the "CustNum" data in the message. If we use the `IdoOnItemInsert` event, the system does not wait, which means that, if you auto-assign the customer ID number, the resulting message has "TBD" in place of the actual customer number.

To set this up:

1. Create the event handler:
 - a. Open the **Event Handlers** form.
 - b. Press F3.
 - c. Press CTRL+N.
 - d. Create the handler with these settings:

Field or Option	Setting / Comments
Event Name	From the drop-down list, select <code>IdoPostItemInsert</code> . Note: For details about this and the other framework events included with the system, see "Core application events" on page 126.
Applies to Initiators	Leave blank.
Applies to Objects	Specify SLCustomers . To determine what object you need, see the procedure provided in the online help for this field.
Keep With	Leave blank.
Chronology	Leave blank.
Ignore Failure	Cleared.

Field or Option	Setting / Comments
Suspend	Cleared.
Synchronous	Cleared. Because this notification does not require any response from the credit manager, it can run asynchronously. For more information, see "Synchronicity" on page 20.
Active	Selected.
Can Override	Selected.
Transactional	Cleared.
Obsolete	Cleared.
Initial State	Leave blank.
Initial Action	Leave blank.

For more information about any of these fields and options, see the help for the **Event Handlers** form.

- e. Save.
2. Define the action for the event handler you just created:
 - a. In the **Event Handlers** form, select the handler you created in Step 1.
 - b. Click **Event Actions**.
 - c. In the **Event Actions** form, in the **Action Sequence** field, specify **10**.

Note: Technically, you can specify any integer you want in this field, and the system treats them sequential order. We recommend using multiples of ten, initially at least, just in case you later need to add more action steps between existing steps, so you do not need to renumber all existing steps.
 - d. From the **Action Type** drop-down list, select **Notify**.
 - e. Click **Edit Parameters**.
 - f. In the **Event Action Notify** form, click the **To** button.
 - g. In the **Event Action Parameter Recipients** form, from the list of recipients, select the user ID of the credit manager (or whoever is serving in that role).

TIP: You can select more than one recipient. Also, to deselect a recipient, click the user ID again.
 - h. Click **Update** and then **OK**.
 - i. In the **Subject** field, specify **New Customer!**
 - j. In the **Category** field, specify **Change Notification**.

k. In the **Body** field, specify **We have a new customer!**

l. Select the **Save in Sent Items** check box.

This parameter tells the system to save a copy of the notification in the **Sent Items** folder of the person who added the new customer.

m. Click **OK**.

On the **Event Actions** form, in the editable field on the **Parameters** tab, you should see this:

```
CATEGORY("Change Notification")
TO("userID")
SUBJECT("New Customer!")
BODY("We have a new customer!")
SAVEMESSAGE(TRUE)
```

where *userID* is the sign-in user ID for the credit manager.

TIP: Note where double quotation marks and parentheses are inserted. Because it can be confusing to know where and when to use these punctuation marks, we recommend that you use the event action parameter forms as described in these scenarios. They insert the correct punctuation marks automatically where and when needed and can help you avoid many time-consuming errors in syntax.

3. (Optional, but recommended) To verify that there are no syntax errors, click **Check Syntax**.

4. Save the action and close the **Event Actions** form.

5. Discard the cached metadata.

For more information, including the procedure, see "Refreshing the metadata cache" on page 54.

Test the event by using the **Customers** form to create a new customer. Then, sign in as the user ID specified in the **TO** parameter, open the **Inbox** form, and verify that the message was received. The message should appear there, along with the properties associated with the new customer, on the **Variables** tab. Note that, because you did not specify any *variable access* rules, all properties (variable values) are display-only.

Refining the message

Suppose you now want to refine the message, to make it even more informative and useful to the recipient. Not only do you want the recipient to get a message, but you want that message to include the customer number and name for the new customer, so the recipient can look up the customer profile more easily.

To include the customer number and name in the message:

1. In the **Event Handlers** form, select the handler you just created and then click **Event Actions**.

2. In the **Event Actions** form, click **Edit Parameters**.

3. In the **Event Action Notify** form, click the **Body** button.

-
4. In the **Event Action Expression Editor** form, from the **Select a function** drop-down list, select **SUBSTITUTE**.

The SUBSTITUTE function allows you to specify the basic text of a message, with "replacement markers" embedded in the message. At run time, the system substitutes specified values for these replacement markers. This effectively allows you to create messages with dynamic content.

5. In the **Argument 1** field, specify **We have added a customer, {0}, customer ID {1}, to our family of customers.**

The numbers enclosed in curly braces (**{0}** and **{1}**) are the replacement markers for which values will be substituted at run time.

Note: Replacement markers must be enclosed in curly braces { }. They must begin with zero (0) and increment sequentially. If you do not begin with zero or you skip integers, they do not work.

6. Create the expression that will be used to supply the value for replacement marker **{0}**:
 - a. Place the cursor in row 1 of the **Arguments** grid and then click **Build Expression**.
 - b. In the **Event Action Expression Editor** form, from the **Select a function** drop-down list, select **PROPERTY**.
 - c. The PROPERTY function picks up the value of the **CustNum** (Customer Number) field.

Note: What if you do not know the name of the property for which you want to retrieve a value? How do you find that property name? For an easy method to find the property name, see "Determining names of IDO collections and components" on page 53.

- d. In the **Argument 1** field, specify **Name** and then click **OK**.
7. Repeat Step 6 for row 2, using **CustNum** for the PROPERTY argument (*propertyname*) in substep d.
 8. In the **Event Action Notify** form, click **OK**.
 9. (Optional) On the **Event Actions** form, click **Check Syntax**.
 10. Save the action and close the **Event Actions** form.
 11. Discard the cached metadata.

For more information, including the procedure, see "Refreshing the metadata cache" on page 54.

Test by creating a new customer record and verifying that the intended recipient receives a notification message that includes the correct new customer name and number.

Refining the recipient

When defining the recipients for this message, it can be a good idea to use a global constant value, rather than a hard-coded user ID. This allows you to use the same global constant value in other places in your application. Then, if the name of the credit manager changes, for instance, it is possible to change the recipients by simply changing the global constant value. It also allows you to

add multiple recipients, for instance, if you have co-credit managers or you have a trainee you want to also receive the messages.

For more information about global constants, see "About event global constants" on page 46.

To redefine the recipients as a global constant:

1. Create the global constant:

- a. Open the **Event Global Constants** form and take it out of filter-in-place mode.
- b. In the **Name** field, specify the name to assign to the constant.
- c. In this case, you might use **CreditMgr**.
- d. In the **Value** field, specify the login user ID for the credit manager.

Note: To add multiple recipients, specify the user IDs separated by semi-colons only—no spaces.

- e. Save the global constant and close the form.

2. Incorporate the global constant in the event handler action:

- a. In the **Event Handlers** form, select the handler and then click **Event Actions**.
- b. In the **Event Actions** form, click **Edit Parameters**.
- c. In the **Event Action Notify** form, click the **To** button.
- d. In the **Event Action Parameter Recipients** form, click the **Recipients** button.
- e. In the **Event Action Expression Editor** form, from the **Select a function** drop-down list, select **GC**.

The GC function calls a specified global constant and uses its value at run time. In this case, you want the global constant you created in Step 1.

- f. From the **Argument 1** drop-down list, select the global constant you created in Step 1 (**CreditMgr**) and then click **OK**.

Notice that the global constant name is not enclosed in double quotation marks. Generally, only literal strings and property names must be enclosed in double quotation marks.

- g. In the **Event Action Parameter Recipients** form, click **OK**.
- h. In the **Event Action Notify** form, click **OK**.
- i. To verify that there are no syntax errors, click **Check Syntax**.
- j. (Optional) On the **Event Actions** form, click the **Substituted Parameters** tab and notice that the **TO** parameter indicates the actual recipient; in other words, the value of the global constant.
- k. Save the action and close the **Event Actions** form.

Test by creating a new customer record and verifying that all designated recipients receive the notification message. You can also check the **Saved Messages** form for the user ID from which you were signed in when the message was sent.

Points to note and remember

In creating this kind of event handler, keep these points in mind:

- If you do not require a response from the recipient, create the handler as an asynchronous handler, to avoid system slow-downs.
- To be able to use a recipient in other handlers and be able to change that recipient when necessary in only one place, use an event global constant for the recipient.
- To use active data in a message, use the SUBSTITUTE and PROPERTY (or P) function constructs.

Scenario 2: Notification of changes to an existing record—changing the credit limit

This scenario is similar to the first one, except that, instead of notifying the credit manager when a customer is added to the database, we want to create an event and handler that notifies the credit manager whenever a customer's credit limit is changed. Because the credit manager prefers email and is not always signed in to the system, we also want to send the notification as an email.

Note: For this scenario to work properly, you must have SMTP enabled and configured on the application server, using the **Intranets** form. You must also have your SMTP server set up to relay the emails that are sent. For information on how to do this, see your Windows operating system documentation. Finally, any recipients must also have email addresses saved as part of their user profiles.

As with the first scenario, we can use an existing framework event, **IdoOnItemUpdate**, and create our own handler for it. And again, because we are simply sending out a notification and the system is not waiting for a response from the credit manager, we can make it an asynchronous event handler.

This event handler requires two actions: one to check whether the **Credit Limit** field has been changed and one to send the email notification.

To accomplish this scenario:

1. Create the event handler:
 - a. Open the **Event Handlers** form.
 - b. Press F3.
 - c. Press CTRL+N.
 - d. Create the handler with these settings:

Field or Option	Setting / Comments
Event Name	From the drop-down list, select IdoOnItemUpdate . NOTE: For details about this and the other framework events included with the system, see "Core application events" on page 126.
Applies to Initiators	Leave blank.

Field or Option	Setting / Comments
Applies to Objects	Specify SLCustomers . To determine what object you need, see the procedure provided in the online help for this field.
Keep With	Leave blank.
Chronology	Leave blank.
Ignore Failure	Cleared.
Suspend	Cleared.
Synchronous	Cleared. Because this notification does not require any response from the credit manager, it can run asynchronously. For more information, see "Synchronicity" on page 20.
Active	Selected.
Can Override	Selected.
Transactional	Cleared.
Obsolete	Cleared.
Initial State	Leave blank.
Initial Action	Leave blank.

For more information about any of these fields and options, see the online help for the **Event Handlers** form.

- e. Save the handler.
2. Create the first action, which checks the condition of the **Credit Limit** field when the customer record is saved:
 - a. In the **Event Handlers** form, select the handler you just created.
 - b. Click **Event Actions**.
 - c. In the **Event Actions** form, in the **Action Sequence** field, specify **10**.
 - d. From the **Action Type** drop-down list, select **Finish**.
This action type tells the system to finish executing the handler when a particular condition has been met and exit.
 - e. Click **Edit Parameters**.
 - f. In the **Event Action Finish** form, click the **Condition** button.
 - g. In the **Event Action Parameter Condition** form, click the **Expression 1** button.

-
- h. In the **Event Action Expression Editor** form, from the **Select a function** drop-down list, select the function **PROPERTYMODIFIED**.

The **PROPERTYMODIFIED** function checks to see whether the named property has been modified since the last save. If the property has been modified, the expression returns a value of **TRUE**.

Note: What if you do not know the name of the property for which you want to retrieve a value? How do you find that property name? For an easy method to find the property name, see "Determining names of IDO collections and components" on page 53.

- i. In the **Argument 1** field, specify **CreditLimit**, which is the name of the property that we want to check.
- j. Click **OK**.

Notice in the **Event Action Parameter Condition** form that the expression has been returned and that double quotation marks have been automatically inserted around the name of the property.

Notice also that the **Operator** and **Expression 2** fields have been disabled. This is because the **PROPERTYMODIFIED** function is a Boolean expression; thus, no comparison is needed to return a Boolean value.

- k. Select the **NOT** check box.

The reason you select this option is because, if you did not, the expression would return a value of **TRUE** whenever the **CreditLimit** property *has* been modified and the handler would finish. But you want the system to continue to the next action when the **CreditLimit** property has been modified; you want the system to finish at this point only if the **CreditLimit** property has *not* been modified.

- l. Click **OK**.

Notice that the system returns the expression to the **Event Action Finish** form correctly formatted.

- m. Click **OK**.

Notice that the system returns the entire parameter to the **Event Actions** form with the syntax correctly formatted.

- n. To verify that the syntax is correct, click **Check Syntax**.
- o. Save the action.

3. Create the second action, which sends the email notification:

- a. To create the action, press **CTRL+N**.
- b. In the **Action Sequence** field, specify **20**.
- c. From the **Action Type** drop-down list, select **Send Email** and then click **Edit Parameters**.
- d. In the **Event Action Send Email** form, click the **To** button.
- e. In the **Event Action Parameter Recipients** form, select the user ID for the credit manager.

If the credit manager has an email address set up as part of the user profile, the email address displays to the right of the user ID. If the credit manager's user ID does not display an email address, you must add the email address to the credit manager's user profile on the **Users** form.

Notice that you could again use a global constant for the credit manager's email address. However, because we are sending email, we cannot reuse the existing **CreditMgr** global constant, but must create a new global constant for the credit manager's email address. The reason for not using an event global constant in this case was simply to give you some experience with the **Event Action Parameter Recipients** form's other capabilities. (In most scenarios, a global constant will be used.)

- f. Click **Update**.

Notice that the system places the email address for the credit manager in the **Recipients** field.

- g. Click **OK**.

- h. In the **Subject** field, specify: **Credit limit change**

- i. In the **Category** field, specify: **Financial**

- j. Click the **Body** button.

- k. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **SUBSTITUTE**.

- l. In the **Argument 1** field, specify: **The credit limit has been changed to \${0} for customer {1}, customer number {2}**.

- m. Place the cursor in the first row of the **Arguments** grid, and click **Build Expression**.

- n. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **FILTERPROPERTY**.

- o. In the **Argument 1** field, specify **CreditLimit** and click **OK**.

- p. Place the cursor in the second row of the **Arguments** grid, and click **Build Expression**.

- q. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **FILTERPROPERTY**.

- r. In the **Argument 1** field, specify **Name** and click **OK**.

- s. Place the cursor in the third row of the **Arguments** grid, and click **Build Expression**.

- t. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **FILTERPROPERTY**.

- u. In the **Argument 1** field, specify **CustNum** and click **OK**.

- v. Click **OK**.

Notice that the system returns the entire SUBSTITUTE expression to the **Event Action Send Email** form, correctly formatted.

Notice also that there is no option to save the message to the user's **Sent Items** folder. This is because this notification is being sent as an email. That being the case, we cannot use the

SAVEMESSAGE parameter to have the system save a copy of the notification in the **Sent Items** folder of the person who added the new customer.

- w. Click **OK**.
 - x. Save the action and close the **Event Actions** form.
4. Verify that you have no syntax errors by clicking **Check Syntax**.
 5. Discard the cached metadata.

For more information, including the procedure, see Refreshing the metadata cache on page 54.

Test this event handler by changing a customer's credit limit and saving the record. The system should generate an email message that gets sent to the credit manager.

Points to note and remember

In creating this kind of event handler, keep these points in mind:

- To create an event handler that sends an email, you must have the SMTP set up on the **Intranets** form. Also, the email service on that computer must be set up to enable the relaying of email automatically.
- To have the handler do something only when certain conditions are met, use the Finish action type and the CONDITION(NOT PROPERTYMODIFIED) parameter and function.
- To eliminate the single quotes that appear around replacement values in the generated messages, use the PROPERTY function in place of the FILTERPROPERTY function we used in this scenario.

Scenario 3: Notification that includes an "old" value

In this scenario, we want to notify a group of inventory stock clerks automatically whenever an item's lot size changes. In the message that is sent, we want to include both the previous lot size and the new lot size. We also want to let them know who initiated the change.

Once again, we will use an existing framework event, IdoOnItemUpdate, and create our own handler for it. Because this handler needs to retrieve the "before" property values, we must make it synchronous, so that during handler execution we can retrieve from the database the original row that is being updated, before it is updated by the IDO request.

This event handler requires three actions:

- One to check whether the **Lot Size** field has been changed and, if not, finish.
- One to retrieve the row being updated and both the original and new values for the **Lot Size** field.
- One to send the notification to the inventory stock clerks.

To accomplish this scenario:

1. Create the event handler:
 - a. Open the **Event Handlers** form.
 - b. Press F3.

- c. Press CTRL+N.
- d. Create the handler with these settings:

Field or Option	Setting / Comments
Event Name	From the drop-down list, select IdoOnItemUpdate . Note: For details about this and the other framework events included with the system, see "Core application events" on page 126.
Applies to Initiators	Leave blank.
Applies to Objects	Specify SLItems . To determine what object you need, see the procedure provided in the online help for this field.
Keep With	Leave blank.
Chronology	Leave blank.
Ignore Failure	Cleared.
Suspend	Cleared.
Synchronous	Selected. Because this notification requires both the original value and the new value, it must be run synchronously. For more information, see "Synchronicity" on page 20.
Active	Selected.
Can Override	Selected.
Transactional	Cleared.
Obsolete	Cleared.
Initial State	Leave blank.
Initial Action	Leave blank.

- e. Save the handler.
2. Create an event global constant for the group of stock clerks.
Putting the user IDs for the entire group into a global constant allows us to change the list, which might be used in other places as well, in a single place easily.
 - a. Open the **Event Global Constants** form.
 - b. Press F3 or F4.
 - c. Press CTRL+N.
 - d. In the **Name** field, specify the name for the global constant, in this case, **StockClerks**.

-
- e. In the **Value** field, specify the user IDs for the stock clerks, separated only by semi-colons (;) and no spaces.
 - f. Save, and close the form.
 3. Create the first action, which checks the condition of the **Lot Size** field when the item record is saved:
 - a. In the **Event Handlers** form, select the handler.
 - b. Click **Event Actions**.
 - c. In the **Event Actions** form, in the **Action Sequence** field, specify **10**.
 - d. From the **Action Type** drop-down list, select **Finish**.
 - e. Click **Edit Parameters**.
 - f. In the **Event Action Finish** form, click the **Condition** button.
 - g. In the **Event Action Parameter Condition** form, click the **Expression 1** button.
 - h. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **PROPERTYMODIFIED**.
 - i. In the **Argument 1** field, specify **LotSize** and then click **OK**.

Note: What if you do not know the name of the property for which you want to retrieve a value? How do you find that property name? For an easy method to find the property name, see "Determining names of IDO collections and components" on page 53.
 - j. In the **Event Action Parameter Condition** form, select the **NOT** check box and then click **OK**.
 - k. In the **Event Action Finish** form, click **OK**.

This parameter tells the system to check the **LotSize** property. If it has *not* been modified, finish handler execution and exit. If it *has* been modified, continue with the second action.
 - l. Save the action.
 4. Create the second action, which retrieves both the original and new values for the **Lot Size** field.

As part of this step, the system retrieves the original value of the **Lot Size** field and stores the value to an event variable named **OldLotSize**.

For this action type to work, you must:

 - Use the IDO() function to identify the same IDO that fired the event.
 - Name the same property in the PROPERTIES() and SET() functions, and both should be the same as the field/component property value the user is changing.
 - Make sure the handler is synchronous.

To create the action:

 - a. Press CTRL+N.
 - b. In the **Action Sequence** field, specify **20**.
-

c. From the **Action Type** drop-down list, select **Load IDO Row**.

d. Click **Edit Parameters**.

e. In the **Event Action Load IDO Row** form, the **IDO** field, specify: **SLItems**

Note: You can also select the IDO you want from the drop-down list, but you might have to filter on the field or increase the record cap for drop-down lists to see this one.

Also, the procedure for figuring out what IDO collection you need is similar to the procedure for figuring out what property name you need. For more information, see "Determining names of IDO collections and components" on page 53.

f. In the **Properties** field, specify **LotSize**.

g. Click the **Filter** button.

h. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **SUBSTITUTE**.

i. In the **Argument 1** field, specify: **Item = {0}**

j. Place your cursor in the first **Arguments** row and then click **Build Expression**.

k. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **FILTERPROPERTY**.

l. In the **Argument 1** field, specify **Item** and then click **OK**.

m. Click **OK**.

n. Click the **Output** button.

o. In the first row of the **Event Action Output Parameters** form, from the **Output Type** drop-down list, select **Return Variable**.

p. In the **Output Object Name** field, specify: **OldLotSize**

q. In the **Value** field, specify **LotSize** and then click **OK**.

r. In the **Event Action Load IDO Row** form, click **OK**.

Your resulting syntax statement should appear like this in the **Parameters** field of the **Event Actions** form:

```
IDO("SLItems")
PROPERTIES("LotSize")
FILTER(SUBSTITUTE("Item = {0}", FP("Item")))
SET(RV(OldLotSize) = "LotSize")
```

s. Verify that the action has no syntax errors.

t. Save the action.

5. Create the third action, which sends the notification:

a. To create the action, press CTRL+N.

b. In the **Action Sequence** field, specify **30**.

-
- c. From the **Action Type** drop-down list, select **Notify**.
 - d. Click **Edit Parameters**.

Setting up the TO parameter:

- a. In the **Event Action Notify** form, click the **To** button.
- b. In the **Event Action Parameter Recipients** form, click **Recipients**.
- c. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **GC**.

The GC function allows you to designate an event global constant to use for the recipients. In this case, we will designate the global constant we created earlier, **StockClerks**.

- d. From the **Argument 1** drop-down list, select **StockClerks** and then click **OK**.
- e. In the **Event Action Parameter Recipients** form, click **OK**.

Setting up the CC, SUBJECT, and CATEGORY parameters:

- a. Click **Cc** button.
- b. In the **Event Action Parameter Recipients** form, click **Recipients**.
- c. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **ORIGINATOR**.

Notice that the ORIGINATOR function takes no arguments.

- d. Click **OK** twice.
- e. In the **Subject** field, specify: **Lot size change**
- f. In the **Category** field, specify: **Change Notification**

Setting up the BODY parameter:

- a. Click the **Body** button.
- b. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **SUBSTITUTE**.
- c. In the **Argument 1** field, specify: **The lot size has been changed for item: {1} The previous lot size was {2}, the new lot size is {3}. Please take note and adjust your activities accordingly. This change was made by {0}.**
- d. Place your cursor in the first row of the **Arguments** grid and then click **Build Expression**.
- e. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **ORIGINATOR** and then click **OK**.

Notice that we can place this first property (ORIGINATOR) last in the message, and, as long as we have the appropriate index number assigned ({0}), it will be correctly displayed in the message.

- f. Place your cursor in the second row of the **Arguments** grid and then click **Build Expression**.
- g. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **FP**.

-
- h. From the **Argument 1** drop-down list, select **Item** and then click **OK**.

In this substep, because we want the item number to be enclosed in single quote marks, we use the FP (alternate for the FILTERPROPERTY) function. However, we do not want single quote marks around the lot size amounts, so we will let those values evaluate as their native datatypes.

- i. Place your cursor in the third row of the **Arguments** grid and then click **Build Expression**.
- j. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **V** from the list of functions.
- k. In the **Argument 1** field, specify **OldLotSize** and then click **OK**.
- l. Place your cursor in the fourth row of the **Arguments** grid and then click **Build Expression**.
- m. In the **Event Action Expression Editor**, from the **Select a function** drop-down list, select **P** (or **PROPERTY**) from the list of functions.
- n. From the **Argument 1** drop-down list, select **LotSize** and then click **OK**.
- o. In the **Event Action Expression Editor**, click **OK**.

The resulting syntax statement looks similar to this:

```
BODY(SUBSTITUTE("The lot size has been changed for item: {1} The previous lot size  
was {2}, the new lot size is {3}. Please take note and adjust your activities accordingly.  
This change was made by {0}.",  
ORIGINATOR(),  
FP("Item"),  
V(OldLotSize),  
P("LotSize"))
```

Note: If you want, you can add line returns to make your syntax statement look like this example. The system ignores white space and line returns when processing the statements.

- p. In the **Event Action Notify** form, click **OK**.
6. Verify that you have no syntax errors.
 7. Save the action and close the **Event Actions** form.
 8. Discard the cached metadata.

For more information, including the procedure, see "Refreshing the metadata cache" on page 54.

Testing the event handler

To test this handler:

1. Open the **Items** form and locate an item that is lot-tracked.
2. On the **General** tab, change the value in the **Lot Size** field and save the record.
3. In the stockers' **Inbox** forms, verify that the message was sent and contains the correct values.

Points to note and remember

In creating this kind of event handler, keep these points in mind:

- To retrieve the previous (existing) value of a field for display in a message, you must make the handler synchronous.
- To display both the original value of a field and the new one, use an event variable to temporarily store the original value.
- Substituted values in a statement can be presented in any order as long as their index numbers match their positions in the list.
- To prevent single quotes from being placed around a substituted value, use the `PROPERTY` function instead of the `FILTERPROPERTY` function.

Extra challenge

Try on your own adding the item description to the body of the message. This might require you to determine the name of the item description property, if you do not already know it. For more information, see "Determining names of IDO collections and components" on page 53.

Requesting approvals

These two scenarios, which are similar, build on the concepts and practices used in the first two scenarios. The big difference is that they both use the Prompt action type to ask a manager to approve some action.

Note: Before trying these scenarios, we recommend that you go through the scenarios so far, if you have not already done so. These scenarios assume that you already know how to accomplish certain tasks without taking you through them in detail.

Scenario 4: Approval for a new record

In this scenario, we want to send a message to the purchasing manager whenever a purchase order (PO) is added. This message prompts the purchasing manager for approval of the PO. The purchasing manager can indicate approval (or disapproval) by clicking a button in the message itself. If approved, the system adds the PO; if not, the system does not add the PO.

As with the previous scenarios so far, the system framework has a built-in event, `IdoOnItemInsert`, which we can use with our handler. The handler itself requires two actions: one to send the prompt, and one to tell the system what to do if approval is denied.

To set the system up to handle this:

1. Create a global constant for the purchasing manager.
 - a. For the **Name** of the global constant, use **PurchasingMgr**.
 - b. For the **Value** field, specify the purchasing manager's user ID.

Remember: To add multiple recipients, specify user IDs separated by semi-colons (;) and no spaces.

2. Create and save an event handler with these settings:

Field or Option	Setting / Comments
Event Name	From the drop-down list, select IdoOnItemInsert . Note: For details about this and the other framework events included with the system, see "Core application events" on page 126.
Applies to Initiators	Leave blank.
Applies to Objects	Specify SLPos . To determine what object you need, see the procedure provided in the online help for this field.
Keep With	Leave blank.
Chronology	Leave blank.
Ignore Failure	Cleared.
Suspend	Cleared. Because this notification does require a response from the purchasing manager, it must run synchronously and be suspended. However, we cannot select this (and make it "stick") until at least one adjourning action exists. So, we must leave this cleared for now and come back to it after our actions have been defined. For more information, see "Suspension" on page 22.
Synchronous	Doesn't matter at this point. Because this notification does require a response from the purchasing manager, it must run synchronously and be suspended. When you later select the Suspend option, this will be automatically selected. For more information, see "Synchronicity" on page 20.
Active	Selected.
Can Override	Selected.
Transactional	Cleared.
Obsolete	Cleared.
Initial State	Leave blank.
Initial Action	Leave blank.

For more information about any of these fields and options, see the online help for the **Event Handlers** form.

3. Create the first action, to send the message:
 - a. In the **Event Actions** form, in the **Action Sequence** field, specify **10**.
 - b. From the **Action Type** drop-down list, select **Prompt** and then click **Edit Parameters**.

This action type not only sends a notification to the designated recipient, it also prompts the recipient for a response.
 - c. Starting with the **Event Action Prompt** form, use the associated forms to create these parameters:

Field / Button	Action	Result / Comments
To button	Click.	The Event Action Parameter Recipients form opens.
Recipients	Click button.	The Event Action Expression Editor opens.
Select a function	Select GC .	The Argument 1 field and drop-down list display.
Argument 1 drop-down list	Select PurchasingMgr . This is the global constant we created earlier. Click OK .	The system returns the expression to the Event Action Parameter Recipients form.
OK button	Click.	The system returns the expression to the Event Action Prompt form.
Subject field	Specify: New purchase order needs your approval	—
Category field	Specify: Order Approval	—
Body button	Click.	The Event Action Expression Editor form opens.
Select a function	Select SUBSTITUTE .	The Argument 1 field and drop-down list, Arguments grid, and buttons display.
Argument 1 field	Specify: A new purchase order has been requested for vendor {0}, {1}. Please review the details on the Variables tab and register your approval on the Response tab.	—
Arguments grid, row 1	With cursor in the field, click Build Expression .	The Event Action Expression Editor form opens.

Field / Button	Action	Result / Comments
Select a function	Select P or PROPERTY .	The Argument 1 field and drop-down list display.
Argument 1 field	Select VendNum and click OK .	The system returns the expression to the parent Event Action Expression Editor .
Arguments grid, row 2	With cursor in the field, click Build Expression .	The Event Action Expression Editor form opens.
Select a function	Select P or PROPERTY .	The Argument 1 field and drop-down list display.
Argument 1 field	Select VendorName and click OK .	The system returns the expression to the parent Event Action Expression Editor .
OK button	Click.	The system returns the entire BODY parameter content to the Event Action Prompt form.
Question field	Specify: Do you approve this new PO?	Note that, when the handler runs, the QUESTION parameter is presented on the Response tab of the recipient's Inbox .
Choices button	Click.	The Event Action Prompt Choices form opens. Note that the CHOICES parameter creates, displays, and enables the voting buttons that will be required for the purchasing manager to signal approval or rejection. When the handler runs, these CHOICES buttons appear directly beneath the QUESTION in the recipient's Inbox.
Return Value field, row 1	Specify: 1	Note that you can specify any value you want here.
Button Caption field, row 1	Specify: sYes	Note that you can use translatable strings from the Strings table. These strings appear in the drop-down list for this field.
Return Value field, row 2	Specify: 0	—
Button Caption field, row 2	Specify: sNo	—
OK button	Click.	The system returns the choices data to the Event Action Prompt form.
OK button	Click.	The system returns all defined parameters to the Event Actions form, correctly formatted.

- d. Verify that there are no syntax errors.
- e. Save the action.

If you have done everything correctly, your syntax for this action step should look like this:

```
TO(GC(PurchasingMgr))
SUBJECT("New purchase order needs your approval")
CATEGORY("Order Approval")
BODY(SUBSTITUTE("A new purchase order has been requested for vendor {0}, {1}. Please
review the details on the Variables tab and register your approval on the Response tab.",
P("VendNum"), P("VendorName") )
SAVEMESSAGE(FALSE)
QUESTION("Do you approve this new PO?")
CHOICES("1,sYes,0,sNo")
```

4. Create the second action, which tells the system how to respond if approval is not granted:
 - a. In the **Action Sequence** field, specify **20**.
 - b. From the **Action Type** drop-down list, select **Fail**.

This action type ends handler execution with an error status. This effectively aborts the process and prevents the PO from being added to the database.
 - c. Starting with the **Event Action Fail** form, use the associated forms to create these parameters:

Field / Button	Action	Result / Comments
Condition button	Click.	The Event Action Parameter Condition form opens.
Expression 1 button	Click.	The Event Action Expression Editor opens.
Select a function	Select VOTINGRESULT .	The Action drop-down list displays. This function and action evaluate the results of whatever action is selected from the list. The action refers by number to the action type. In this case, we have only one other action, and that is the correct action, the Prompt action.
Action drop-down list	Select 10 Prompt and click OK .	The system returns the expression to the Expression 1 field on the Event Action Parameter Condition form.
Operator drop-down list	Select = (equals) .	—

Field / Button	Action	Result / Comments
Expression 2 field	Specify: 0 (zero) and click OK .	This value tells the system to fail the handler with an error if the recipient responds with a "No" (0). The system returns the entire condition statement to the Event Action Fail form.
Result field	Specify: The PO request was rejected by the purchasing manager.	This message appears on the Event Status form if the purchasing manager responds with a "No."
OK button	Click.	The system returns both parameters to the Event Actions form, correctly formatted.

- d. Verify that there are no syntax errors and save the action.
 5. Return to the **Event Handlers** form and select the **Suspend** check box.
 6. Save the handler.
 7. Discard the cached metadata.
- For more information, including the procedure, see "Refreshing the metadata cache" on page 54.

Testing the event handler

To test this handler:

1. Open the **Purchase Orders** form and create a new purchase order. Save it.

After you save the PO, when the **Purchase Orders** form refreshes the display, the new record should disappear from the display. It remains hidden until/unless it has been approved.
If you do not assign a **PO** number, the generated message displays it with a PO number of **TBD**.
2. Open the **Inbox** form for the individual designated as the Purchasing Manager and verify that the message was received and that the **Response** tab displays the question and choice buttons.
3. (Optional) With the **Purchase Orders** form selected, from the **Actions** menu, select **View Event Status**.

This opens the **Event Status** form. Navigate to the last row and verify that the status for this event is **Running**.
4. In the **Inbox** form, click the button labeled **Yes**.
5. Refresh the collection on the **Purchase Orders** form and verify that the new PO now displays in the list.

You can also do a second test, clicking the button labeled **No** to reject the request. In this case, when you refresh the **Purchase Orders** form, the new PO record is never added to the database and does not appear in the list of POs.

Points to note and remember

When creating this kind of event handler, keep these points in mind:

- When creating a message that requires a response from the recipient (usually a Prompt action type), you must mark the handler so that it suspends when executed. This means that it is also automatically marked as a synchronous handler.
- Because these event handlers must be suspended, pending the purchasing or credit manager's response, the Framework Event Service must be enabled for the configuration in which you are logged on.

Extra Challenge

Try changing the **Subject** line so that it displays the user ID of the person who created the new PO and the PO number.

Hint: You will need to use the SUBSTITUTE function.

Scenario 5: Requesting approval by external email for changes to an existing record

This scenario is similar to the one in Scenario 2: Notification of changes to an existing record—changing the credit limit on page 78, except that in this case, we want the credit manager's approval for the credit limit change, and we are sending the request to the manager's external email address. If the credit manager approves the change, the system writes and saves it. If the credit manager does not approve the change, the system rolls back the record to the previously approved credit limit.

As with the other scenarios so far, we can use existing framework events and IDOs to accomplish this. This time, however, because we are sending out a prompt and requiring a response from the credit manager, we must make it a synchronous *and* suspending event handler.

This event handler requires three actions:

- One checks whether the **Credit Limit** field has been changed. If it has *not*, it finishes with a status of Success.
- One sends the prompt message and external email.
- If the credit manager does *not* approve the change, the third one fails the event and rolls back the record.

To accomplish this scenario:

1. Set up the recipient in the **Users** form to allow external email and to have the appropriate default language code.
 - Select **Send Email Prompts**
 - Ensure that the **Email Address** is correct
 - Specify the **Default Language** to use for formatting text strings
2. Create an event handler with these settings:
 - **Event Name** = **IdoOnItemUpdate**

-
- **Applies to Objects = SLCustomers**

3. Save the handler.

Note: If you still have the handler you created for Scenario 2: Notification of changes to an existing record—changing the credit limit on page 78 active, it is a good idea to clear the **Active** check box or mark it as **Obsolete**, so that this event handler and that one do not create duplicate and possibly confusing messages.

4. Create the first action, which checks the condition of the **Credit Limit** field when the customer record is saved:

- Click **Event Actions** for the handler you just created.
- In the **Event Actions** form, create a new action with these settings:
 - **Action Sequence = 10**
 - **Action Type = Finish**
- Click **Edit Parameters**.
- Starting with the **Event Action Finish** form, use the associated forms to create these parameters:

Field / Button	Action	Result / Comments
Condition button	Click.	The Event Action Parameter Condition form opens.
Expression 1 button	Click.	The Event Action Expression Editor opens.
Select a function	Select PROPERTYMODIFIED .	The system displays the Argument 1 button and field.
Argument 1 field	Specify: CreditLimit Then click OK .	The system returns the expression to the parent Event Action Parameter Condition form and disables the Operator and Expression 2 options.
NOT check box	Select. Then click OK .	This tells the action to finish with a status of Success if the Credit Limit field has <i>not</i> been changed. The system returns the expression to the Event Action Finish form.
OK button	Click.	The system returns the entire parameter to the Event Actions form, correctly formatted.

e. Verify that the syntax is correct.

f. Save the action.

5. Create the second action, to send the prompt message.

This action sends the prompt to the credit manager, through both the **Inbox** form and external email, and suspends the handler until the credit manager responds to the request.

- a. In the **Event Actions** form, create a new action with these settings:
 - **Action Sequence = 20**
 - **Action Type = Prompt**
- b. Click **Edit Parameters**.
- c. Starting with the **Event Action Prompt** form, use the associated forms to create these parameters:

Field / Button	Action	Result / Comments
To button	Click.	The Event Action Parameter Recipients form opens.
Recipients button	Click.	The Event Action Expression Editor form opens.
Select a function	Select GC .	The system displays the Argument 1 button and field.
Argument 1 drop-down list	Select CreditMgr . Then click OK .	The system returns the expression to the Event Action Parameter Recipients form.
OK button	Click.	The system returns the expression to the Event Action Prompt form.
Subject button	Click.	The Event Action Expression Editor form opens.
Select a function	Select SUBSTITUTE .	The system displays the buttons and fields associated with the SUBSTITUTE function.
Argument 1 field	Specify: Credit limit change request for customer ID: {0}	Notice that the SUBSTITUTE function is being used to present the customer's ID number in the Subject line, so that messages can be saved and tracked more easily.
Arguments grid, row 1	Place the cursor in the field and then click Build Expression .	The Event Action Expression Editor form opens.
Select a function	Select P or PROPERTY . Note: These are equivalent functions.	The system displays the Argument 1 button and field.
Argument 1 drop-down list	Select CustNum . Then click OK .	The system returns the expression to the parent Event Action Expression Editor form.
OK button	Click.	The system returns the entire expression to the Event Action Prompt form.

Field / Button	Action	Result / Comments
Category field	Specify: Financial	—
Body button	Click.	The Event Action Expression Editor form opens.
Select a function	Select SUBSTITUTE .	The system displays the buttons and fields associated with the SUBSTITUTE function.
Argument 1 field	Specify: You have a request for a credit limit change to \${0} for {1}, Customer ID {2}. Please respond to the question and indicate your approval on the Response tab.	This sets up the basic message with three replacement markers.
Arguments grid, row 1	Place the cursor in the field and then click Build Expression .	The Event Action Expression Editor form opens.
Select a function	Select P .	The system displays the Argument 1 button and field.
Argument 1 drop-down list	Select CreditLimit . Then click OK .	The system returns the expression to the first row of Arguments grid on the parent Event Action Expression Editor form.
Arguments grid, row 2	Place the cursor in the field and then click Build Expression .	The Event Action Expression Editor form opens.
Select a function	Select P .	The system displays the Argument 1 button and field.
Argument 1 drop-down list	Select Name . Then click OK .	The system returns the expression to the second row of Arguments grid on the parent Event Action Expression Editor form.
Arguments grid, row 3	Place the cursor in the field and then click Build Expression .	The Event Action Expression Editor form opens.
Select a function	Select P .	The system displays the Argument 1 button and field.
Argument 1 drop-down list	Select CustNum . Then click OK .	The system returns the expression to the third row of Arguments grid on the parent Event Action Expression Editor form.
OK button	Click.	The system returns the entire SUBSTITUTE expression to the Event Action Prompt form.

Field / Button	Action	Result / Comments
Question field	Specify: Do you approve this credit limit change?	Note that you have an 80-character limit in the Question field.
Choices button	Click.	The Event Action Prompt Choices form opens.
Return Value , row 1	Specify 1 .	Tip: Theoretically, you can use any value you want here, as long as you remember what it is and use the same value later in the Fail action step.
Button Caption , row 1	Specify sYes .	Notice that this is a translatable string from the Strings table. Also notice that you can select a string from the Strings table from the drop-down list.
Return Value , row 2	Specify 0 .	—
Button Caption , row 2	Specify sNo .	—
OK button.	Click.	The system returns the Choices values to the Event Action Prompt form.
Save in Sent Items	Select.	This sends a copy of the message to whomever initiated the credit limit change.
OK button	Click.	The system returns the entire set of Prompt action parameters to the Event Actions form.

Note: You could incorporate other field values from the **Event Action Prompt** form before saving and closing, but in the interest of relative brevity, we will finish with what we have here.

- d. Verify that the syntax is correct.
 - e. Save the action.
6. Create the third action, which tells the system how to respond if approval is not granted.

This parameters tells the system to consider the action as having failed if the credit manager rejects the credit limit change. In other words, if the credit manager votes "No" [0] on the second action (**Action Sequence = 20**), then this action fails.

 - a. In the **Event Actions** form, create a new action with these settings:
 - **Action Sequence = 30**
 - **Action Type = Fail**

This action type ends handler execution with an error status. This effectively aborts the process and prevents the credit limit from being changed for the customer.

- b. Click **Edit Parameters**.
- c. Starting with the **Event Action Fail** form, use the associated forms to create these parameters:

Field / Button	Action	Result / Comments
Condition button	Click.	The Event Action Parameter Condition form opens.
Expression 1 button	Click.	The Event Action Expression Editor opens.
Select a function	Select VOTINGRESULT .	The system displays the Action drop-down list field.
Action drop-down list	Select 20 Prompt .	Notice that the system displays only the 20 in the field. The action type name in the drop-down list is there to help you select the correct action step.
OK button	Click.	The system returns the expression to the Event Action Parameter Condition form.
Operator drop-down list	Select = (Equals) .	—
Expression 2 field	Specify 0 (zero).	NOTE: This is a reference to the value you designated for a disapproval (sNo). If you had used some value other than 0, that is what you would specify here.
OK button	Click.	The system returns the expression to the Event Action Fail form.
Result field	Specify: The credit manager has disapproved the credit limit change.	This is the text that appears in the Result field of the Event Status form if the credit manager disapproves the change. Since that is the only time and place this message is displayed, you can use a literal value. To make it clearer, you could (and probably should) use a SUBSTITUTE function with the customer name and ID number.

Note: You could also at this point set another event action to notify the original sender by message that the change has been approved or disapproved, but that is beyond the scope for this exercise.

- d. Verify that there are no syntax errors.
 - e. Save the action and close the **Event Actions** form.
7. Return to the **Event Handlers** form and select the **Suspend** check box.

-
8. Save the handler.
 9. Refresh the cached metadata.

For more information, including the procedure, see "Refreshing the metadata cache" on page 54.

Testing this event handler

To test this handler:

1. Open the **Customers** form and change the credit limit for a customer. Save it.

Notice that the entire record for this customer is now temporarily disabled, because the update has been suspended pending approval. Therefore, no further changes can be made to this customer record until this event is resolved one way or the other.

Also notice that, at this point, all fields, including the **Credit Limit** field, display their original values. Anyone (including you) who views this suspended record sees the original values, until this suspended event finishes successfully, at which time the new values are saved in the database and displayed on the **Customers** form.
2. (Optional) Open the **Saved Messages** form for your current logon ID and verify that a copy of the message has been saved there.
3. Open the **Inbox** form for the individual designated as the credit manager and verify that the message was received and that the **Response** tab displays the question and choice buttons.
4. Open the credit manager's external email system and verify that the email was received, and the question and choice links display.
5. (Optional) With the **Customers** form open and customer record that was changed selected, from the **Actions** menu, select **View Event Status**.

This opens the **Event Status** form. Verify that the status for this event is **Running**. You can also open the **Event Status** form manually.
6. In the credit manager's email, click the link labeled **Yes**. Verify that the ASP processes the message and returns a success response.
7. In the credit manager's **Inbox** form, verify that the message is automatically marked as **Expired**, the **Choices** buttons are now disabled, and the Selected Choice is **Yes**.
8. Refresh the collection on the **Customers** form and verify that the new credit limit was saved.

Notice too that the entire customer record is once again enabled for editing.

You can also do a second test, clicking the link labeled **No** to reject the request. In this case, when you refresh the **Customers** form, notice that the **Credit Limit** field has retained its original amount.

Points to note and remember

In creating this kind of event handler, keep these points in mind:

-
- You can use the SUBSTITUTE function other places than just in the body of a message. You can use it in the Subject line and other places. And, as we will see later, you can also use it for purposes other than replacing text in messages.
 - When checking on a voting result, the number referred to in the syntax is the action sequence number for the action that contains the choice.

Extra challenges

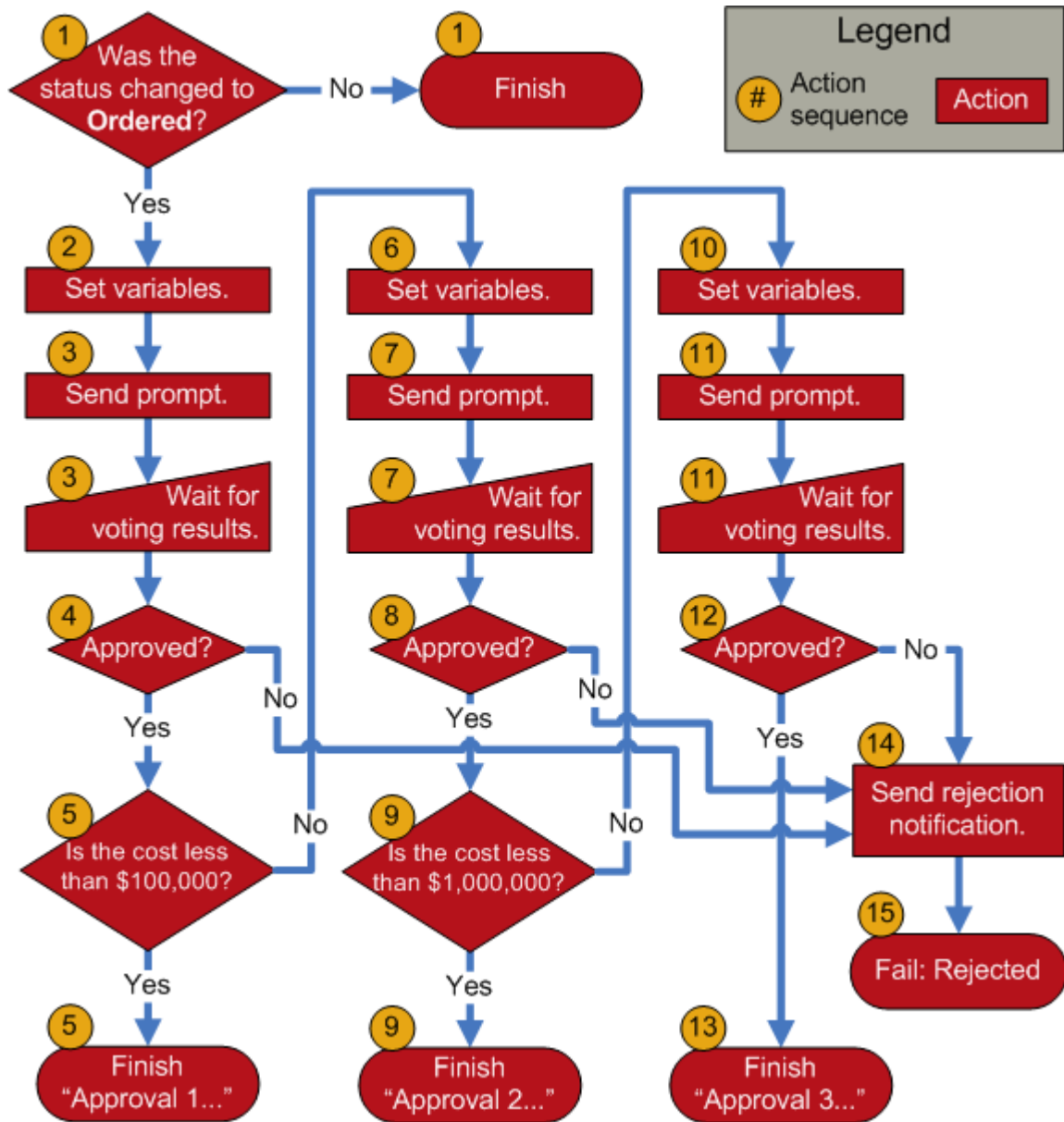
- Change the body of the message to include both the original credit limit and the proposed new limit.
Hint: You will need to save the old credit limit in an event variable.
- Create another event action to notify the original sender by message that the change has been approved or disapproved.

Scenario 6: Requesting multiple and complex approvals

In this scenario, when a purchase order (PO) status is changed to **Ordered**, it requires the approval of the purchasing manager. At the same time, if the PO is for more than \$100, 000, the PO also requires the approval of the purchasing manager's supervisor. Finally, if the PO is for more than \$1,000,000, the PO requires the further approval of two senior executives.

If the PO is disapproved at any level, the PO is rolled back to the previous values, and any changes made to it are lost. While it is in the process of being approved, the PO remains suspended until approval or disapproval is determined. This means, among other things, that if one or more approvers fail to respond to the request, the PO is locked and cannot be changed until all required approvers respond.

This flow diagram illustrates what must happen with this handler:



For this scenario we will:

- Use the same global constant for the purchasing manager (**PurchasingMgr**) that we used for a previous scenario (Scenario 4: Approval for a new record on page 88).
- Use the `IdoOnItemUpdate` framework event for the **SLPos** IDO. This will cause the event to be generated whenever a PO record is updated either in the **Purchase Orders** form or by another process that attempts to update that record.
- Pass property values and an identifying property (**ItemId**) to the event as input parameters so the system can store them with the event as event parameters.

To accomplish this scenario:

1. Create and save an event handler with these settings:
 - **Event Name = IdOnItemUpdate**
 - **Applies to Objects = SLPos**
2. Create the require event global constants to make this scenario work. In the **Event Global Constants** form, create these global constants:

Name	Value	Comments
PurchasingMgr	<i>userID1</i>	This is the logon user ID for the purchasing manager required for initial approval. This can be the same as the one we created for Scenario 4: Approval for a new record on page 88. If you have multiple recipients, separate them with semi-colons (;) and no spaces.
PurchasingSuper	<i>userID2</i>	This is the logon user ID for the purchasing supervisor required to approve POs over \$100,000.
PurchasingSenior	<i>userID3;userID4</i>	These are the logon user IDs for the two senior executives required to approve POs over \$1,000,000.
SuperCost	100000	This global constant represents the minimum amount that must be approved by a supervisor. We are using a global constant for it so that it can be changed globally at some future time. The value is a literal amount, no commas.
SeniorCost	1000000	This global constant represents the minimum amount that must be approved by two senior-level executives. We are using a global constant for it so that it can be changed globally at some future time. The value is a literal amount, no commas.
POApprovalPrompt	[See below.]	Note that, because we want to use the same basic prompt for all levels of approvals, we are putting it into a global constant. This is accomplished with the use of a few variables.

Value for POApprovalPrompt:

```
SUBJECT(" Purchase Order Update Approval Needed")
CATEGORY("Order Approval")
TO(GC(TV(Approver)))
BODY(SUBSTITUTE("A purchase order, {0}, has been updated to Ordered status for vendor,
{1}, number: {2}. Please review the details on the Variables tab and then indicate your
approval on the Response tab.",
FP("PoNum"),
P("VendorName"),
FP("VendNum")))
QUESTION("Do you approve this PO change?")
CHOICES("1, sYes, 0, sNo")TV(CountMethod)
FILTERFORM("PurchaseOrders")
FILTER( SUBSTITUTE("PoNum={0}", FP("PoNum"))))
```

3. Using the **Event Actions** form, add the first event action:

- **Action Sequence = 1**
- **Action Type = Finish**
- a. Starting with the **Edit Parameters** button, use the event action parameter forms to complete the event action:

Field / Button	Action	Result / Comments
Condition button	Click.	The Event Action Parameter Condition form opens.
Expression 1 button	Click.	The Event Action Expression Editor opens.
Select a function	Select PROPERTYMODIFIED .	The system displays the Argument 1 button and field.
Argument 1 field	Specify Stat .	—
OK button	Click.	The system returns the expression to the Event Action Parameter Condition form.
NOT check box	Select.	—

Field / Button	Action	Result / Comments
Condition field	Add this text: OR P("Stat") <> "O"	Note: The reason this is required is that the Event Action Parameter Condition form can only be used to construct simple condition statements. For complex conditions, you can start with that form, but you must then manually edit the condition statement. The "O" in this case is the capital letter, not a zero. The final result of this condition is: If the Stat property has <i>not</i> been modified, or if the value of the Stat field is not O , then finish. Or, in other words, if the Stat property has been modified <i>and</i> the value of the field is O , then continue to the next action.
OK button	Click.	The system returns the expression to the Event Action Finish form.

b. Check the syntax and save the action.

4. Add the second event action:

- **Action Sequence = 2**
- **Action Type = Set Values**

a. Starting with the **Edit Parameters** button, use the event action parameter forms to complete the event action:

Field / Button	Action	Result / Comments
Variables button	Click.	The Event Action Set Name/Value Pairs form opens.
Variable Name column, row 1	Specify: Approver	—
Value column, row 1	Click Build Expression .	The Event Action Expression Editor opens.
Select a function drop-down list	Select GC .	The form displays the Argument 1 button and drop-down list.
Argument 1 drop-down list	Select PurchasingMgr and then click OK .	The system returns to the Event Action Set Name/Value Pairs form.
Variable Name column, row 2	Specify: CountMethod	—
Value column, row 2	Specify: VOTINGRULE(Plurality)	Although this looks like a function, the system in this case does not treat VOTINGRULE() like the other functions.

Field / Button	Action	Result / Comments
OK button.	Click.	The system returns to the Event Action Set Values form.
OK button.	Click.	The system returns the parameters to the Event Actions form, correctly formatted.

This action step sets the values of two variables as follows:

- The variable named **Approver** is set to the value of the global constant, *PurchasingMgr*, which is the user ID for the purchasing manager.
- The variable named *CountMethod* is set to count the votes using the **Plurality** rule, which simply says that the choice with the greatest number of votes wins. Since we have only one individual set to vote at this point, the purchasing manager's vote alone determines what happens next.

Note: If you have more than one user ID in the *PurchasingMgr* global constant, you might want to use a different voting rule. For more information about voting rules, see [Voting Rules](#) on page 5-5.

b. Check the syntax and save the action.

5. Add the third event action:

- **Action Sequence = 3**
- **Action Type = Prompt**

a. In the **Parameters** text input field, specify this:

TGC(POApprovalPrompt)

Note: Because TGC() is a pre-parser function, you cannot use the event action parameter forms to create or set it. You must specify this statement directly in the **Parameters** field of the **Event Actions** form. This also means that you cannot use the **Check Syntax** button to check the syntax.

This statement performs a text evaluation of the *POApprovalPrompt* global constant to obtain the parameters for a prompt action. Part of this text evaluation includes an evaluation and insertion of the values for the two variables we set in the previous step.

After evaluating the *POApprovalPrompt* global constant, this action also sends out the prompt message to the purchasing manager and suspends the handler pending the manager's response. Because we did not specify any Variable Access rules, the message allows the purchasing manager to modify any variable values before approving it.

b. Save the action.

6. Add the fourth event action:

- **Action Sequence = 4**
- **Action Type = Branch**

- a. Starting with the **Edit Parameters** button, use the event action parameter forms to complete the event action:

Field / Button	Action	Result / Comments
Condition button	Click.	The Event Action Parameter Condition form opens.
Expression 1 button	Click.	The Event Action Expression Editor opens.
Select a function drop-down list	Select VOTINGRESULT .	The Action drop-down list appears.
Action drop-down list	Select 3 Prompt .	The Action field displays 3 .
OK button	Click.	The system returns to the Event Action Parameter Condition form.
Operator drop-down list	Select <> .	—
Expression 2 field	Specify: 1	—
OK button	Click.	The system returns to the Event Action Branch form.
Destination drop-down list	Specify: 14	Even though we have not yet created action Sequence 14, we can specify the number here. This is the action to which we will eventually be jumping if the purchasing manager rejects the request.
OK button	Click.	The system returns the action parameters to the Event Actions form, correctly formatted.

This action step evaluates the purchasing manager's response (from action Sequence 3) and directs the handler to the next action depending on that response. If the manager approves the request, the handler continues to the next action. If the manager rejects the request, the system goes to the destination, action Sequence 14, and continues from there.

- b. Check the syntax and save the action.

7. Add the next event action:

- **Action Sequence = 5**
- **Action Type = Finish**

Starting with the **Edit Parameters** button, use the event action parameter forms to complete the event action:

Field / Button	Action	Result / Comments
Condition button	Click.	The Event Action Parameter Condition form opens.
Expression 1 button	Click.	The Event Action Expression Editor opens.

Field / Button	Action	Result / Comments
Select a function drop-down list	Select P .	The Argument 1 button and field display.
Argument 1 field	Specify: POCost .	This property name is derived from the Purchase Orders form.
OK button	Click.	The system returns the statement to the Event Action Parameter Condition form.
Operator drop-down list	Select < .	—
Expression 2 button	Click.	The Event Action Expression Editor opens.
Select a function drop-down list	Select GC .	The Argument 1 button and field display.
Argument 1 drop-down list	Select SuperCost .	—
OK button	Click.	The system returns the statement to the Event Action Parameter Condition form.
OK button	Click.	The system returns the condition parameter to the Event Action Finish form.
Result field	Specify: Approved by Purchasing Manager .	—
OK button	Click.	The system returns to the Event Actions form with the parameters correctly formatted.

This action step determines whether the cost of the PO is less than \$100,000, the value of the *SuperCost* global constant. If it is, then the handler commits the PO record to the database, writes the result to the event state (viewable on the **Event States** form), and finishes with a status of **Success**. If the PO cost is \$100,000 or greater, then the handler continues to the next action.

8. Add the next event action:

- **Action Sequence = 6**
- **Action Type = Set Values**

This action step is similar to action Sequence 2, the difference being:

- The variable named *Approver* is set to the value of the global constant, *PurchasingSuper*, which is the user ID for the purchasing manager's supervisor.
- The variable named *CountMethod* is set to count the votes using the **Majority** rule, which says that any choice that gets more than 50% of the vote wins. If we had three supervisors voting, for instance, whichever choice gets the first two votes determines the outcome. Since we have only one individual set to vote at this point, the purchasing supervisor's vote alone determines what happens next.

9. Add the next event action:

- **Action Sequence = 7**

- **Action Type = Prompt**

In the **Parameters** text input field, specify this:

TGC(POApprovalPrompt)

As in action Sequence 3, this action step performs a text evaluation of the *POApprovalPrompt* global constant to obtain the parameters for a prompt action. This time, the prompt uses the new variable values for the purchasing supervisor that were set in Step 8.

After evaluating the *POApprovalPrompt* global constant, this action sends out the prompt message to the purchasing supervisor and suspends the handler again, pending the supervisor's response. Again, because we did not specify any Variable Access rules, the message allows the purchasing supervisor to modify any variable values before approving it.

10. Add the next event action:

- **Action Sequence = 8**
- **Action Type = Branch**

This action is similar to action Sequence 4, with the sole difference being that for the `VOTINGRESULT( )` expression, we look at action Sequence 7 instead of Sequence 3.

This action, then, evaluates the purchasing supervisor's response (from Action Sequence 7). As soon as any choice has a majority (more than 50% of the votes), the system directs the handler to the next action depending on that response.

In this case, if the supervisor approves the request, the handler continues to the next action. If the supervisor rejects the request, the system goes to the destination, action Sequence 14, and continues from there.

11. Add the next event action:

- **Action Sequence = 9**
- **Action Type = Finish**

This action is similar to action Sequence 5, with these differences:

- The global constant to use for the condition expression is *SeniorCost*, instead of *SuperCost*.
- The **Result** statement should read: **Approved by both the purchasing manager and the purchasing supervisor.**

This action step determines whether the cost of the PO is less than \$1,000,000, the value of the *SeniorCost* global constant. If it is, then the handler commits the PO record to the database, writes the result to the event state (viewable on the **Event States** form), and finishes with a status of **Success**. If the PO cost is \$1,000,000 or greater, then the handler continues to the next action.

12. Add the next event action:

- **Action Sequence = 10**
- **Action Type = Set Values**
SETVARVALUES(Approver="PurchasingSenior",
CountMethod="VOTINGRULE(MinimumPercentage) MINIMUM(100)")

This action step is similar to action Sequences 2 and 6, the differences being:

- The variable *Approver* is set to the value of the global constant, *PurchasingSenior*, which is the user IDs for the senior-level executives who must approve requests over \$1,000,000.

-
- The variable named *CountMethod* is set to count the votes using the **MinimumPercentage** rule, which says that the first choice to reach a minimum percentage determines the next action. In this case, the minimum percentage is 100%, so all recipients must approve for the PO to reach final approval. If any recipient rejects the request, the entire request is rejected, no matter who has approved it to that point.

TIP: The **MinimumPercentage** voting rule requires that you specify a minimum percentage for passage. This means that, in addition to the **VOTINGRULE()** keyword, you must also specify the **MINIMUM()** keyword as part of the variable definition. The resulting declaration for the **Value** column of the *CountMethod* variable is: **VOTINGRULE(MinimumPercentage) MINIMUM(100)**

13. Add the next event action:

- **Action Sequence = 11**
- **Action Type = Prompt**

In the **Parameters** text input field, specify this:

TGC(POApprovalPrompt)

As in action Sequences 3 and 7, this action step performs a text evaluation of the *POApprovalPrompt* global constant to obtain the parameters for a prompt action. This time, the prompt uses the new variable values for the senior-level executives that were set in the previous step.

After evaluating the *POApprovalPrompt* global constant, this action sends out the prompt message to the senior executives. Again, because we did not specify any Variable Access rules, the message allows the executives to modify any variable values before approving it.

14. Add the next event action:

- **Action Sequence = 12**
- **Action Type = Branch**

This action is similar to action Sequences 4 and 8, with the difference being that for the **VOTINGRESULT()** expression, we look at action Sequence 11.

This action step evaluates the senior executives' responses (from action Sequence 11). If both executives vote to approve the request, then the handler moves on to the next action. If either or both of them vote to reject the request, then the handler goes to the destination, action Sequence 14, and continues from there.

15. Add the next event action:

- **Action Sequence = 13**
- **Action Type = Finish**

- a. In the **Event Action Finish** form (reached by clicking **Edit Parameters**), in the **Result** field, specify: **Purchase order change approved by senior purchasing executives.**

This action sequence commits the PO record to the database, writes the result to the event state (viewable on the **Event States** form), and finishes with a status of **Success**.

- b. Save all actions.

16. Add the next event action:

- **Action Sequence = 14**

- **Action Type = Notify**

For any result which ends up in a disapproval of the request change, this action step sends a notification message to the individual who made the original change to the PO status, letting that individual know that the change request has been disapproved at some level.

- a. Use the event action parameter forms to create this notification message event action:

```
TO(ORIGINATOR())  
SUBJECT(SUBSTITUTE("Purchase order {0} change request disapproved",  
FP("PoNum")))  
CATEGORY("Notification")  
BODY(SUBSTITUTE("Your purchase order change request for PO {0} for {1}, vendor number:  
{2} has been disapproved. If you have questions, please see the required approvers.",  
FP("PoNum"),  
P("VendorName"),  
FP("VendNum")))  
SAVEMESSAGE(FALSE)
```

- b. Check the syntax and save the action.

17. Add the final event action:

- **Action Sequence = 15**

- **Action Type = Fail**

In the **Event Action Fail** form, set the **Result** field to:

PO change not approved.

This action step writes the result of the rejection to the event state (viewable on the **Event States** form), and exits with a status of **Failure**.

18. Now that all event actions (including the requisite adjourning actions) have been created and saved, go back to the **Event Handlers** form and select the **Suspend** check box.

19. Save the handler.

20. (Optional) Click the **Diagram** button to view the diagrammatic view of the event handler flow in the **Event Handler Diagram** form.

21. Discard the cached metadata.

For more information, including the procedure, see Refreshing the metadata cache on page 54.

Test 1 for this handler

In the first test, create a purchase order for less than \$100,000 and let the Purchasing Manager disapprove it.

Expected result: The PO status is not changed to **Ordered**.

To perform this test:

1. On the **Purchase Orders** form create a new PO and save it.
2. On the **Purchase Order Lines** form, create a line for an amount of less than \$100,000.

-
3. On the **Purchase Orders** form, change the status for the line you just created to **Ordered** and save the PO.

Notice that the PO record is disabled because it is now in a suspended state. The status appears to revert to **Planned**, because it has not yet been approved and thus, it has not yet actually been changed in the database.

4. Logged in using the Purchasing Manager's user ID, open the **Inbox** form.
5. Read the new prompt message that the system generated, and on the **Response** tab, select the **No** option.
6. Open or refresh the **Purchase Orders** form, and verify that the new PO line status is still **Planned** and that the PO is again enabled for changes.

Test 2 for this handler

This is the same as Test 1, except this time have the purchasing manager approve the change. Verify that the status is changed to **Ordered** and the PO is again enabled for change.

Test 3 for this handler

Create a PO with a total cost of between \$100,000 and \$1,000,000. Have the purchasing supervisor disapprove and verify that the change is rolled back.

Test 4 for this handler

Do the same as Test 3, except have both the purchasing manager and purchasing supervisor approve the change. Verify that the change is written to the database and that the record is again enabled for other changes.

Additional tests

For complete thoroughness, other tests could (and should) be devised and conducted before making this event handler live on an active system.

For example, you could change the status to something other than **Ordered** and make sure that the PO change does not suspend. You should also test for the senior executive approvals and disapprovals.

Points to note and remember

- Whenever possible, use the event action parameter forms (accessible by way of the **Edit Parameters** button). This is your best insurance against syntax errors.

-
- Not all actions can be created using the event action parameter forms. This is particularly true of the pre-parser functions. In these cases, you cannot check the syntax for these actions using the **Check Syntax** button.
 - You can create whole actions using global constants. You cannot subsequently check these for syntax errors, though, using the **Check Syntax** button, so proceed with caution.

Extra challenges

- Add the user ID to the Result statements for both approvals and disapprovals.
- Have the system generate a message to the user who initiated the request notifying that user that the request has been approved.

Modifying records

Scenario 7: Adding information to a record

In this scenario, we want to notify a credit manager that a new customer is being added and ask the credit manager to determine what the credit limit will be for that customer. So, we must send a notification that prompts the credit manager for a response and then uses the data from that response to add data to the pending new customer record and commit the changes to the database.

For this scenario we will:

- Use the same global constant (*CreditMgr*) we used for previous scenarios.
- Use the `IdoOnItemInsert` framework event.

To accomplish this scenario:

1. Create and save the event handler with these settings:
 - **Event Name** = `IdoOnItemInsert`
 - **Applies to Objects** = `SLCustomers`
2. Create one action, which sends a prompt to the credit manager requesting a response:
 - **Action Sequence** = `10`
 - **Action Type** = `Prompt`
 - a. Use the **Event Action Prompt** form and associated event action parameter forms to create these parameters:
 - **TO** = `GC(CreditMgr)`
 - **SUBJECT** = `SUBSTITUTE("Need credit limit for customer ID {0}, {1}", FP("CustNum"), P("Name"))`
 - **CATEGORY** = `"Financial"`
 - **BODY** = `SUBSTITUTE("Please use the Variables tab to provide a credit limit for customer ID {0}, {1}. Then use the Post button on the Response tab to register the new credit limit.", FP("CustNum"), P("Name"))`

-
- **SAVEMESSAGE = FALSE**
 - **QUESTION = To post the credit limit, click the button below.**
 - **CHOICES = 1,sPost**
 - **FILTERFORM = Customers**
 - **FILTER = SUBSTITUTE("CustNum={0}", FP("CustNum"))**

b. On the **Variable Access** tab:

- **Name = CreditLimit**
- **Access = Mandatory**

This forces a response from the credit manager.

c. Save the action and close the **Event Actions** form.

3. Return to the **Event Handlers** form and select the **Suspend** check box.
4. Save the handler.
5. Discard the cached metadata.

For more information, including the procedure, see "Refreshing the metadata cache" on page 54.

Testing the event handler

1. Open the **Customers** form and create a new customer record. Save it.

Notice that the newly saved record does not appear in the list of customers at this point when you refresh the **Customers** form.

2. Using the credit manager's logon, open the credit manager's **Inbox**.

The new message should appear in the **Inbox**.

3. On the **Variables** tab, locate the *Row.CreditLimit* variable and specify an amount.
4. Save the record.
5. On the **Response** tab, click **Post**.
6. Back on the **Customers** form, refresh the form and verify that the newly created record appears in the list.

You can (and should) also devise other test to verify that the system behaves as expected when the credit manager posts the response without specifying a value in the *Row.CreditLimit* variable field.

Points to note and remember

When creating a handler like this one, keep these points in mind:

- Because the variables are listed on the **Variables** tab and the question and response buttons are on the **Response** tab of the message, you should design your message body to include brief but detailed instructions for responding to the request. Do not assume the recipient will know or remember.

-
- Because we did not specify variable access rules to address property variables other than the *CreditLimit* variable, all the variable property values associated with the **Customers** form are displayed and writable. That means that the credit manager, if desired, can change any variable data before saving the data and posting it to the database by clicking **Post**. (To make other variables non-writable, you must set the variable access for each individually.)
 - The fact that the prompt message was sent to a single recipient (in this case) means that only one vote is required for a quorum, and once the credit manager posts the response, the vote is final and the database is updated. If there are multiple recipients associated with the *CreditMgr* global constant, then you might also need to set voting rules to determine how the responses will be handled. In this case, it is not necessary, because the system assumes a Plurality voting rule, and with only one recipient, that means that the first to respond is the one whose data is committed.
 - If the credit manager never votes, the record is never committed to the database, but it remains adjourned indefinitely.
 - The QUESTION parameter has a limit of 80 characters.

Voting

Scenario 8: Voting for various choices

In this scenario, we need several managers at the same level to approve an engineering change, by means of a response to a message. So, we must send a notification that prompts the managers for a response. If at least two of the managers send responses approving the change, we then approve the requested change in the application.

For this scenario we will:

- Assume that global constants were created for *EngineeringMgr*, *ProjectMgr*, and *ProgramMgr*. The creation of global constants is described in previous scenarios.
- Use the `IdoOnItemUpdate` framework event.

To accomplish this scenario:

1. Create and save the event handler with these settings:
 - **Event Name** = `IdoOnItemUpdate`
 - **Applies to Objects** = `SLECNs`
 - **Description** = `ECN Approval`
2. Create one action, which sends a prompt to the managers requesting a response:
 - **Action Sequence** = `10`
 - **Action Type** = `Prompt`
 - a. Use the **Event Action Prompt** form and associated event action parameter forms to create these parameters:
 - **TO** = `GC(EngineeringMgr) + ';' +GC(ProjectMgr) + ';' +GC(ProgramMgr)`
 - **SUBJECT** = `SUBSTITUTE("Need approval for engineering change {0}, {1}", P("EcnNum"), P("ReasonCodeDescription"))`

- **CATEGORY = "Engineering"**
- **BODY = "Please review the proposed engineering change on the Variables tab. Then use the Approve or Reject buttons on the Response tab to register your response. "**
- **SAVEMESSAGE = FALSE**
- **QUESTION = To approve or reject, click the buttons below.**
- **CHOICES = 1,sApprove, 0,sReject**
- **VOTINGRULE = Minimum Count Preferred Choice**
- **PREFERREDCHOICE = 1**
- **MINIMUM = 2**
- **FILTERFORM = EngineeringChangeNotices**
- **FILTER = SUBSTITUTE("ECNNum={0}", FP("ECNNum"))**

c. Save the action.

If you have done everything correctly, your syntax for this action step should look like this:

```
TO(" + GC(EngineeringMgr) + ';' + GC(ProjectMgr) + ';' + GC(ProgramMgr))
CATEGORY("Engineering")
SUBJECT(SUBSTITUTE("Need approval for engineering change {0}, {1}", P("EcnNum"),
P("ReasonCodeDescription")))
BODY("Please review the proposed engineering change on the Variables tab. Then use
the Approve or Reject buttons on the Response tab to register your response.")
SAVEMESSAGE(FALSE)
QUESTION("To approve or reject, click the buttons below.")
CHOICES("1,sApprove,0,sReject")
VOTINGRULE(MinimumCountPreferredChoice)
MINIMUM(2)
PREFCHOICE("1")
FILTERFORM("EngineeringChangeNotices")
FILTER(SUBSTITUTE("ECNNum={0}", FP("ECNNum"))))
```

3. Create the second action, which tells the system how to respond if approval is not granted:

- **Action Sequence = 20**
- **Action Type = Fail**

This action type ends handler execution with an error status. This effectively aborts the process and prevents the ECN from being changed in the database.

a. Starting with the **Event Action Fail** form, use the associated forms to create these parameters:

- **CONDITION(VOTINGRESULT(10) = "0")**
- **RESULT("The ECN change request was rejected by the managers.")**

b. Save the action and close the **Event Actions** form.

4. Return to the **Event Handlers** form and select the **Suspend** check box.

5. Save the handler.

6. Refresh the cached metadata.

For more information, including the procedure, see "Refreshing the metadata cache" on page 54.

Testing the event handler

To test this handler:

1. Open the **Engineering Change Notices** form and update an existing ECN. Save it. After you save the ECN, when the **Engineering Change Notices** refreshes the display, the record should be disabled for updating. It remains read-only until/unless it has been approved.
2. (Optional) With the **Engineering Change Notices** form selected, from the **Actions** menu, select **View Event Status**. This opens the **Event Status** form. Navigate to the last row and verify that the status for this event is **Running**.
3. Open the **Inbox** form for the individual designated as the Engineering Manager and verify that the message was received and that the **Response** tab displays the question and choice buttons. Click the button labeled **Yes**.
4. Open the **Inbox** form for the individual designated as the Project Manager and verify that the message was received and that the **Response** tab displays the question and choice buttons. Click the button labeled **Yes**.
5. Refresh the collection on the **Engineering Change Notices** form and verify that the ECN now displays normally (read/write) and shows your changes. As soon as two managers vote for the preferred choice, voting is closed and the change is approved. The third manager's vote is not needed.

You can also do a second test, clicking the button labeled **No** to reject the request by all three managers. In this case, when you refresh the **Engineering Change Notices** form, the ECN record displays normally but your changes are gone.

Points to note and remember

When creating this kind of event handler, keep these points in mind:

- When creating a message that requires a response from the recipient (usually a Prompt action type), you must mark the handler so that it suspends when executed. This means that it is also automatically marked as a synchronous handler.
- Because these event handlers must be suspended, pending the managers' responses, the Framework Event Service must be enabled for the configuration in which you are logged on.

Localizing message contents

Scenario 9: Translating captions in a purchase request

This scenario uses strings in captions so the text can be read by users in different countries.

Metadata setup

These strings must exist in the Forms database Strings tables:

Strings.Name	Strings.String	SpainString.String
sItem	Item	Prod
sWhse	Whse	Alm
sPoitemApprovalQuestion	Do you approve of purchasing %1 %2 of [%3: %4] for delivery to [%5: %6]	Usted aprueba de comprar %1 %2 del %3 "%4" para la entrega al %5 "%6"
sPoitemApproval	Purchase Approval	Aprobación de Comprar el Artículo
sOrderApproval	Order Approval	Aprobación del Documento

Event message category

In the **Event Message Categories** form, set up the Category **FORMAT(sOrder Approval)**, with the description **Approval of an Order**.

Event action parameters

Create an event action that includes these parameters:

```
...
QUESTION (
  CLIENTSUBSTITUTE (
    "sPoitemApprovalQuestion",
    P(QtyOrdered),
    P(UMDesc),
    "STRINGS(sItem)",
    P(Item), "STRINGS(sWhse)",
    P(Whse)
  )
)
SUBJECT (
  CLIENTSUBSTITUTE (
    P(PoitemApproval)
  )
)

CATEGORY("FORMAT(sOrder Approval)")
...
```

After the event action execution, the full components of the message in the database include these:

-
- EventMessage.Question
 - FORMAT(sPoltemApprovalQuestion, ~LIT~(100.0), ~LIT~(Metric Tons), STRINGS(sltem), ~LIT~(5" screw/x03 chrome\x04hex head\x05), STRINGS(sWhse), ~LIT~(MAIN))
 - EventMessage.Subject
 - FORMAT(sPoltemApproval)
 - EventMessage.Category
 - FORMAT(sOrderApproval)

Results (English)

An English-speaking user who refreshes the **Inbox** sees this message:

Component	Text
Question	Do you approve of purchasing 100.0 Metric Tons of [Item: 5" screw, chrome (hex head)] for delivery to [Whse: MAIN]
Subject	Purchase Approval
Category	Order Approval

Results (Spanish)

A Spanish-speaking user who refreshes the **Inbox** sees this message:

Component	Text
Question	Usted aprueba de comprar 100.0 Metric Tons del Prod "5" screw, chrome (hex head)" para la entrega al Alm "MAIN"
Subject	Aprobación de Comprar el Artículo
Category	Aprobación del Documento

More advanced scenarios

Scenario 10: Opening a session in a remote environment

In this scenario, we want to call in to another site or another Mongoose application environment and return a specific discrete piece of information from the remote application database. In this scenario, we want to return the **On Hand** quantity of an item in another site.

For the details of and procedure for this scenario, see the *Integrating IDOs with External Applications* guide.

Scenario 11: Cross-site event firing - Adding a message to another site's Inbox

This scenario illustrates the general requirements used to set up cross-site event firing. Our example adds a message to another site's **Inbox** form.

- An event handler called **GenericNotify** is available in every target site. You can fire this event to perform the "remote" work.
This event has one handler with one action that performs a Notify, using event parameters for **To**, **Subject**, **Category**, and **Body**.
- A stored procedure called `dbo.FireGenericNotifySp` is available in every target site. You can use this stored procedure to fire the event. This stored procedure accepts the T-SQL parameters `@To`, `@Subject`, `@Category` and `@Body`, and it fires the `GenericNotify` event, passing in the information that the event needs.

Note: You could do the same thing using a hand-coded IDO Method that calls `Mongoose.EventSystem.EventHandlers.FireApplicationEvent()`, in case the event needs to perform IDO-level actions. However, for this scenario, using the stored procedure is easier.

- Where you want to add a message to another site's inbox, your event handler includes a Dispatch IDO Request action, providing these parameters:

Parameter	Description
URL()	URL of an IDO Request Service that serves the target site Sample value: <code>http://MyUtilityServer/IDORequestService/RequestService.aspx</code>
CONFIGNAME()	Name of a configuration whose application database contains the target site. (Mongoose requires a configuration that is named after each site, so you can use the site name here.) Sample value: E(SiteId)
IDOREQUEST()	Invoke "SP!" (or the name of an IDO) plus the name of the stored procedure (in this case <code>FireGenericNotifySp</code>), passing in the event parameters as method parameters to the "method." Sample value (shown as XML to allow pasting into the form): <code>SUBSTITUTE('<RequestHeader Type="Invoke"> <InitiatorType /> <InitiatorName /> <SourceName /> <TargetName /> <RequestData> <Name>SP!</Name> <Method>FireGenericNotifySp</Method> <Parameters> <Parameter>{0}</Parameter> <Parameter>{1}</Parameter> <Parameter>{2}</Parameter> <Parameter>{3}</Parameter> <Parameter ByRef="Y" /> </Parameters> </RequestData> </RequestHeader>', E(ToParm), E(SubjectParm), E(CategoryParm), E(BodyParm))</code>

-
- If you have multiple intranets that do not serve all sites, and you want the event to fire across intranets, you may need to set up an event global constant for each intranet (or site) URL, and select one of those at runtime.
 - The sample action shown here assumes that the originating user has the same password on the source and target sites, which allows it to log in for the Invoke. If this may not be the case, you can instead set up a generic remote user, with permissions only to perform this one Invoke, and specify the USERNAME() and PASSWORD() parameters on the Dispatch IDO Request action. Alternatively, you can adjust the USELOCALPASSWORD() parameter.
 - If you want to perform a different type of remote work, you can set up your own events with handlers and actions to address the work you want to do. Then you can create stored procedures to fire those events, and finally call those stored procedures using Dispatch IDO Request actions in the appropriate existing handlers.

Appendix B: Reference Tables

B

This appendix provides several reference tables containing detailed information about various aspects of the Application Event System. Topics include:

- Firing events
- Summary of synchronous functionality
- Core application events
- Framework Event Attributes
- Event Action Types
- Event Action Parameters
- Expression Functions
- Expression Operators

Firing events

This table provides details about:

- Where events can be generated from (**Tier**)
- What can be used to generate them from that location (**Triggered by**)
- How to set them up (**Details for construction**)
- Whether the event is generated as a synchronous or asynchronous event (**Synchronous?**)

Tier	Triggered by	Details for construction	Synchronous?
Client	Event handler in form	Use a response type of Generate Application Event , select the Synchronous option.	Yes
		Use a response type of Generate Application Event , clear the Synchronous option.	No
	Script in form	Generate a custom form event. Create a form event handler for that custom event with a response type of Generate Application Event , with the SYNCHRONOUS() parameter.	Yes
		Generate a custom form event. Create a form event handler for that custom event with a response type of Generate Application Event , without the SYNCHRONOUS() parameter.	No
Middle	Custom IDO method	Invoke the <i>FireApplicationEvent()</i> static .NET method with the Synchronous parameter passed in with a value of True .	Yes
		Invoke the <i>FireApplicationEvent()</i> static .NET method with the Synchronous parameter passed in with a value of False .	No
Database	T-SQL	Use the command: EXEC FireEventSp	Yes
		Use the command: EXEC PostEventSp	No
Any	An event action	Use the GenerateEvent action type with a parameter of SYNCHRONOUS(true) .	Yes
		Use the GenerateEvent action type with a parameter of SYNCHRONOUS(false) .	No
	An event trigger	Select the Synchronous option.	Yes
		Clear the Synchronous option.	No

Summary of synchronous functionality

These tables list and describe the way synchronous and asynchronous events work:

Source	Synchronous?	Consists of	Requester	Initial executor
Framework	Yes	Synchronous (none marked Suspend) & asynchronous event handlers	WinStudio	IDO Runtime
			IDO Runtime	IDO Runtime
			Database	Database
		Synchronous (some marked Suspend) & asynchronous event handlers	WinStudio	IDO Runtime (validating mode)
			IDO Runtime	Event Service (Committing mode)
FireEvent OR Generate Event with Synchronous (True)	Yes	Synchronous and asynchronous event handlers	WinStudio	IDO Runtime
			IDO Runtime	IDO Runtime
			Database	Database
			Event service	Event service
PostEvent OR Generate Event with Synchronous (False)	No	Synchronous and asynchronous event handlers	WinStudio	Event service
			IDO Runtime	
			Database	
			Event service	

Context	Synchronous?	Suspending?	Event's or prior event handler's executor	Initial executor	Can adjourn?
Suspending event (always generated synchronously)	Yes	No	IDO Runtime (validating mode)	IDO Runtime	No
			Event service (committing mode)	Event service	
		Yes	Event service	Event service	Yes

Context	Synchronous?	Suspending?	Event's or prior event handler's executor	Initial executor	Can adjourn?
Non-suspending event generated synchronously	Yes	N/A	WinStudio	IDO Runtime	Yes
			IDO Runtime	IDO Runtime	
			Database	Database	
			Event service	Event service	
Non-suspending event generated asynchronously	Yes	N/A	WinStudio	Event service	Yes
			IDO Runtime		
			Database		
			Event service		
Any event	No	N/A	WinStudio	Event service	Yes
			IDO Runtime		
			Database		
			Event service		

Core application events

The tables in this section list and describe the Core application events that ship with the system.

Each event belongs to one of four groups:

- Framework events (non-IDO)
- Framework IDO events (non-suspendible)
- Suspendible framework IDO events
- Non-framework events

Framework events (non-IDO)

Non-IDO framework events fire whenever the system performs a session operation or when a BOD (Business Object Document) interaction is detected. You can create event handlers to respond to these events, but you cannot create your own triggers for them: The triggers are built-in and work automatically.

This table lists and describes the non-IDO framework events. The second column describes what cause the event to fire. The third column of the table lists or describes the attributes that are passed either as event expression functions or as event parameters, in addition to the object name:

Event name	Trigger/Fires when...	Context attributes passed
BodOnReceive	An inbound BOD is received	BOD noun BOD verb BOD XML
SessionOnLogin	A new session is requested	User name
SessionOnLoginFailed	A new session is requested but fails	User name Configuration name Configuration site Application name Application ID Domain user name Language ID Name of the computer from with the login request was initiated Reason for the failure Trusted client Workstation identifier Web Form Only?
SessionOnLogout	A session is closed	User name
SessionOnVarChanged	A request is received to change a session variable's value	The name and new value of the session variable
SessionPostVarChanged	A session variable's has been changed	The name and new value of the session variable

Framework IDO events (non-suspendible)

Non-suspendible framework IDO events fire whenever an IDO performs an operation whose effects cannot be postponed. You can create event handlers to respond to these events, but you cannot create your own triggers for them: The triggers are built-in and work automatically.

This table lists and describes the non-suspendible framework IDO events. The second column describes what cause the event to fire. The third column of the table lists or describes the attributes that are passed either as event expression functions or as event parameters, in addition to the object name:

Event name	Trigger/Fires when...	Context attributes passed
IdoOnInvoke	An IDO method is being invoked	Name of the IDO method that was invoked Number of parameters passed to the method Values of the parameters passed to the method
IdoOnLoadCollection	An IDO collection is being loaded into the system.	Load flags Property names Post query actions
IdoOnPersistFailed	An update collection request fails or there is a database exception error	Initiator Configuration name Data associated with the error Help file associated with the failed form or component Name of the IDO collection Inner exception message Line number where the error occurred Error message Target site Error number or code Name of the stored procedure or RPC that generated the error Name of the server on which the error occurred Source of the error String representation of the immediate frames on the call stack Error state User message Action attempted Property value submitted that failed Indicator of whether the value had been modified before the failure

Event name	Trigger/Fires when...	Context attributes passed
IdoOnUpdateCollection	An IDO collection update is being requested	Custom insert specification Custom update specification Custom delete specification
IdoPostInvoke	An IDO method has been invoked	Name of the IDO method that was invoked Number of parameters passed to the method Values of the parameters passed to the method
IdoPostItemDelete	An IDO item has been deleted	Row (IDO item) that was deleted
IdoPostItemInsert	An IDO item has been inserted into a collection	Row (IDO item) that was inserted
IdoPostItemUpdate	An IDO item has been updated	Row (IDO item) that was updated Modified flags
IdoPostLoadCollection	An IDO collection has been loaded into the system	Result set
IdoPostUpdateCollection	All IDO collection updates have been processed	—

Note: The `IdoOnPersistFailed` event is always fired inside the application database. Any synchronous handlers attached to this event cannot perform actions related to the middle tier, such as IDO operations. To perform IDO operations on a handler attached to the `IdoOnPersistFailed` event, change the handler to asynchronous.

Suspendible framework IDO events

Suspendible framework IDO events fire whenever an IDO performs a row operation whose effects can be postponed until after successful completion of the event's handlers. You can create event handlers to respond to these events, but you cannot create your own triggers for them: The triggers are built-in and work automatically.

This table lists and describes the suspendible framework IDO events. The second column describes what cause the event to fire. The third column of the table lists or describes the attributes that are passed either as event expression functions or as event parameters, in addition to the object name:

Event name	Trigger/Fires when...	Context attributes passed
IdoOnItemDelete	An IDO item is marked for deletion	Row (IDO item) being deleted
IdoOnItemInsert	A request is being made to insert an IDO item into a collection	Row (IDO item) being inserted
IdoOnItemUpdate	When an IDO item is being updated	Row (IDO item) being updated Modified flags

Non-framework events

Non-framework events are application events that perform specific tasks for specific framework features. You can create triggers that generate these events, but you should not create your own handlers for these events. Instead, use the handlers already associated with them.

This table lists and describes the non-framework events:

Event name	Trigger	Comments
GenericNotify	• TaskListCheck event handler	Notifies designated users of specified information using the internal messaging system Inbox. The system comes with one generic event handler for this event built-in. For specific instructions on how to use this event and handler, see the online help topic, "Using the GenericNotify event and handler".
GenericSendEmail	n/a	Notifies designated users of specified information using email. The system comes with one generic event handler for this event built-in. For specific instructions on how to use this event and handler, see the online help topic, "Using the GenericSendEmail event and handler".

Event name	Trigger	Comments
ProcessNewDataMaintenance	User clicks Finish in the New Data Maintenance Wizard to complete the creation of a form, IDO, etc.	Orchestrates and performs operations for completion of the New Data Maintenance Wizard process.
TaskListCheck	An event trigger (every 60 seconds)	Notifies all users of any tasks and appointments assigned to them for which the reminder date has passed.

This table provides usage information about the GenericNotify and GenericSendEmail events:

Event name	Input parameters	Purpose
GenericNotify	ToVar	The primary recipients of the notification, separated by semi-colons
	SubjectVar	The string to be used as the subject line in the notification
	CategoryParm	The category to which the notification pertains
	BodyVar	The contents of the body of the message to be sent through the internal messaging system
	HTMLBodyVar	The contents of the body of the message to be formatted as HTML and sent by email to recipients who have Send External Notifications enabled
GenericSendEmail	EmailTo	The primary recipients of the email message, separated by semi-colons
	EmailCc	The secondary recipients of the email message, separated by semi-colons
	EmailSubject	The string to be used as the subject line in the email message
	EmailMessage	The contents of the body of the email message
	EmailHTMLFormat	Indicator that the body of the email is to be formatted as HTML

Framework Event Attributes

This table lists and describes the expression functions associated with each framework (Core) event.

For more information about function syntax, see “Expression Functions” on page 171.

Note: Wherever property names are mentioned in this table, *<Property#>*, as represented here, specifies the name as a string expression. To specify the property name as a constant value, enclose it in double quotes (“...”)

Event name	Applicable functions	Contents
BodOnReceive	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	BODNOUN()	Name of the BOD
	BODVERB()	Type of action to be performed by the BOD
	BODXML()	Contents of the BOD data area, as a parsable string
IdoOnInvoke	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	METHOD()	Name of the method to be invoked
	METHODPARMS()	Number of parameters in the lists that follow
	METHODPARM(< <i>n</i> >)	The value of the <i>n</i> th parameter passed to the method, where <i>n</i> is an integer
	FILTERMETHODPARM(< <i>n</i> >)	The value of the <i>n</i> th parameter passed to the method as a string

Event name	Applicable functions	Contents
IdoOnItemDelete	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTY(<Property1>)	Value of the first property of the row to be deleted from the database
	...	...
	PROPERTY(<PropertyM>)	Value of the M th property of the row to be deleted from the database
	P(<Property>)	Value of the specified property
	FILTERPROPERTY(<Property>) or FP(<Property>)	Value of the specified property as a string
IdoOnItemInsert	PROPERTYMODIFIED()	Always returns 0
	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of properties to be saved
	PROPERTY(<Property1>)	Value of the first property of the row to be saved into the database
	...	...
	PROPERTY(<PropertyM>)	Value of the M th property of the row to be saved into the database
	P(<Property>)	Value of the specified property
	FILTERPROPERTY(<Property>) or FP(<Property>)	Value of the specified property as a string
	PROPERTYMODIFIED(<Property1>)	1 if the first property was updated from its original value; otherwise 0
	...	...
	PROPERTYMODIFIED(<PropertyM>)	1 if the M th property was updated from its original value; otherwise 0

Event name	Applicable functions	Contents
IdoOnItemUpdate	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of properties to be saved
	PROPERTY(<Property1>)	Value of the first property of the row to be saved into the database
	...	...
	PROPERTY(<PropertyM>)	Value of the M th property of the row to be saved into the database
	P(<Property>)	Value of the specified property
	FILTERPROPERTY(<Property>) or FP(<Property>)	Value of the specified property as a string
	PROPERTYMODIFIED(<Property1>)	1 if the first property was updated from its original value; otherwise 0
	... PROPERTYMODIFIED(<PropertyM>)	... 1 if the M th property was updated from its original value; otherwise 0
IdoOnLoadCollection	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of properties to be retrieved, separated by commas
	POSTQUERYACTIONS()	List of post-query actions
	FILTERSTRING()	String to be used to filter the collection
	RECORDCAP()	Maximum number of records to be returned

Event name	Applicable functions	Contents
IdoOnUpdateCollection	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	CUSTOMINSERT()	Specification for a custom insert
	CUSTOMUPDATE()	Specification for a custom update
	CUSTOMDELETE()	Specification for a custom delete
IdoPostInvoke	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	METHOD()	Name of the method that was invoked
	METHODPARMS()	Number of parameters in the lists that follow
	METHODPARAM(< <i>n</i> >)	The value of the parameter referenced by <i>n</i> , where <i>n</i> is an integer
	FILTERMETHODPARAM(< <i>n</i> >)	The value of the <i>n</i> th parameter passed to the method
IdoPostItemDelete	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection

Event name	Applicable functions	Contents
IdoPostItemInsert	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTY(<Property1>)	Value of the first property of the row that was saved into the database
	...	...
	PROPERTY(<PropertyM>)	Value of the M th property of the row that was saved into the database
	P(<Property>)	Value of the specified property
IdoPostItemUpdate	FILTERPROPERTY(<Property>) or FP(<Property>)	Value of the specified property as a string
	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTY(<Property1>)	Value of the first property of the row that was saved into the database
	...	...
	PROPERTY(<PropertyM>)	Value of the M th property of the row that was saved into the database
	P(<Property>)	Value of the specified property
	FILTERPROPERTY(<Property>) or FP(<Property>)	Value of the specified property as a string

Event name	Applicable functions	Contents
IdoPostLoadCollection	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of property names that were retrieved, separated by commas
	PROPERTY(1, <Property1>)	Value of the first property from the first row retrieved
	...	...
	PROPERTY(N, <PropertyM>)	Value of the M th property from the N th row retrieved
	P(N, <Property>)	Value of the specified property of the N th row
	FILTERPROPERTY(N, <Property>) or FP(N, <Property>)	Value of the specified property of the N th row as a string
	FILTERSTRING()	String to be used to filter the collection
IdoPostUpdateCollection	RECORDCAP()	Maximum number of records to be returned
	ROWS()	Number of rows that were loaded
SessionOnLogin	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	USERNAME()	User requesting the session login

Event name	Applicable functions	Contents
SessionOnLoginFailed	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	USERNAME()	User requesting the session login
	CONFIGNAME()	Name of the configuration the user was attempting to log in to
	This event also retrieves this information related to the failed login attempt, as non-output event parameters:	
	AdditionalFailureInformation	Information related to the failed login attempt not covered by other parameters
	ApplicationID	Name of the application associated with the configuration
	DomainUserName	Domain and user login name, separated by a forward slash
	FailureReason	Technical reason for the failed attempt
	LanguageID	Alphanumeric language ID code for the system language
	LoginConfigurationName	Name of the configuration the user was attempting to log in to
	LoginUserName	User requesting the session login
	MachineName	Name of the computer from which the user attempted the session login
	MG_CurrentSite	Name of the site associated with the configuration
	ToolName	Name of the Mongoose-based application or "Mongoose"
	TrustedClient	Indicator that the request is from a trusted client, so middle tier security and license checks can be disabled
	WebFormOnly	When set to TRUE, indicates that the web client request include the page=formonly parameter
	Workstation	Network domain and user login name, separated by a forward slash

Event name	Applicable functions	Contents
SessionOnLogout	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	USERNAME()	User logging off the session
SessionOnVarChanged	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	VARIABLENAME()	Name of the session variable
	VARIABLEVALUE()	Value of the session variable
SessionPostVarChanged	INITIATOR()	Composite initiator (period-separated): • Type (form or other client) • Name (form name or other client ID)
	VARIABLENAME()	Name of the session variable
	VARIABLEVALUE()	Value of the session variable

Event Action Types

This table lists and describes the action types available for use when defining event actions. For details about the associated event action parameters, including their syntax and examples, see “Event Action Parameters” on page 147.

Note: Adjourning event action types (that is, Prompt, Wait, and Sleep) cannot be assigned to event actions on a transactional event handler. For more information about adjournment, see “Adjournment and resumption” on page 25. For more information about transactional event handlers, see “Transactions” on page 28.

Mid-tier event action types (that is, Call IDO Method, Dispatch IDO Request, Execute IDO Request, Load Collection, Load IDO Row, and Update Collection) cannot be executed on a synchronous event handler whose event is fired from the database layer (that is, via FireEventSp, or attached to any Session framework event). If this is attempted, the handler fails with an error.

Action type	What it does	Associated event action parameters	Adjourning?
Achieve Milestone	Changes the visual state of a running event when a specified condition is met. This action type can be particularly useful when you want the system to allow for fast, visual indications of what action step a handler has reached.	• Condition • State (text) • Image (file)	No
Attach	Allows you to: • Attach or link additional files to a record as part of an event action sequence • Updated files already attached to a record • Detach obsolete attachments from a record as part of an event action sequence	• Document action (Attach, Link, Update, or Detach) • Document name • Document sequence • Document type • Document description • Document (file) extension • Document internal? • Document media type • Document file name • BLOB property • Attachment embedded • Attachment name • Attachment sequence	No
Audit	Adds an entry to the Audit Log . This action type can be useful when you want to track various changes of values or milestones achieved, for example.	• Description • Key value • Old value • New value	No
Branch	Goes to a specified other step when a specified condition is met.	• Condition • Destination	No

Action type	What it does	Associated event action parameters	Adjourning?
Call Database Method	Calls a stored procedure, passing any parameters required by or returned from the stored procedure.	• Stored procedure name • Parameters	No
Call IDO Method	Invokes an IDO method, passing any parameters required by or returned from the IDO method.	• IDO • Method • Parameters	No
Call Web Service	Invokes a Web service, passing any parameters required by or returned from the Web service.	• URL • Operation (Method) • Parameters • Output parameters	No
Dispatch IDO Request	Dispatches an IDO request to execute some action on a system to which the user is not currently connected or signed in. This IDO request can be, for example, to load a collection, update a collection, or invoke an IDO method on an IDO runtime service at another "enterprise" (for example, one of your customers or vendors who also uses the same Mongoose application) or with another site in your own enterprise. This action type opens a session on the remote database server, performs the requested action, and closes the session. If appropriate, this action type also receives any response data from the remote database server and places it in a return variable or return parameter. You can then use the Transform XML action type to parse the return data.	• IDO Request XML • URL • Configuration Name • Username • Password • Use Local Password • Response	No

Action type	What it does	Associated event action parameters	Adjourning?
Execute IDO Request	Executes an IDO request on the system to which the user is currently connected or signed in. This IDO request can be, for example, to load a collection, update a collection, or invoke an IDO method. This action type: Optionally, opens a new session (closing it when the requested action is finished). Performs the requested action. If appropriate, receives any response data generated by the IDO action and places it in a return variable or return parameter. You can then use the Transform XML action type to parse the return data.	• IDO Request XML • Response	No
Fail	Ends the handler execution with an error status when a specified condition is met and, optionally, specifies a failure or error message to be displayed on the Event Status form.	• Condition • Result (error message)	No
Finish	Marks the event handler as being finished successfully, optionally when a specified condition is met. Optionally, specifies a failure or error message to be displayed on the Event Status form.	• Condition • State • Image • Result	No
Generate Event	Generates another event, optionally when a specified condition is met, passing any parameters required by or returned from the other event.	• Condition • Event name • Parameters • Output parameters • Synchronous • Transactional	No

Action type	What it does	Associated event action parameters	Adjourning?
Goto	Directs the handler flow to an action step other than the next one in the sequence. This can be useful when you want to redirect the flow based on some condition that has been met in another action step.	• Destination	No
Load Collection	Loads records from a specified collection associated with an IDO, up to the record cap. You can use various options on this form to limit the amount of data returned by this event action. The data retrieved by this action is held in a "result set", which you must then refer to for further action.	• IDO • Properties • Filter • Order By • Custom Load • Custom Load Params • Record cap • Result Set Assignment	No
Load IDO Row	Loads a specified record (row) from a collection associated with an IDO.	• IDO • Properties • Filter • Order By • Custom Load • Custom Load Params • Output	No
Notify	Sends a system-generated message to recipients (system users), optionally when a specified condition is met. This notification is transmitted internally within the WinStudio system, without having to use external email or other messaging systems.	• Condition • To • Cc • Subject • Category • Body • HTML Body • Attach • Save in Sent Items • Filter form • Filter spec • Payload Access • Modified Payload Access • Unmodified Payload Access	No

Action type	What it does	Associated event action parameters	Adjourning?
Prompt	Sends a system-generated message to recipients (system users) and awaits a response from each. Also sends an email to recipients for whom Send Email Prompts is specified in the Users form. To send emails for prompts, you must have an SMTP server designated on the Intranets form and automatic relaying of email messages configured on the SMTP server. Counts votes according to a specified rule. If no response or an insufficient response is received within a certain time period, optionally goes to an alternate action, such as these: Perform an alternate prompt to an alternate set of recipients. Set a default and branch to the subsequent event action. Go directly to the subsequent event action (the same as if a sufficient number of responses were received). For information about voting rules, see “Voting rules” on page 66.	• Condition • To • Cc • Subject • Category • Body • HTML Body • Attach • Save in Sent Items • Question • Choices • Voting rule • Preferred choice • Minimum • Quorum • Wait for Quorum • Timeout • Timeout destination • Filter form • Filter specification • Entry form • Payload access • Modified payload access • Unmodified payload access	Yes
Run Background Task	Places a task on the background task queue.	• Task name • Task parameters • Task status • Task number	No

Action type	What it does	Associated event action parameters	Adjourning?
Send BOD	Allows you to to programmatically create and send a Business Object Document (BOD) to the Replication Outbox using an event and handler.	• BOD Noun • BOD Verb • Select XML Template from Template or File • Tree View of BOD • Expand/Collapse All • Add Element • Add Attribute • Remove • Rename • Create and save Node Text as an expression or XML • XPath	No
Send Email	Sends an external email to recipients (system users). To use this action, you must have an SMTP server designated on the Intranets form and automatic relaying of email messages configured on the SMTP server. This action allows you to also include attachments with the email.	• Condition • To • Reply To • Cc • Subject • Body • Body is in HTML format • Attach	No
Set Attributes	Sets text that displays in the Event Title field of the Event Status form.	• Title	No
Set Values	Sets event variable(s) and/or event parameter(s) to given values or expressions, optionally when a specified condition is met.	• Condition • Variables • Parameters • Session variables • Method parameters • Value • Row • Properties	No
Sleep	Waits until a specified period of time has elapsed before proceeding to the next action.	• Interval	Yes

Action type	What it does	Associated event action parameters	Adjourning?
Transform XML	Takes XML as input and transforms it for use elsewhere. This action type transforms specified XML, based on transformation rules contained in the XSLT, with the resulting XML stored in a result variable or parameter. A transform XML operation can range from simple (picking out a single specific element from a large document, to be used as part of a SUBSTITUTE expression in a Notify action) to complex (translating an entire document from one format to another, to be used as input to some other process).	• XML • XSLT • Result XML	No
Update Collection	Updates an IDO collection by inserting, updating, or deleting records within that collection. Using this action type, you can either update a collection directly from a specified IDO, or you can update a collection that is already loaded into a named result set.	• Update using IDO or loaded collection • IDO • Result Set ID • Action • Properties • Row • Optimistic Lock • Commit	No
Wait	Instructs the system to wait for a specified condition to be met before proceeding to the next action. If the condition does not become true within a certain time period, optionally goes to an alternate action. A retest interval of zero (0) means that the condition is never retested. A timeout setting of zero (0) means that the event action never times out, that is, that it waits indefinitely.	• Condition • Retest interval • Timeout • Timeout Destination	Yes

Event Action Parameters

This table lists (in alphabetic order) and describes all the event action parameters and their functions. For a list of all action types with the possible parameters associated with them, see “Event Action Types” on page 139.

For information about constructing and using event expressions, see “Expression Functions” on page 171 and “Expression Operators” on page 194. For the complete grammar available for constructing expressions, see “Expression Grammar” on page 197.

When using the table, keep these points in mind:

This:	Refers to:
ALLCAPS Parentheses () Equal signs (=) Commas	Functions and punctuation to be specified and stored verbatim.
Content in italics	Content that varies according to the context. For example, for a SUBJECT(<i>expr</i>) parameter, <i>expr</i> represents the actual content that is sent in the message.
Ellipses (...)	Indicates more like the preceding element, separated by commas. Parentheses indicate that the repetition of elements is optional.
scalarExpr	Scalar (numeric, string, date, or typeless) event expressions. For more information about constructing and using event expressions, see “Expression Functions” on page 171.
<i>Boolean</i>	A Boolean event expression. For more information about construction and using event expressions, see “Expression Functions” on page 171.
<i>name</i> <i>var</i> (variable) <i>parm</i> (parameter) <i>collection</i>	Identifiers used for named or contextual purposes. These identifiers can consist of: An initial uppercase or lowercase letter or underscore (_), followed by Any combination of letters, numerals, underscores, square brackets ([]), and/or periods.
<i>prop</i> (property)	Specifies the property name as a string expression. To specify the property name as a literal value, enclose it in double quotation marks (" ").
action	Event action sequence number (values to be adjusted automatically if actions are resequenced).

Parameter	Used with action type(s)	Description/Purpose	Syntax
Action	Update Collection	Indicates what the system is to do with the data.	One of these: ACTION(INSERT) ACTION(UPDATE) ACTION(DELETE)
Attach	- Notify - Prompt - Send Email	Specifies which attached documents (files) are to be forwarded along with the messages generated by the current action. Has these possible options: - ALL - NONE - INTERNAL - EXTERNAL	ATTACH(<i>option</i>)
Attachment Embedded	Attach	Specifies whether the attachment is linked to email messages in such a way that it can be embedded into the HTML version of the message body.	ATTACHMENTEMBEDDED (<i>Boolean</i>)
Attachment Name	Attach	Specifies the name by which this document is identified within the application event system. If this value is not specified for these actions, a default value is constructed by concatenating the Document Name with the Document Sequence, separated by a period; for example: MyDocument.000000001.	ATTACHMENTNAME (<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Attachment Sequence	● Attach	Specifies the name of the event variable that is to receive the attachment sequence number of the attached, linked, or updated document. This variable can be used in a subsequent action to look up the resulting Attachment Name or other attributes of the document.	ATTACHMENTSEQ(<i>ID</i>)
BLOB Property	● Attach	Specifies a previously retrieved BLOB (binary large object) property containing the document to be attached or to replace the currently attached document. Must be used with either a P or PROPERTY parameter.	DOCBLOB(P PROPERTY (<i>scalarExpr</i>))
BOD Noun	● Send BOD	Specifies the noun as part of the noun/verb combination that defines the Business Object Document (BOD) to process. This noun is used to specify the actual BOD XML document name.	BODNOUN(<i>scalarExpr</i>)
BOD Verb	● Send BOD	Specifies the BOD verb to be used as part of the noun/verb combination that defines the BOD to process.	BODVERB(<i>scalarExpr</i>)
BOD XML	● Send BOD	Contains the actual XML content of the BOD to be sent. Can be constructed manually, by using a template, by using an Expression Editor, by loading in from an XML file, or by using a combination of these approaches.	BODXML("XMLcontent")
BOD XML Name	● Send BOD	Indicates the name of the XML file from which the base XML code was derived, whether from a template, an XML file, or neither.	BODXMLNAME (<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Body	• Notify • Prompt • Send Email	Indicates the content that is to be generated for the body of the message, whether for an internal system message or an email. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each Inbox user and email prompt recipient (applicable only for Notify and Prompt).	BODY(<i>scalarExpr</i>)
Body is in HTML format	• Send Email	When selected (set to TRUE), specifies that the body of the email is to be rendered using HTML.	HTMLFORMAT(<i>Boolean</i>)
Category	• Notify • Prompt	Specifies a category to group the message with others of a similar purpose. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each Inbox user and email prompt recipient.	CATEGORY(<i>scalarExpr</i>)
Cc	• Notify • Prompt • Send Email	Indicates a list of secondary recipients of the message, delimited by semi-colons (;). Do not leave spaces between multiple recipients in the list. For Notify and Prompt actions, recipients are identified by their user IDs on the system. For Send Email actions, recipients are identified by their email addresses.	CC(<i>scalarExpr</i>) Examples: CC("jdoe;tsmith") CC("jdoe@provider.net") Note that literal values for user IDs or email addresses must be enclosed in double quotes ("").

Parameter	Used with action type(s)	Description/Purpose	Syntax
Choices	Prompt	Indicates a comma-separated list of return values and button labels (or link labels for email). Note that the parser strips out whitespace, so it is OK to include spaces between items in the list. In each pair, the first item is the value to be returned if the recipient selects that option. The second item is the label that appears on the selection button/link. For the button/link labels, you can use strings defined in the Strings table or literal values. For emails, strings are translated and formatted based on the Default Language specified for the recipient in the Users form. On the Inbox form, strings are translated and formatted based on the Language setting specified on the Runtime Behavior Settings. The <i>scalarExpr</i> must consist of an even number of list items. The default button/link labels are Approve and Disapprove, with return values of 1 and 0 respectively.	CHOICES(<i>scalarExpr</i>) Examples: CHOICES("" + GC(YesValue) + ",sYes," + GC(NoValue) + ",sNo") Note that the expression must be enclosed in double-quotes (""). For more information about designating strings for this parameter, see "Prompts and responses" on page 65.
Commit	Update Collection	Determines whether or not the changes made up to this point in the action flow are to be committed to the database.	COMMIT(<i>Boolean</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Condition	• Achieve Milestone • Branch • Fail • Finish • Generate Event • Notify • Prompt • Send Email • Set Values • Wait	Sets up the condition that determines when the action is executed.	CONDITION(<i>Boolean</i>) Examples: CONDITION(NOT PROPERTYMODIFIED("CreditLimit")) CONDITION(P("Quantity") < 500)
Configuration Name	• Dispatch IDO Request	Determines the configuration on a remote server from which you want to make an IDO request.	CONFIGNAME(<i>scalarExpr</i>)
Custom Load	• Load Collection • Load IDO Row	Indicates the name of the custom load method to use in loading the collection data.	CUSTOMLOAD(<i>scalarExpr</i>)
Custom Load Parms (Custom load method parameters)	• Load Collection • Load IDO Row	Indicates the parameters to be passed to the custom load method.	CUSTOMLOADPARMS(<i>scalarExpr</i>)
Description	• Audit	Defines the description to be passed to the audit log. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each user viewing the Audit Log form.	DESC(<i>scalarExpr</i>)
Destination	• Branch • Goto	Indicates the action sequence number of the event action the system is to go to when executing this action.	DEST(<i>action</i>) Example: DEST(40)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Document Action	● Attach	Determines what type of action is to be taken with the attached document (file). Options include ATTACH, LINK, UPDATE, DETACH.	DOCACTION(<i>action</i>)
Document Description	● Attach	Specifies a brief description of the document type or attached document (file).	DOCDESC(<i>scalarExpr</i>)
Doc Extension	● Attach	Indicates the extension of the document (file) to be attached for view or export. If this value is not specified, the document extension from the associated Document Type field is used.	DOCEXT(<i>scalarExpr</i>)
Document File Name	● Attach	Specifies a file on the system that is to be attached or updated with a new version. The specified file must reside on the database server.	FILENAME(<i>scalarExpr</i>)
Document Internal	● Attach	Restricts access to an attached document (file) so that it cannot be accessed by means of the Customer Portal or the Vendor Portal. The Send Email action ignores files for which this option is set. Internal files can only be sent to email recipients using a Notify or Prompt action.	DOCINTERNAL(<i>Boolean</i>)
Document Media Type	● Attach	Indicates the Internet media type (formerly MIME, or Multipurpose Internet Mail Extensions) to be associated with files of that document type when attached to an email message. This value suggests to the email client how to present the document, either inline with the message body, or when previewed or opened (for example, image/jpeg or application/pdf).	DOCMEDIATYPE(<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Document Name	● Attach	Specifies the name of the document (file) that is attached or is to be attached. This name is an internal name, used only to identify the document in the list of documents.	DOCNAME(<i>scalarExpr</i>)
Document Sequence	● Attach	Specifies a number identifier to distinguish multiple documents (files) having the same Document Name.	DOCSEQ(<i>int</i>)
Document Type	● Attach	Indicates the type of the document (file) to be attached.	DOCTYPE(<i>scalarExpr</i>)
Entry Form	● Prompt	Indicates the name of the form to be launched to edit the event variables for the event handler.	ENTRYFORM(<i>scalarExpr</i>)
Event Name	● Generate Event	Indicates the name of the event to be generated.	EVENTNAME(<i>scalarExpr</i>)
Filter or Filter Spec	● Load IDO Row ● Load Collection ● Notify ● Prompt	Identifies the filter to use in restricting data retrieval. This parameter is not valid with CUSTOMLOAD. Functionally, this is the same as the Permanent Filter Expression property in WinStudio development.	FILTER(<i>scalarExpr</i>)
Filter Form	● Notify ● Prompt	Indicates the name of the form to be launched to display information about the row represented by the event variables for the event handler.	FILTERFORM(<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Html Body	• Notify • Prompt	Specifies the content that is to be generated for the body of the message and formatted in HTML for an email. The content in this field is used only for message content being sent by email; internal system messages ignore this field.	HTMLBODY (<i>"HTMLcontent"</i>)
IDO	• Call IDO Method • Load Collection • Load IDO Row • Update Collection	Indicates the name of the IDO on which to perform the operation.	IDO(<i>scalarExpr</i>)
IDO Request XML	• Dispatch IDO Request • Execute IDO Request	Contains the correctly formatted XML content of the IDO request.	IDOREQUEST(<i>scalarExpr</i>)
Image	• Achieve Milestone • Finish	Specifies the name of the graphics file to display with the event in the status forms. This path and file name must be in the format: <i>Drive:\folder\subfolder\... myGraphicFile.ext</i> The specified graphics file must be one of the supported types. For more information, see the online help.	IMAGE(<i>scalarExpr</i>) Example: IMAGE("C:\WINDOWS\Web\Wallpaper\Tulips.jpg")
Interval	• Sleep	Specifies the amount of time (in seconds) that the system is to pause before moving to the next action.	INTERVAL(<i>numericExpr</i>)
Key value	• Audit	Specifies the text that is to appear in the Primary Key column of the Audit Log form when this action runs.	KEY(<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Method	• Call IDO Method	Specifies the name of the IDO method to call.	METHOD(<i>scalarExpr</i>)
Method Parameters	• Set Values	Changes method parameter values during an IdoOnInvoke event. <i>paramNumber</i> is the 1-based position of the parameter to change. <i>scalarExpr</i> is the value to which the parameter is to be set. For multiple parameters, include them as a comma-separated list.	SETMETHODPARAM-VALUES(<i>paramNumber</i> = <i>scalarExpr</i> [, ...])
Minimum (Count or Percentage)	• Prompt	• Count—Indicates the minimum number of votes required for a choice to win. Must be used in conjunction with a ConditionalPlurality, or MinimumCount voting rule. • Percentage—Indicates the minimum percentage of the total vote (based on recipients of the message) required for a choice to win. Must be used in conjunction with a MinimumPercentage voting rule. For more information about voting rules, see “Voting rules” on page 66.	MINIMUM(<i>numericExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Modified Payload Access	• Notify • Prompt	Specifies how message payload elements that represent modified properties are handled: • Default – The modified parts of the payload are visible; the recipient can further modify any of their contents. • Hidden – The modified parts of the payload are hidden, for example, for a secret ballot. • Read-Only – The modified parts of the payload are display-only, so the recipient cannot further modify the contents. This specification applies to event message variables with names in the form Row.Property that have a corresponding Row.Property.Modified element with a value of 1. This setting is overridden by settings for the Default Access (Event Variable Groups form) or Access (Event Actions form, Variables grid) for the same property name.	One of these: • MODIFIEDPAYLOAD-ACCESS(DEFAULT) • MODIFIEDPAYLOAD-ACCESS(HIDDEN) • MODIFIEDPAYLOAD-ACCESS(READONLY)
New Value	• Audit	Specifies the text that is to appear in the New Value column of the Audit Log form when this action runs. Represents the new value of something that was changed.	NEW(<i>scalarExpr</i>)
Old Value	• Audit	Specifies the text that is to appear in the Old Value column of the Audit Log form when this action runs. Represents the original value of something that was changed.	OLD(<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Operation	Call Web Service	Specifies the name of the Web service to call.	METHOD(<i>scalarExpr</i>)
Optimistic Lock	Update Collection	Determines whether optimistic locking will be employed for safety.	OPTIMISTICLOCKING(<i>Boolean</i>)
Order By	- Load Collection - Load IDO Row	Specifies the name of a valid property to be used for the sort order on the collection when it is loaded and displayed. For example, if you select CreateDate as the property for this field, the system sorts the collection according to the dates on which transactions were recorded. You can use multiple properties, and the system prioritizes multiple properties in the order in which they are listed. In other words, the first property would be the primary property for sorting; the next property would be the secondary property for sorting; and so on. Multiple properties must be specified as a comma-separated list. This parameter is not valid with CUSTOMLOAD.	ORDERBY(<i>scalarExpr</i>) Examples: ORDERBY("PoLine") ORDERBY("CoRelease ASC, Coltem") Note that, in the second example, there is an attribute "ASC" specified. This tells the system to sort the list in ascending order, which is also the default. You could instead use DESC to tell the system to sort the list in descending order.
Output	Load IDO Row	Specifies where the system is to place the IDO row data. You can use return variables (RV) or return parameters (RE). Multiple outputs can be designated, using a comma-separated list of return value-name pairs. A separate value-name pair is required for each property being returned.	SET(RV(<i>var</i>) RE(<i>param</i>) = <i>scalarExpr</i> [, ...]) Each RV or RE represents a name-name pair. <i>var</i> is the name of an event variable; <i>param</i> is the name of an event parameter; <i>scalarExpr</i> evaluates to a property name of the IDO that was loaded. Example: SET(RV(myVar) = "")

Parameter	Used with action type(s)	Description/Purpose	Syntax
Output Parameters	• Generate Event • Call Web Service	Specifies where the system is to place the output (return) data from the event or Web service. You can use return variables (RV) or return parameters (RE). Multiple output parameters can be returned, using a comma-separated list of return value-name pairs. Each name is the name of a node in the XML response document.	<code>SET(RV(<i>var</i>) RE(<i>param</i>) = <i>scalarExpr</i> [, ...])</code> Each RV or RE represents a name-name pair. In the case of a Generate Event action, the <i>scalarExpr</i> evaluates to the name of an event output parameter of the generated event. In the case of a Call Web Service action, the <i>scalarExpr</i> evaluates to the name of a node in the response XML. Example: <code>SET(RV(RecordsDone) = "rows_succeeded", RE(PostResult) = "result_message")</code>
Parameters [1]	• Call Database Method • Call IDO Method	Specifies a comma-separated list of parameters required by the IDO method or stored procedure. For alternate ways to pass parameters in these cases, see “Passing parameters from actions” on page 44. If an event variable is named using this function and that variable does not already exist for the current handler, the system creates it automatically as a non-persistent variable. If a parameter is set using this function, and that parameter was not passed into the current event, the system creates it automatically as an output parameter.	<code>PARMS(<i>scalarExpr</i> RV(<i>var</i>) RE(<i>param</i>)[, ...])</code>
Parameters [2]	• Call Web Service • Generate Event	Specifies a comma-separated list of parameters to be passed as input to the event or Web service. These parameters must be in the form of name-value pairs.	<code>PARMS(<i>name=scalarExpr</i>[, ...])</code>

Parameter	Used with action type(s)	Description/Purpose	Syntax
Parameters [3]	Set Values	Specifies parameters as name-value pairs. For multiple parameters, include them as a comma-separated list of name-value pairs. If a parameter is set using this function, and that parameter was not passed into the current event, the system creates it automatically as an output parameter.	SETPARMVALUES(<i>paramName</i> = <i>scalarExpr</i>[, ...])
Password	Dispatch IDO Request	Indicates the password associated with the user ID specified in the User Name field.	PASSWORD(<i>scalarExpr</i>)
Payload Access	- Notify - Prompt	Sets the recipient's access to payloads for variables whose name is in the form Row.Property: Default – Payload variables are visible and the recipient can modify any of their contents. Hidden – The payload is hidden, for example, because it contains no information relevant to the voting. Read-Only – The entire payload is display-only, so the recipient cannot modify any of the contents. This specification applies to event message variables with names in the form Row.Property. This setting is overridden by settings for the Default Access (Event Variable Groups form) or Access (Event Actions form, Variables grid) for the same property name.	One of these: - PAYLOADACCESS (DEFAULT) - PAYLOADACCESS (HIDDEN) - PAYLOADACCESS (READONLY)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Preferred Choice	Prompt	Specifies the preferred choice among multiple voting choices. This parameter must be used in conjunction with one of the PreferredChoice voting rules.	PREFCHOICE(<i>scalarExpr</i>)
Properties [1]	- Load Collection - Load IDO Row	Identifies a comma-separated list of properties to load from the IDO collection that is specified in the IDO field.	PROPERTIES(<i>scalarExpr</i>)
Properties [2]	Update Collection	Specifies the primary keys identifying the row(s) to be updated, followed by the properties for which you want to set values, both as name-value pairs, in a comma-separated list.	SETPROPVALUES(<i>propName</i> = <i>scalarExpr</i> [, ...])
Properties [3]	Set Values	Specifies the properties for which you want to set values as name-value pairs. For multiple properties, include them as a comma-separated list. Used only with IdoOnItemInsert, IdoOnItemUpdate, and IdoOnItemDelete framework (Core) events.	SETPROPVALUES(<i>propName</i> = <i>scalarExpr</i> [, ...])
Question	Prompt	Specifies the content of the direct question that recipients are to respond to when they receive the message. This question appears in the Question field on the Response tab of the Inbox form. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each Inbox user and email prompt recipient.	QUESTION(<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Quorum	Prompt	Specifies the number of votes to await before closing voting, expiring the remaining unvoted messages, and continuing to the subsequent event action. If this field contains a value, the number overrides the automatic quorum algorithm used to calculate whether further votes are required in order to make the VOTINGRESULT() non-blank. If the Quorum value is positive, it implies that Wait for Quorum is TRUE. If the Quorum value is non-positive, it implies that Wait for Quorum is FALSE.	QUORUM(<i>numericExpr</i>)
Record Cap	Load Collection	Designates the maximum number of records to be returned when the collection is loaded. The value you set for this field overrides any system or personal settings when executing this handler.	RECORDCAP(<i>numericExpr</i>) Example: RECORDCAP(500)
Reply To	Send Email	Specifies the email addresses of any users or email recipients who are to receive replies to the email generated by this action.	REPLYTO(<i>scalarExpr</i>)
Response	- Dispatch IDO Request - Execute IDO Request	Specifies the type—return variable (RV) or return parameter (RE)—and name of the container that is to hold the XML response (RESULT) data from the IDO request.	SET(RV(<i>var</i>) = RESULT) SET(RE(<i>param</i>) = RESULT)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Result	● Finish ● Fail	Specifies the content of the message that is to be displayed if the event handler fails or finishes at this point. This message displays in the Results field of the Event Status form. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each user viewing the Event Status form.	RESULT(<i>scalarExpr</i>) Example: RESULT("The transaction was disapproved by the purchasing manager.")
Result Set Assignment	● Load Collection	Designates the name (ID) of the result set where the system is to put the loaded collection.	SET(R(<i>ID</i>) = RESULT)
Result Set ID	● Update Collection	Identifies the name (ID) of a result set that was previously loaded using a Load Collection action type.	COLLECTION(<i>ID</i>)
Result XML	● Transform XML	Specifies the return type—return variable (RV) or return parameter (RE)—and name of the variable or parameter that is to receive the resulting XML transformation output.	SET(RV(<i>var</i>) = RESULT) SET(RE(<i>param</i>) = RESULT)
Retest Interval	● Wait	Designates the amount of time, in seconds, that the system is to wait before retesting for the condition.	RETEST(<i>numericExpr</i>)
Row	● Update Collection ● Set Values	Indicates the row (record) to be updated in the collection.	ROW(<i>numericExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
ROW (Row number and property/Expression pairs)	Set Values	Used only for the IdoPostLoadCollection framework (Core) event. Consists of the row number in the just-loaded collection, and a comma-separated list of: The name of the property to be set Equal sign (=) The expression to be evaluated	<code>ROW(<i>numericExpr</i>)</code> <code>SETPROPVALUES(<i>prop=scalarExpr, ...</i>)</code>
Save in Sent Items	- Notify - Prompt	Determines whether a copy of the message is to be saved in the originator's Sent Items folder.	<code>SAVEMESSAGE(<i>Boolean</i>)</code>
Session Variables	Set Values	Specifies session variables as name-value pairs. For multiple variables, include them as a comma-separated list.	<code>SETGLOBVALUES(<i>varName = scalarExpr[, ...]</i>)</code>
State	- Achieve Milestone - Finish	Indicates the text to display on the Event Status form, Event State field, when the action runs. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each user viewing the Event Status form.	<code>STATE(<i>scalarExpr</i>)</code>
Stored Procedure	Call Database Method	Specifies the name of the stored procedure to call.	<code>METHOD(<i>scalarExpr</i>)</code>
Subject	- Notify - Prompt - Send Email	Specifies the subject line for the message, whether for an internal system message or an email. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each Inbox user and email prompt recipient (applicable only for Notify and Prompt).	<code>SUBJECT(<i>scalarExpr</i>)</code>

Parameter	Used with action type(s)	Description/Purpose	Syntax
Synchronous	Generate Event	Instructs the system whether to treat the generated event as a synchronous event. TRUE = Synchronous FALSE = Asynchronous	SYNCHRONOUS(<i>Boolean</i>)
Task Name	Run Background Task	Specifies the name of the task to be placed on the background task queue, as defined on the Background Task Definitions form.	TASKNAME(<i>scalarExpr</i>)
Task Number	Run Background Task	Indicates the name of the event variable that is to receive the task number as output from the Task Manager when the task is placed on the background task queue.	TASKNUMBER(<i>ID</i>)
Task Parameters	Run Background Task	Specifies any parameters that the background task requires to run correctly.	TASKPARMS(<i>scalarExpr</i>)
Task Status	Run Background Task	Specifies the task status that the status is to have initially when it is placed on the Background Task Queue. READY – Places the task on the Background Task Queue to be run as soon as possible. WAITING – Places the task on the Background Task Queue to be run only when the scheduling requirements for that task, as set on the Background Queue form, are	Either of these: TASKSTATUS(READY) TASKSTATUS(WAITING)
Timeout	- Prompt - Wait	The amount of time (in seconds) that the system is to wait for a response (in the case of a Prompt action) or that it must wait before retesting (in the case of a Wait action).	TIMEOUT(<i>numericExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Timeout Destination	● Prompt ● Wait	Specifies which action sequence step the system should move to when the time-out limit has been reached. The action sequence step is identified by the Action Sequence number on the Event Actions form.	DEST(<i>action</i>)
Title	● Set Attributes	Specifies the Event Title as you want it to appear on the Event Status form, when the event is running or finished.	SET(EVENTTITLE = <i>scalarExpr</i>)
To	● Notify ● Prompt ● Send Email	Designates the list of the primary recipients of the message, delimited by semi-colons (;). Do not leave spaces between multiple recipients. For Notify and Prompt actions, recipients are identified by their user IDs. For Send Email actions, recipients are identified by their email addresses.	TO(<i>scalarExpr</i>) Examples: TO("jdoe;tsmith") TO("jdoe@provider.net") Note that literal values for user IDs or email addresses must be enclosed in double quotes ("").
Transactional	● Generate Event	Instructs the system whether to treat the generated event as transactional. For more information about transactional event handlers, see "Transactions" on page 28.	TRANSACTIONAL(Boolean)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Unmodified Payload Access	• Notify • Prompt	Specifies how message payload elements that represent unmodified properties are handled: • Default – The unmodified parts of the payload are visible; the recipient can modify any of their contents. • Hidden – The unmodified parts of the payload are hidden. This allows the recipient to focus on the modified parts of the payload when evaluating the question. • Read-Only – The unmodified parts of the payload are display-only, so the voter cannot modify the contents. This specification applies to event message variables with names in the form Row.Property that do not have a corresponding Row.Property.Modified element with a value of 1. This setting is overridden by settings for the Default Access (Event Variable Groups form) or Access (Event Actions form, Variables grid) for the same property name.	One of these: • UNMODIFIEDPAYLOAD ACCESS(DEFAULT) • UNMODIFIEDPAYLOAD ACCESS(HIDDEN) • UNMODIFIEDPAYLOAD ACCESS(READONLY)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Update Using IDO/Update Loaded Collection	Update Collection	Specifies what IDO collection is to be updated (mutually exclusive choices): Update Using IDO allows you to specify an IDO that is not currently loaded into the form. Update Loaded Collection (default) updates a collection that has already been loaded into a named result set. Each option enables a different set of parameter options.	N/A
URL	- Call Web Service - Dispatch IDO Request	Identifies the URL that is to receive the request. The URL must be in standard HTTP format.	URL(<i>scalarExpr</i>)
Use Local Password	Dispatch IDO Request	Instructs the system whether to automatically retrieve the password associated with the user ID (User Name) and include it in the IDO request. TIP: If you want to use this option, you must make sure that the remote system has an identical login (user name and password) to the one you want to use from the current local system.	USELOCALPASSWORD(<i>Boolean</i>)
User Name	Dispatch IDO Request	Designates a valid user ID for the remote system configuration. This allows a session to open. TIP: Make sure you specify a user ID that has permissions and licensing sufficient to do whatever you are requesting from the IDO on the remote system.	USERNAME(<i>scalarExpr</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
Value	Set Values	Used only for the SessionOnVarChanged framework event. Specifies a value to be assigned to the session variable that, when changed, generated the event. This value replaces whatever value was requested by the process that initiated the change. In effect, this overrides the original change request behind the scenes. For example, suppose you have an event that is set to be executed whenever a user tries to reset a particular session variable. This event can be used to override the user-initiated change and reset the variable to whatever value you designate here.	SETVALUE(<i>scalarExpr</i>)
Variables	Set Values	Specifies variable definitions as name-value pairs. For multiple variables, include them as a comma-separated list. If a variable does not exist when the event is executed, the system creates the variable when needed. TIP: Including a null value for a variable (such as myVar =) generates a syntax error. To designate a null value for the value of a variable, use a double set of quotation marks (myVar = "").	SETVARVALUES(<i>varName</i> = <i>scalarExpr</i> [, ...])

Parameter	Used with action type(s)	Description/Purpose	Syntax
Voting Rule	Prompt	Designates the method to be used for counting and evaluating votes. Possible values include: - Majority - Plurality - ConditionalPlurality - MinimumCount - MinimumPercentage - EarliestResponse - PreferredChoice - MinimumCountPreferredChoice - MinimumPercentagePreferredChoice For more information about these values, see “Voting rules” on page 66.	VOTINGRULE(<i>rule</i>) Example: VOTINGRULE(Plurality)
Wait for Quorum	Prompt	Specifies whether to await a quorum of responses before continuing to the subsequent event action on a prompt action: FALSE - Voting remains open, no messages are expired, and Timeout and Timeout Destination are ignored. TRUE - Execution is adjourned until a quorum is attained. Default - When Quorum is not specified, the default is TRUE. When Quorum is specified with a positive value, the default is TRUE. When Quorum is specified with a non-positive value, the default is FALSE.	WAITFORQUORUM(<i>Boolean</i>)

Parameter	Used with action type(s)	Description/Purpose	Syntax
XML	Transform XML	Specifies the XML code (content) to be transformed. The result can be retrieved using the RESULT function.	<i>XML(scalarExpr)</i>
XSLT	Transform XML	Designates the stylesheet (XSLT) or a reference to the XSLT file containing the transformation rules (using the FILECONTENTS function).	<i>XSLT(scalarExpr)</i>

Expression Functions

Standard Functions

To construct the expressions used in “Event Action Parameters” on page 147, use the standard expression functions in the table below, if necessary in conjunction with the “Expression Operators” on page 194.

For the complete grammar available for constructing expressions, see “Expression Grammar” on page 197.

When using the table, keep these points in mind:

This:	Refers to:
<i>FUNCTION(value)</i>	Syntax for functions and values.
Content in italics	Content that varies according to the context. For example, for a V(<i>varName</i>) function, <i>varName</i> represents the actual value (the variable name) to be used by the V (variable) function.
The identifiers listed as arguments below are not enclosed in double-quotes (" ") and should not be unless they are cast as literal string values in the resulting expression.	

Function / Syntax	What it does	Arguments	Return data type
ACTIONSEQ()	Returns the design-time sequence of the currently executing event action.	—	Numeric
ACTIONTYPENAME()	Returns the English-like name of the currently executing event action's type. This function returns the same value as the T-SQL function <code>dbo.EventActionTypeName()</code> when it is passed the <code>EventAction.ActionType</code> for this event action.	—	String
ANYHANDLERS-FAILED()	Indicates whether any event handlers for the running event have exited with a status of Failure .	—	Boolean
APPNAME()	Returns the name of the application to which the user is currently signed in. TIP: The name of the application is set in the Configuration Manager, Edit Application dialog box.	—	String
BEGINDATE()	Returns the date and time the event began executing.	—	Date/Time
CAST(<i>expr</i> AS <i>DATATYPE</i>)	Causes the system to treat the specified expression as the designated data type. For example, <code>CAST(P("CreditLimit") AS STRING)</code> would cause the system to treat the value of the property "CreditLimit" as a string, rather than as a number.	• <i>expr</i> – Expression of any data type • AS – Keyword, literal • <i>DATATYPE</i> – Keyword (must be STRING, DATE, or NUMBER)	Varies
CEILING()	Returns the least integer greater than or equal to a specified numeric expression. Note: This function is equivalent to CEILING in T-SQL.	—	Numeric

Function / Syntax	What it does	Arguments	Return data type
CLIENTSUBSTITUTE	Creates a string combined of literal values and replacements for which values are substituted at run time. CLIENTSUBSTITUTE() allows inclusion of literal commas and parentheses in the string. You can concatenate the string that results from evaluating the CLIENTSUBSTITUTE() expression with other strings. You can pass the string to SUBSTITUTE in order to build a larger expression, for example, BODY(), that contains the unchanged ~LIT~ () expressions.	• <i>pattern</i> – Evaluated into a string name • <i>arg</i> – Each argument is evaluated into a string, passed through WinStudio literal massaging, which converts commas and parentheses into special characters, and, unless the argument consists of an embedded STRINGS() or FORMAT() call, is wrapped in ~LIT~(). The resulting strings are separated by commas, in the same order given, and wrapped in a FORMAT() statement. This allows the presence of commas and/or parentheses within the arguments. WinStudio can still parse the arguments as originally separated. .	String
COMPANYNAME()	Returns the name of the company associated with the application to which the user is currently signed in. Tip: The name of the company is set using the Configuration Manager, Edit Application dialog box.	—	String
CONFIGNAME()	Returns the configuration name associated with the running event.	—	String

Function / Syntax	What it does	Arguments	Return data type
CURDATETIME()	Returns the current date and time, corrected for the server's time zone (except for the database server). Note: This function is equivalent to <code>dbo.GetSiteDate(getdate())</code> in T-SQL.	—	Date/Time
CUSTOMDELETE()	Returns the specification for a Custom Delete attribute.	—	String
CUSTOMINSERT()	Returns the specification for a Custom Insert attribute.	—	String
CUSTOMUPDATE()	Returns the specification for a Custom Update attribute.	—	String
DATE(<i>year, month, day</i>) or DATE(<i>year, month, day, hour, minute, second, millisecond</i>)	Returns a date and/or time with the given attributes. You must use one of the two syntax models, or the system returns an error.	• <i>year</i> – A four-digit integer • <i>month</i> – A one-digit or two-digit integer (1–12) • <i>day</i> – A one-digit or two-digit integer (1–31) • <i>hour</i> – A one-digit or two-digit integer (1–23) • <i>minute</i> – A one-digit or two-digit integer (0–59) • <i>second</i> – A one-digit or two-digit integer (0–59) • <i>millisecond</i> – A one-digit to three-digit integer (0–999)	Date/Time

Function / Syntax	What it does	Arguments	Return data type
DATEADD(<i>datePart</i> , <i>number</i> , <i>date</i>)	Returns a new date/time value based on adding or subtracting a number of intervals to the specified date. Note: This function is equivalent to DATEADD in T-SQL. For more information about possible values for arguments, see the online help topic for this function.	• <i>datePart</i> – The type of interval to be added to or subtracted from <i>date</i> • <i>number</i> – number of intervals to be added to or subtracted from <i>date</i>. For addition (resulting in a value later than <i>date</i>), use a positive value; for subtraction (resulting in a value earlier than <i>date</i>), use a negative value. • <i>date</i> – An expression that returns the Date/Time value to or from which you want to add or subtract.	Date/Time
DATEDIFF(<i>datePart</i> , <i>startDate</i> , <i>endDate</i>)	Returns the number of date or time boundaries crossed between two specified dates. For example, you could use this function to find the number of days in a certain interval of dates. The <i>startDate</i> value is subtracted from the <i>endDate</i>. If the <i>startDate</i> is later than the <i>endDate</i>, then the return value is a negative number. If the result is out of range for integer values, the system returns an error. Note: This function is equivalent to DATEDIFF in T-SQL. For more information about possible values for arguments, see the online help topic for this function.	• <i>datePart</i> – Specifies the type of interval on which to calculate the difference • <i>startDate</i> – The beginning date for the calculation • <i>endDate</i> – The ending date for the calculation	Numeric

Function / Syntax	What it does	Arguments	Return data type
DATEPART(<i>datePart</i> , <i>date</i>)	Returns a number indicating the value of a part of a date value for a specified date. For example, if you wanted the value of the month for a date, you might get a return value of 10 (for October). Note: This function is equivalent to DATEPART in T-SQL. For more information about possible values for arguments, see the online help topic for this function.	• <i>datePart</i> – The part of the date for which you want to return the value • <i>date</i> – The date for which you want to return the part value	Numeric
DBFUNCTION(<i>functionName</i> , <i>arg1</i> , <i>arg2</i> , ...)	Calls a database function. Note: This function is available only for database operations.	• <i>functionName</i> – The name of the database function you want to call • <i>arg1</i>, <i>arg2</i>, ... – Any arguments required by the database function	Typeless
E(<i>param</i>)	Returns the value of an event parameter.	<i>param</i> – The name of the event parameter	Typeless
EVENTNAME()	Returns the name of the event being processed.	—	String
EVENTREVISION()	Returns the design-time revision number of the active group of handlers of the event being processed.	—	Numeric
EVENTSTATE()	Returns the designer-defined state of the running event.	—	String
EVENTTITLE()	Returns the title associated with a specified event.	—	String
FE(<i>param</i>)	Retrieves the value of an event parameter as a string enclosed in quotes. If the parameter is empty, returns NULL as an unquoted string.	<i>param</i> – Name of the parameter for which the value is being returned	String

Function / Syntax	What it does	Arguments	Return data type
FGC(<i>constant</i>)	Retrieves the value of an event global constant as a string enclosed in quotes. If the global constant has no value assigned, returns NULL as an unquoted string.	<i>constant</i> – Name of the global constant for which the value is being returned	String
FILECONTENTS(<i>location</i>)	Returns the contents of a named text file at a specified location.	<i>location</i> – Location of the text file using the standard format: C:\myFiles\myFolder\myTextFile.txt Tip: This file is not required to be a *.txt file, necessarily, but it must be a file that is text-based to be useful. Other useful file types include (but are not limited to) *.htm, *.rtf, *.xml, and so on. For more information, see the online help topic for this function.	String
FILTER(<i>expr</i>)	Converts and presents an expression as a string enclosed in quotes. If the expression is empty, presents it as NULL with no quotes.	<i>expr</i> – The expression to be converted and presented	String
FILTERMETHODPARM (<i>numExpr</i>)	Returns the value of the parameter for an IDO method call, at a specified index number. The value is returned as a string in quotes. If the parameter value is empty, returns NULL as an unquoted string. This function is used only with the IdoOnInvoke and IdoPostInvoke framework events.	<i>numExpr</i> – An integer or expression that resolves to an integer	String

Function / Syntax	What it does	Arguments	Return data type
FILTERPROPERTY([<i>collection</i> ,] <i>row#</i> ,] <i>propertyName</i>)	Returns the value of a specified property, optionally in a designated row and collection result set, as a string enclosed in quotes. If the value is empty, returns NULL as an unquoted string. NOTE: Functionally, this is equivalent to the FP function. For more information, see the online help for this function.	• <i>collection</i> – The name of a result set containing a collection you loaded earlier using a Load Collection action. Optional. • <i>row#</i> – The one-based number of the row containing the property value you want to retrieve NOTE: This argument is used only with the <code>IdoOnLoadCollection</code> and <code>IdoPostLoadCollection</code> framework events, or with a Load Collection action. • <i>propertyName</i> – The name of the property for which you want to retrieve the value Note: When using the <code>IdoOnItem</code> or <code>IdoPostItem</code> framework events to retrieve a property that is part of that event, this is the only accepted parameter.	String
FILTERSTRING()	Returns filter used during an event. Applies only to the <code>IdoOnLoadCollection</code> and <code>IdoPostLoadCollection</code> framework (Core) events.	—	String

Function / Syntax	What it does	Arguments	Return data type
FLOOR()	Returns the greatest integer that is less than or equal to the value of a number or numeric expression. Note: This function is equivalent to FLOOR in T-SQL.	—	Numeric
FP([[collection,] row#,) propertyName)	See FILTERPROPERTY.		String
FSV(var)	Returns the value of a session variable as a string enclosed in quotes. If the variable is empty, returns NULL as an unquoted string.	var – The name of the session variable for which you want to return the value	String
FV(var)	Returns the value of an event variable as a string enclosed in quotes. If the variable is empty, returns NULL as an unquoted string.	var – The name of the variable for which you want to return the value	String
GC(constant)	Retrieves the value of an event global constant.	constant – The name of the event global constant for which you want to return the value	Typeless
HANDLERIGNORESFAILURE()	Indicates whether the currently executing event handler ignores failures.	—	Boolean
HANDLERSEQ()	Returns the design-time sequence of the currently executing event handler.	—	Numeric
HANDLERSUSPENDS()	Indicates whether the currently executing event handler is part of a chain that suspends a framework IDO event.	—	Boolean
HANDLERSYNCHRONOUS()	Indicates whether the currently executing event handler is part of a chain that was fired synchronously and has not adjourned.	—	Boolean

Function / Syntax	What it does	Arguments	Return data type
HANDLERTRANSACTIONAL()	Indicates whether the unadjoined portion of the currently executing event handler is (or was) enclosed in a database transaction.	—	Boolean
HASBEGUN(<i>action</i>)	Indicates whether a specified action has begun executing.	<i>action</i> – Action sequence number	Boolean
HASFINISHED(<i>action</i>)	Indicates whether the specified action has finished executing.	<i>action</i> – Action sequence number	Boolean
IDO()	Returns the name of the specified IDO collection for which an operation was requested.	—	String
IF(<i>condition</i> , <i>true_expression</i> , <i>false_expression</i>)	Depending on the outcome of a condition test, returns one of two expressions.	• <i>condition</i> – The condition to be tested for • <i>true_expression</i> – The expression to be returned if the condition tests TRUE • <i>false_expression</i> – The expression to be returned if the condition tests FALSE	Can vary
INITIATOR()	Returns an identifier indicating the object or process that initiated an event. This specification normally consists of a composite initiator type (form or other client) and name (form name or other client ID), separated by a period.	—	String
INSIDEDATABASE()	Indicates whether the current event action is executing inside the database.	—	Boolean

Function / Syntax	What it does	Arguments	Return data type
INSTR(<i>string1</i> , <i>string2</i>)	Returns the starting position of the first character in one string (<i>string2</i>) within another string (<i>string1</i>). If <i>string2</i> is not found within <i>string1</i> , then this function returns zero (0).	• <i>string1</i> – String to be evaluated • <i>string2</i> – The (sub)string you are looking for within <i>string1</i>, for which the starting position is to be returned	Integer
LEN(<i>string</i>)	Returns the length of a specified string, as a number of characters.	<i>string</i> – The literal string or an expression that resolves to a string	Integer
LOADFLAGS()	Returns a comma-separated list of the options applicable to an event: • A Position/Direction indicator: First, Next, Prev, or Last • An optional Distinct indicator: Distinct (If load is not flagged Distinct, this indicator is not present.) • An optional Read Mode indicator: ReadCommitted, or ReadUncommitted (If the load is flagged Default Read Mode, this indicator is not present.) Indicators appear in the order shown above, if present. Applies only to the IdoOnLoadCollection and IdoPostLoadCollection framework (Core) events	—	String
LOWER(<i>string</i>)	Returns a string with uppercase characters converted to lowercase.	<i>string</i> – String to be converted to lowercase	String

Function / Syntax	What it does	Arguments	Return data type
MESSAGE(<i>msgID</i> [, <i>msgParam1</i> , <i>msgParam2</i> , ...])	Constructs and returns one or more messages using MsgAppSp. For more information, see the online help for this function.	• <i>msgID</i> – Message identifier containing the object name • <i>msgParam#</i> – The parameters required to construct the message	String
METHOD ()	Returns the name of the method being invoked. This function is used only with the IdoOnInvoke and IdoPostInvoke framework events.	—	String
METHODPARAM(<i>numExpr</i>)	Returns the value of a parameter for an IDO method call, at the specified index number. This function is used only with the IdoOnInvoke and IdoPostInvoke framework events.	<i>numExpr</i> – A number or numeric expression that tells the system what index number to go to for the parameter value	Typeless
METHODPARMS()	Returns a count of the number of parameters required by the method. This function is used only with the IdoOnInvoke and IdoPostInvoke framework events.	—	Integer
NEWGUID()	Generates a new globally unique identifier (GUID).	—	String
NONRESPONDERLIST(<i>action</i>)	Returns a semicolon (;) delimited list of recipients who have not responded to a specified Prompt message.	<i>action</i> – Action sequence number	String

Function / Syntax	What it does	Arguments	Return data type
ORIGINATOR()	Returns the user ID for the user who initiated the running of the event handler. This function can be useful when you need to identify who initiated the action that triggered the event or when you want the user who initiated the action to receive a copy of the message, for example.	—	String
P([<i>collection</i> ,] <i>row#</i> ,] <i>propertyName</i>)	See PROPERTY.		Typeless
POSTQUERY-ACTIONS()	Returns the list of post-query actions to be executed (server side) for every row returned by an event. Applies only to the IdoOnLoadCollection and IdoPostLoadCollection framework (Core) events.	—	String
POWER(<i>numExpr</i> , <i>power</i>)	Returns the value of a specified numeric expression to a designated power. NOTE: This function is equivalent to POWER in T-SQL.	• <i>numExpr</i> – A number or an expression that resolves to a numeric value • <i>power</i> – the exponent, or power to which the numExpr is to be raised	Numeric

Function / Syntax	What it does	Arguments	Return data type
PROPERTY([<i>collection</i> ,] <i>row#</i> , <i>propertyName</i>)	Returns the value of a specified property, optionally in a designated row and collection result set.	• <i>collection</i> – The name of a result set containing a collection you loaded earlier using a Load Collection action. Optional. • <i>row#</i> – The one-based number of the row containing the property value you want to retrieve Note: This argument is used only with the <code>IdoOnLoadCollection</code> and <code>IdoPostLoadCollection</code> framework events, or with a Load Collection action. • <i>propertyName</i> – The name of the property for which you want to retrieve the value Note: When using the <code>IdoOnItem</code> or <code>IdoPostItem</code> framework events to retrieve a property that is part of that event, this is the only accepted parameter.	Typeless
PROPERTY-MODIFIED(<i>propertyName</i>)	Indicates whether a specified property has been modified or not. This function is used only with the <code>IdoOnItem</code> framework events.	<i>propertyName</i> – The name of the property for which you want to know whether it has been modified	Boolean
PROPERTY-NAMES()	Returns a list of properties to be retrieved in the current operation.	—	String

Function / Syntax	What it does	Arguments	Return data type
RECIPIENTLIST(<i>action</i>)	Returns a semicolon (;) delimited list of recipients who received a message sent as the result of a specified action. The target action step must be a Notify or Prompt action type.	<i>action</i> – The action sequence number for the action that sent the message (Notify or Prompt)	String
RECIPIENTS(<i>action</i>)	Returns the number of recipients who received the message sent by a specified action. The target action step must be a Notify or Prompt action type.	<i>action</i> – The action sequence number for the action that sent the message (Notify or Prompt)	Integer
RECORDCAP()	Returns the record cap in effect during an event. Applies only to the IdoOnLoadCollection and IdoPostLoadCollection framework (Core) events.	—	Integer
REPLACE(<i>string1</i> , <i>string2</i> , <i>string3</i>)	Within a string, replaces all instances of a specified substring with a different substring.	• <i>string1</i> – String in which the replacements are to be made • <i>string2</i> – Substring to be replaced • <i>string3</i> – Replacement substring Any of these strings can be referenced using an expression.	String

Function / Syntax	What it does	Arguments	Return data type
RESPONDERLIST(<i>action</i> [, <i>choice</i>])	Returns a semicolon (;) delimited list of recipients who have responded to a specified Prompt message, optionally with a selected choice.	● <i>action</i> – Action sequence number for the prompt action that produced the messages whose responses are to be checked. ● <i>choice</i> – The particular choice for which you want to see who selected it. Optional. If omitted, the list includes all responders who selected any choice. The value for this argument must be a valid return value for the Prompt choices as defined on the Event Action Prompt and Event Action Prompt Choices forms.	String

Function / Syntax	What it does	Arguments	Return data type
RESPONDERS(<i>action</i> [, <i>choice</i>])	Indicates the number of recipients who have responded to a specified Prompt message, optionally for a selected choice.	● <i>action</i> – The action sequence number for the Prompt action ● <i>choice</i> – The particular choice for which you want to see how many selected it. Optional. If omitted, the return value is a total of all who responded, regardless of their choices. The value for this argument must be a valid return value for the Prompt choices as defined on the Event Action Prompt and Event Action Prompt Choices forms.	Integer

Function / Syntax	What it does	Arguments	Return data type
ROUND(<i>numExpr</i> , <i>length</i>)	Returns a numeric expression rounded to the specified length or precision. NOTE: This function is equivalent to ROUND in T-SQL.	• <i>numExpr</i> – The numeric expression containing the value you want to have rounded • <i>length</i> – The number of places or the precision to which the number should be rounded. When <i>length</i> is a positive number, <i>numExpr</i> is rounded to the number of decimal positions specified by <i>length</i>. When <i>length</i> is a negative number, <i>numExpr</i> is rounded on the left side of the decimal point, as specified by <i>length</i>.	Numeric
ROWS(<i>collection</i>)	Returns a number indicating how many records (rows) are in a specified result set (as the result of a Load Collection action).	<i>collection</i> – The name of the result set for which you want to know how many records have been loaded	Integer
ROWS()	Returns a number indicating how many rows were loaded by the collection that generated an event. Applies only to the IdopostLoad-Collection event.	—	Integer

Function / Syntax	What it does	Arguments	Return data type
SUBSTITUTE(<i>string</i> , <i>arg1</i> [, <i>arg2</i> , ...])	Creates a string combined of literal values and "replacement zones" for which values are substituted at run time. Replacement zones are indicated by the use of curly braces { } enclosing numbers. Numbers must start with zero and be consecutive, although they need not necessarily appear in the string in the correct order. For more information, see the online help topic for this function.	• <i>string</i> – The basic literal string containing the replacement zones • <i>arg#</i> – The sources for the replacement text to go into the replacement zones. These are typically expressions. These must appear in the order they are numbered within the <i>string</i>. That is, the first argument (<i>arg1</i>) is used for {0}; the second argument (<i>arg2</i>) is used for {1}, and so forth.	String

Function / Syntax	What it does	Arguments	Return data type
SUBSTRING(<i>string</i> , <i>start</i> , <i>length</i>)	Returns part of a designated string.	● <i>string</i> – An expression containing the string to be processed ● <i>start</i> – A number indicating the starting position from which the substring is to be returned. The first character of a string is position 1. If <i>start</i> indicates a position beyond the length of <i>string</i>, the system returns an empty string. ● <i>length</i> – (Optional) A number indicating how many characters are to be returned. If <i>length</i> indicates more characters than are remaining in the string from the start location to the end of the string, or if this argument is not used, the system returns the entire remainder of the string from the start location.	String
SV(<i>var</i>)	Retrieves the value of a designated session variable.	<i>var</i> – The name of the session variable	Typeless

Function / Syntax	What it does	Arguments	Return data type
TRUNC(<i>numExpr</i> , <i>length</i>)	Returns a numeric value, rounded to the specified length or precision and then truncated to the appropriate number of places. NOTE: This function is equivalent to ROUND in T-SQL, with a third argument (function) of 1.	• <i>numExpr</i> – The numeric expression containing the value you want to have rounded and then truncated • <i>length</i> – The number of places or the precision to which the number should be rounded and then truncated When <i>length</i> is a positive number, <i>numExpr</i> is rounded to the number of decimal positions specified by <i>length</i>. When <i>length</i> is a negative number, <i>numExpr</i> is rounded on the left side of the decimal point, as specified by <i>length</i>.	Numeric
UPPER(<i>string</i>)	Returns a string with lowercase characters converted to uppercase.	<i>string</i> – the string expression to be converted to uppercase	String
USERDESC()	Returns the user description, as specified in the User Description field of the Users form, for the user whose action initiated the event handler.	—	String
USERNAME()	Returns the user logon ID for the system user either logging on or logging off the system.	—	String
V(<i>var</i>)	Retrieves the value of a specified event variable.	<i>var</i> – The name of the variable for which you want to retrieve the value	Typeless
VARIABLENAME()	Returns the name of the session variable for which a requested change triggered the SessionOnVarChanged event.	—	String

Function / Syntax	What it does	Arguments	Return data type
VARIABLEVALUE()	Returns the new value of the session variable for which a requested change triggered the SessionOnVarChanged event.	—	String
VOTINGDISPARITY(<i>action</i>)	Indicates whether any two selected choices disagree for a specified Prompt action. A return of: ● FALSE indicates that there is no disagreement. ● TRUE indicates that at least two responders' selections disagree. For more information, see “Dealing with indeterminate voting results” on page 69.	<i>action</i> – The action sequence (step) number for the Prompt action	Boolean
VOTINGRESULT(<i>action</i>)	Returns the winning choice resulting from the voting on a specified Prompt action.	<i>action</i> – The action sequence (step) number for the Prompt action	String
VOTINGTIE(<i>action</i>)	Indicates whether a specified Prompt action has resulted in a tie vote. A return of: ● FALSE indicates that it was not a tie vote. ● TRUE indicates that it was a tie vote.	<i>action</i> – The action sequence (step) number for the Prompt action	Boolean
WORKINGDIR()	Returns the working directory for the event handler that is running. For more detailed information, see the online help topic for this function. Note: This function is equivalent to the WORKINGDIR() method in the WinStudio API.	—	String

Pre-parser Functions

These additional functions can be used to expand an event variable or event global constant textually, that is, with no assumptions about the structure or data-type of the contained value. These functions are useful in cases where the expanded value contains other grammar elements that must be further evaluated, for example, to share common expressions, expression elements, or groups of functions.

Function / Syntax	What it does	Arguments	Return data type
TGC(<i>conName</i>)	Retrieves the value of an event global constant for which the value can contain other grammar elements.	<i>conName</i> – The name of the global constant	Varies
TV(<i>varName</i>)	Retrieves the value of an event variable in which that value can contain other grammar elements.	<i>varName</i> – The name of the variable containing other elements	Varies

When an event action begins, the system first evaluates all TV() references recursively, until no more references to known variables remain. Next, the system evaluates all TGC() references recursively, until no more references to known global constants remain. Finally, the resulting parameters string is passed to the parser to evaluate all other functions and operators contextually using the grammar found in Appendix C, “Expression Grammar.”

For example, consider this parameters string:

```
CONDITION(TGC(Over1WeekOld)) TGC(OldHandlerPromptParms)
```

Assume that the system is using this event global constant metadata:

Name	Value
Over1WeekOld	DATEDIFF(day, BEGINDATE(), CURDATETIME()) > 7
OldHandlerPromptParms	TO(USERNAME()) SUBJECT("Old Handler Alert") QUESTION("Do you really want to continue this old Handler?") CHOICES("1,sYes,0,sNo")

The effective parameters passed to the event system parser would be:

```
CONDITION(DATEDIFF(day, BEGINDATE( ), CURDATETIME( )) > 7) TO(USERNAME( ))  
SUBJECT("Old Handler Alert") QUESTION("Do you really want to continue this  
old Handler?") CHOICES("1,sYes,0,sNo")
```

Complex Expansions

More complex expansions are possible. For example:

```
GC(MyVarTV(VarSuffix))
```

In this example, first, the value of the current event handler's "VarSuffix" variable is retrieved. Then that value is appended to the name "MyVar" to construct an event global constant name, for which the value is then retrieved.

The value of the "VarSuffix" variable might be set dynamically by an event action on the handler, or it might be included in different event initial states that are linked from referring handlers, in which case the GC() reference itself could be moved to its own global constant and referred to from all actions of the handlers using TGC().

Note that no operator or other syntax is used around the TV() reference, because it is evaluated and substituted in-place textually, without regard to data type or context.

Nested Expansions

Nested expansions are also possible. For example, consider this parameters string:

```
TGC(MarkItUp)
```

Assume that the system is using this event global constant metadata:

Name	Value
StdMarkup	* 0.1 + 5
MarkItUp	CONDITION(V(Price)TGC(StdMarkup) < 100) SETVARVALUES(Price=Price TGC(StdMarkup))

This has the effect of increasing the value of the "Price" variable by 10% plus 5, but only if the resulting value is less than 100.

Expression Operators

Expression operators can be either unary or binary, meaning they can operate on either one or two expressions. Most operators are limited as to what kind of expressions they can operate with. The kinds of expressions possible include:

- Scalar expressions (*scalarExpr*) – These expressions can be either of a known type (numeric, string, or date) or an unknown type (typeless).
In the table below, if the expression type is given as *scalarExpr*, it can be any of these four types.
- Numeric expressions (*numericExpr*) – These expressions evaluate using numeric values. These are the expressions used to perform mathematic operations. If a string or typeless expression is supplied where a numeric expression is expected, it is automatically converted into a numeric value. If that is not possible due to the presence of non-numeric characters, the current handler fails with an error.

- String expressions (*stringExpr*) – These expressions are text-based sets of characters. They may include numbers, but if so, the numbers are treated as text characters, not numerals. If another type of expression is supplied where a string expression is expected, it is automatically converted into a string representation.
- Date expressions (*dateExpr*) – These expressions involve dates or parts of dates, including times such as 10:00AM.
- Typeless expressions (*typelessExpr*) – These are expressions that could be one of at least two different types. The way the system treats these expressions depends on the context in which the expression is used.
- Boolean expressions (*BooleanExpr*) – These expressions consist of an OR conjunction of one or more AND conjunctions.

To construct the expressions used in “Event Action Parameters” on page 147, you can use the expression operators in this table in conjunction with the “Expression Functions” on page 171

Operator	Purpose	Example
<i>scalarExpr</i> = <i>scalarExpr</i>	Is equal to	V(<i>var</i>) = 1
<i>scalarExpr</i> != <i>scalarExpr</i> <i>scalarExpr</i> <> <i>scalarExpr</i>	Is not equal to	V(<i>var</i>) != 1
<i>scalarExpr</i> > <i>scalarExpr</i>	Is greater than	V(<i>var</i>) > 1
<i>scalarExpr</i> < <i>scalarExpr</i>	Is less than	V(<i>var</i>) < 1
<i>scalarExpr</i> >= <i>scalarExpr</i>	Is greater than or equal to	V(<i>var</i>) >= 1
<i>scalarExpr</i> <= <i>scalarExpr</i>	Is less than or equal to	V(<i>var</i>) <= 1
<i>scalarExpr</i> : <i>list</i> <i>scalarExpr</i> IN <i>list</i>	Is in the list. The list is enclosed in parentheses and the elements separated by semi-colons.	V(<i>var</i>) IN (1;2;3;4) "b" : ("a";"b";"c")
<i>scalarExpr</i> !: <i>list</i>	Is not in the list. The list is enclosed in parentheses and the elements separated by semi-colons.	V(<i>var</i>) !: (1;2;3;4)
<i>BooleanExpr</i> AND <i>BooleanExpr</i>	Boolean AND	V(<i>var</i>) = 1 AND V(<i>var2</i>) > 5
<i>BooleanExpr</i> OR <i>BooleanExpr</i>	Boolean OR	V(<i>var</i>) = 1 OR V(<i>var2</i>) > 5
NOT <i>BooleanExpr</i>	Boolean negation	NOT(V(<i>var</i>) = 1 OR V(<i>var2</i>) > 5)
<i>numericExpr</i> + <i>numericExpr</i>	Addition	V(<i>var</i>) + 1
<i>stringExpr</i> + <i>stringExpr</i>	String concatenation	V(<i>var</i>) + "Item"
<i>numericExpr</i> – <i>numericExpr</i>	Subtraction	V(<i>var</i>) – 1

Operator	Purpose	Example
--numericExpr	Unary negative	-- V(var)
$\text{numericExpr * numericExpr}$	Multiplication	V(var) * 5
$\text{numericExpr / numericExpr}$	Division	V(var) / 2
$\text{stringExpr LIKE stringExpr}$	String similarity using WinStudio filtering syntax: • The underscore (<code>_</code>) character represents a single wildcard character. • The asterisk (<code>*</code>) represents any number of wildcard characters.	V(var) LIKE 'A*'

For the complete grammar available for constructing expressions, see Appendix C, “Expression Grammar.”

Appendix C: Expression Grammar

A red square containing a white capital letter 'C'.

This appendix contains the complete grammar available for constructing expressions for Event Action Parameters. Major sections include:

- Restrictions
- Start Symbol
- Character Sets
- Terminals
- Rules
 - Variable, Constant, and Event Parameter References
 - Expressions
 - Boolean Rules
 - Typeless Rules
 - String Rules
 - Numeric Rules
 - Date Rules
 - Restricted Arguments
- Keyword Paren Lists
- Enumerations

Restrictions

Expressions in this grammar are subject to these restrictions:

- Superfluous parentheses are usually not allowed.

For example, this produces a parsing error:

```
V(this)=1 and 2=V(that)) and 1=(2)
```

Instead, you can write it as follows to work around the grammar limitation:

```
V(this)=1 and 2=V(that) and 1=2
```

- Operation chains containing a mix of typed and typeless arguments must begin with a typed value.

For example, this example produces an error because the first argument is not a typed value:

```
'12'=V(a)+'B'+V(c)
```

Instead, and because string concatenation is not commutative, you can write it as:

```
'12'='' + V(a) + 'B' + V(c)
```

Alternatively, you can declare the type for the first argument as:

```
'12'=CAST(V(a) AS STRING)+'B'+V(c)
```

Again, this example produces an error because the first argument is not a typed value:

```
12>V(b)+1+5
```

Instead, you can write the expression in one of these ways:

```
12>0+V(b)+1+5
```

```
12>CAST(V(b) AS NUMBER)+1+5
```

```
12>1+V(b)+5
```

The last solution works because numeric addition *is* commutative.

- Functions cannot be used for variable, parameter, or constant names.

For example, because the letter *y* is an abbreviation for year in the `DATEPART()` and `DATEDIFF()` event functions, this expression produces an error:

```
y = 5
```

Instead, you can declare *y* as *yVar*. This expression does not produce an error.

```
yVar = 5
```

Start Symbol

"Start Symbol" = *<functionParenList>*

Character Sets

This table lists and describes the acceptable characters for elements of the code described. You can include any amount of white space between elements.

This	Designates
<i>Number</i>	The set of numerals: 0123456789
<i>Letter</i>	The set of all uppercase and lowercase letters: abcdefghijklmnopqrstuvwxyz ABCDEFGHIJKLMNOPQRSTUVWXYZ
<i>Alphanumeric</i>	The set of all characters listed as part of the <i>Number</i> and <i>Letter</i> sets
<i>Printable</i>	The set of all standard characters that can be printed onscreen. This includes the characters from #32 to #127 and #160 (nonbreaking space). The nonbreaking space character is included because it is often used in source code.
<i>Whitespace</i>	The set of all characters that are normally considered "white space" and ignored by the parser. The set consists of: • A space (regular) • Horizontal tab • Line feed • Vertical tab • Form feed • Carriage return • Nonbreaking space

$\{ID\ Head\} = \{Letter\} + \square$

$\{ID\ Tail\} = \{Alphanumeric\} + \square + [''] + ['] + [.]$

$\{String\ Ch\ 1\} = \{Printable\} - [''] + \{LF\} + \{CR\}$

$\{String\ Ch\ 2\} = \{Printable\} - [''] + \{LF\} + \{CR\}$

Terminals

String constants are constructed using one of these rules. The string can be:

- Enclosed in double-quotes, containing no double-quote characters as illustrated by this example:

```
StringLiteral = "{String Ch 1}*"
```
- Enclosed in double-quotes, containing paired double-quotes that are each interpreted as a single embedded double-quote character as illustrated by this example:

```
StringLiteral = "\"\"|{String Ch 1})*"
```

-
- Enclosed in single-quotes, containing no single-quote characters as illustrated by this example:

```
StringLiteral = '{String Ch 2}*' 
```

- Enclosed in single-quotes, containing paired single-quotes that are each interpreted as a single single-quote character as illustrated by this example:

```
StringLiteral = '({String Ch 2})*' 
```

Each string constant in an expression or parameter list can be constructed using any of these rules, independently of other string constants.

Integer constants contain no decimal point:

```
IntegerLiteral = {Digit}+ 
```

Real constants contain a decimal point:

```
RealLiteral = {Digit}+.{Digit}+ 
```

Identifiers (variable, constant, and event parameter names) must begin with a letter or underscore, and continue with zero or more alphanumeric characters and/or underscores:

```
Id = {ID Head}{ID Tail}* 
```

Rules

Variable, Constant, and Event Parameter References

<IdValue> consists of one of these:

```
V(Id)
GC(Id)
SV(Id)
E(Id)
```

<FilterIdValue> consists of one of these:

```
FV (Id)
FGC (Id)
FSV (Id)
FE (Id)
```

Expressions

A *scalar expression* is either of a known type (Number, Date, or String), or its type is unknown.

`<Scalar Exp>` consists of one of these:

```
<Numeric-castable Exp>
<Date Exp>
<String Exp>
<Typeless Exp>
```

A *numeric-castable expression* is either Number, String, or an unknown type.

`<Numeric-castable Exp>` consists of one of these:

```
<Numeric Exp>
```

Boolean Rules

A *Boolean expression* is an OR-conjunction of one or more AND-conjunctions.

`<Boolean Exp>` consists of one of these:

```
<And Exp>
<And Exp> OR <Boolean Exp>
```

An *AND expression* is an AND-conjunction of one or more negatable predicates.

`<AND Exp>` consists of one of these:

```
<Not Exp>
<Not Exp> AND <And Exp>
```

A *NOT expression* is a negatable predicate.

`<Not Exp>` consists of one of these:

```
NOT <Predicate>
<Predicate>
```

A *predicate* consists of one of these:

- A comparison of like-typed expressions
- A call to a Boolean function
- A Boolean sub-expression enclosed in parentheses

`<Predicate>` consists of one of these:

```
<String Exp> LIKE <String Exp>
```

```
<String Exp> IN <Scalar Tuple>
<String Exp> : <Scalar Tuple>
<String Exp> !: <Scalar Tuple>
<String Exp> = <String Exp>
<String Exp> <> <String Exp>
<String Exp> != <String Exp>
<String Exp> > <String Exp>
<String Exp> >= <String Exp>
<String Exp> < <String Exp>
<String Exp> <= <String Exp>
<String Exp> LIKE <Typeless Exp>
<String Exp> = <Typeless Exp>
<String Exp> <> <Typeless Exp>
<String Exp> != <Typeless Exp>
<String Exp> > <Typeless Exp>
<String Exp> >= <Typeless Exp>
<String Exp> < <Typeless Exp>
<String Exp> <= <Typeless Exp>
<Typeless Exp> LIKE <String Exp>
<Typeless Exp> = <String Exp>
<Typeless Exp> <> <String Exp>
<Typeless Exp> != <String Exp>
<Typeless Exp> > <String Exp>
<Typeless Exp> >= <String Exp>
<Typeless Exp> < <String Exp>
<Typeless Exp> <= <String Exp>
<Date Exp> IN <Scalar Tuple>
<Date Exp> : <Scalar Tuple>
<Date Exp> !: <Scalar Tuple>
<Date Exp> = <Date Exp>
<Date Exp> <> <Date Exp>
<Date Exp> != <Date Exp>
<Date Exp> > <Date Exp>
<Date Exp> >= <Date Exp>
<Date Exp> < <Date Exp>
<Date Exp> <= <Date Exp>
<Date Exp> = <Typeless Exp>
<Date Exp> <> <Typeless Exp>
<Date Exp> != <Typeless Exp>
<Date Exp> > <Typeless Exp>
<Date Exp> >= <Typeless Exp>
<Date Exp> < <Typeless Exp>
<Date Exp> <= <Typeless Exp>
<Typeless Exp> = <Date Exp>
<Typeless Exp> <> <Date Exp>
<Typeless Exp> != <Date Exp>
<Typeless Exp> > <Date Exp>
<Typeless Exp> >= <Date Exp>
```

```

<Typeless Exp> < <Date Exp>
<Typeless Exp> <= <Date Exp>
<Numeric Exp> IN <Scalar Tuple>
<Numeric Exp> : <Scalar Tuple>
<Numeric Exp> !: <Scalar Tuple>
<Numeric Exp> = <Numeric Exp>
<Numeric Exp> <> <Numeric Exp>
<Numeric Exp> != <Numeric Exp>
<Numeric Exp> > <Numeric Exp>
<Numeric Exp> >= <Numeric Exp>
<Numeric Exp> < <Numeric Exp>
<Numeric Exp> <= <Numeric Exp>
<Numeric Exp> = <Typeless Exp>
<Numeric Exp> <> <Typeless Exp>
<Numeric Exp> != <Typeless Exp>
<Numeric Exp> > <Typeless Exp>
<Numeric Exp> >= <Typeless Exp>
<Numeric Exp> < <Typeless Exp>
<Numeric Exp> <= <Typeless Exp>
<Typeless Exp> = <Numeric Exp>
<Typeless Exp> <> <Numeric Exp>
<Typeless Exp> != <Numeric Exp>
<Typeless Exp> > <Numeric Exp>
<Typeless Exp> >= <Numeric Exp>
<Typeless Exp> < <Numeric Exp>
<Typeless Exp> <= <Numeric Exp>
<Typeless Exp> IN <Scalar Tuple>
<Typeless Exp> : <Scalar Tuple>
<Typeless Exp> !: <Scalar Tuple>
<Typeless Exp> = <Typeless Exp>
<Typeless Exp> <> <Typeless Exp>
<Typeless Exp> != <Typeless Exp>
<Typeless Exp> > <Typeless Exp>
<Typeless Exp> >= <Typeless Exp>
<Typeless Exp> < <Typeless Exp>
<Typeless Exp> <= <Typeless Exp>
TRUE
FALSE
ANYHANDLERSFAILED ( )
HANDLERSYNCHRONOUS ( )
HANDLERSUSPENDS ( )
HANDLERTRANSACTIONAL ( )
HANDLERIGNORESFailure ( )
VOTINGDISPARITY (<EventActionRef>)
VOTINGTIE (<EventActionRef>)
HASBEGUN (<EventActionRef>)
HASFINISHED (<EventActionRef>)
INSIDEDATABASE ( )

```

```
PROPERTYMODIFIED (<String Exp>)  
PROPERTYMODIFIED (<Typeless Exp>)
```

Sub-expression:

```
(<Boolean Exp>)
```

Typeless Rules

A *typeless expression* is a concatenation or sum of elements whose type (between String, Numeric, and Date) we cannot distinguish without context.

<Typeless Exp> consists of one of these:

```
<Typeless Exp> + <Typeless Value>  
<Typeless Value>
```

A *typeless value* can be any of these:

- A variable reference
- A function call whose type cannot be determined without context
- A typeless sub-expression enclosed in parentheses

<Typeless Value> consists of one of these:

```
<IdValue>  
IF (<Boolean Exp>, <Typeless Exp>, <Typeless Exp>)  
DBFUNCTION (<Parameter List>)
```

Framework Event Parameters:

```
PROPERTY (Id, <Numeric Exp>, <String Exp>)
PROPERTY (Id, <Numeric Exp>, <Typeless Exp>)
PROPERTY (Id, <Typeless Exp>, <String Exp>)
PROPERTY (Id, <Typeless Exp>, <Typeless Exp>)
P (Id, <Numeric Exp>, <String Exp>)
P (Id, <Numeric Exp>, <Typeless Exp>)
P (Id, <Typeless Exp>, <String Exp>)
P (Id, <Typeless Exp>, <Typeless Exp>)
PROPERTY (<Numeric Exp>, <String Exp>)
PROPERTY (<Numeric Exp>, <Typeless Exp>)
PROPERTY (<Typeless Exp>, <String Exp>)
PROPERTY (<Typeless Exp>, <Typeless Exp>)
P (<Numeric Exp>, <String Exp>)
P (<Numeric Exp>, <Typeless Exp>)
P (<Typeless Exp>, <String Exp>)
P (<Typeless Exp>, <Typeless Exp>)
PROPERTY (<String Exp>)
PROPERTY (<Typeless Exp>)
P (<String Exp>)
P (<Typeless Exp>)
METHODPARM (<Numeric Exp>)
METHODPARM (<Typeless Exp>)
```

Sub-expression:

```
(<Typeless Exp>)
```

Parameter List

A *parameter list* is a comma-separated list of scalar expressions of any type.

<Parameter List> consists of one of these:

```
<Scalar Exp>
<Scalar Exp>, <Parameter List>
```

Scalar Tuples

A *scalar tuple* is a parenthesized set of one or more scalar expressions.

<Scalar Tuple> consists of this:

```
(<Scalar Expr Set>)
```

Scalar Expression Sets

A *scalar expression set* is a semi-colon-separated list of one or more scalar expressions, as in this.

<Scalar Expr Set> consists of one of these:

```
<Scalar Exp>; <Scalar Expr Set>
<Scalar Exp>
```

String Rules

A *string expression* is a concatenation of expressions of type String or of unknown type (see the second restriction under Restrictions on page 198).

<String Exp> consists of one of these:

```
<String Value>
<String Exp> + <String Value>
<String Exp> + <Typeless Value>
```

A *string value* can be any of these:

- A string literal
- A quoted variable reference
- A function call returning a string value
- A string sub-expression enclosed in parentheses

<String Value> consists of one of these:

```
StringLiteral
<FilterIdValue>
IF (<Boolean Exp>, <String Exp>, <String Exp>)
IF (<Boolean Exp>, <String Exp>, <Typeless Exp>)
IF (<Boolean Exp>, <Typeless Exp>, <String Exp>)
```

String Built-in Functions:

```
CLIENTSUBSTITUTE (<String Exp>, <String Expr List>)
CLIENTSUBSTITUTE (<Typeless Exp>, <String Expr List>)
SUBSTITUTE (<String Exp>, <String Expr List>)
SUBSTITUTE (<Typeless Exp>, <String Expr List>)
SUBSTRING (<String Exp>, <Numeric Exp>)
SUBSTRING (<String Exp>, <Typeless Exp>)
SUBSTRING (<String Exp>, <Numeric Exp>, <Numeric Exp>)
SUBSTRING (<String Exp>, <Numeric Exp>, <Typeless Exp>)
SUBSTRING (<String Exp>, <Typeless Exp>, <Numeric Exp>)
```

```
SUBSTRING (<String Exp>, <Typeless Exp>, <Typeless Exp>)
SUBSTRING (<Typeless Exp>, <Numeric Exp>)
SUBSTRING (<Typeless Exp>, <Typeless Exp>)
SUBSTRING (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>)
SUBSTRING (<Typeless Exp>, <Numeric Exp>, <Typeless Exp>)
SUBSTRING (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>)
SUBSTRING (<Typeless Exp>, <Typeless Exp>, <Typeless Exp>)
UPPER (<String Exp>)
UPPER (<Typeless Exp>)
LOWER (<String Exp>)
LOWER (<Typeless Exp>)
REPLACE (<String Exp>, <String Exp>, <String Exp>)
REPLACE (<String Exp>, <String Exp>, <Typeless Exp>)
REPLACE (<String Exp>, <Typeless Exp>, <String Exp>)
REPLACE (<String Exp>, <Typeless Exp>, <Typeless Exp>)
REPLACE (<Typeless Exp>, <String Exp>, <String Exp>)
REPLACE (<Typeless Exp>, <String Exp>, <Typeless Exp>)
REPLACE (<Typeless Exp>, <Typeless Exp>, <String Exp>)
REPLACE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp>)
```

Event Attributes:

```
EVENTNAME ()
ORIGINATOR ()
CONFIGNAME ()
EVENTSTATE ()
EVENTTITLE ()
ACTIONTYPENAME ()
VOTINGRESULT (<EventActionRef>)
RECIPIENTLIST (<EventActionRef>)
RESPONDERLIST (<EventActionRef>)
RESPONDERLIST (<EventActionRef>, <String Exp>)
RESPONDERLIST (<EventActionRef>, <Typeless Exp>)
```

Environment Attributes:

```
APPNAME ()
COMPANYNAME ()
USERNAME ()
USERDESC ()
WORKINGDIR ()
FILECONTENTS (<String Exp>)
```

FILECONTENTS (<Typeless Exp>)
NEWGUID()

Framework Event Parameters:

IDO()
INITIATOR()
FILTERSTRING()
LOADFLAGS()
PROPERTYNAMES()
POSTQUERYACTIONS()
CUSTOMINSERT()
CUSTOMUPDATE()
CUSTOMDELETE()
VARIABLENAME()
VARIABLEVALUE()
FILTERPROPERTY (Id, <Numeric Exp>, <String Exp>)
FILTERPROPERTY (Id, <Numeric Exp>, <Typeless Exp>)
FILTERPROPERTY (Id, <Typeless Exp>, <String Exp>)
FILTERPROPERTY (Id, <Typeless Exp>, <Typeless Exp>)
FP (Id, <Numeric Exp>, <String Exp>)
FP (Id, <Numeric Exp>, <Typeless Exp>)
FP (Id, <Typeless Exp>, <String Exp>)
FP (Id, <Typeless Exp>, <Typeless Exp>)
FILTERPROPERTY (<Numeric Exp>, <String Exp>)
FILTERPROPERTY (<Numeric Exp>, <Typeless Exp>)
FILTERPROPERTY (<Typeless Exp>, <String Exp>)
FILTERPROPERTY (<Typeless Exp>, <Typeless Exp>)
FP (<Numeric Exp>, <String Exp>)
FP (<Numeric Exp>, <Typeless Exp>)
FP (<Typeless Exp>, <String Exp>)
FP (<Typeless Exp>, <Typeless Exp>)
FILTERPROPERTY (<String Exp>)
FILTERPROPERTY (<Typeless Exp>)
FP (<String Exp>)
FP (<Typeless Exp>)
METHOD ()
FILTERMETHODPARM (<Numeric Exp>)
FILTERMETHODPARM (<Typeless Exp>)
FILTER (<String Exp>)
FILTER (<Typeless Exp>)
CAST (<Scalar Exp> AS STRING)

Sub-expression:

```
(<String Exp>)  
<Message Conversation>
```

String Expression Lists

A *string expression list* is an ordered, comma-separated list of string and/or scalar expressions.

<String Expr List> consists of one of these:

```
<Scalar Exp>  
<String Expr List>, <Scalar Exp>
```

Message Conversations

A *message conversation* is one or more `MESSAGE()` calls separated by
| newline/concatenation operators.

<Message Conversation> consists of one of these:

```
<Message>  
<Message Conversation> | <Message>
```

Messages

A *message* is the `MESSAGE` function surrounding a parenthesized, ordered, comma-separated list of string and/or typeless expressions.

<Message> consists of one of these:

```
MESSAGE (<String Exp>)  
MESSAGE (<Typeless Exp>)  
MESSAGE (<String Exp>, <String Expr List>)  
MESSAGE (<Typeless Exp>, <String Expr List>)
```

Numeric Rules

`<Numeric Exp>` consists of this:

`<Sum>`

Sums

A *sum* is a chain of sums and/or differences:

`<Sum>` consists of one of these:

`<Addend>`

`<Sum> + <Addend>`

`<Sum> - <Addend>`

Addends

An *addend* is a negatable expression, or a product.

These can be added to or subtracted from one another without parentheses surrounding each, due to the accepted arithmetic operator precedence.

`<Addend>` consists of one of these:

`<Negate Exp>`

`<Addend> * <Negate Exp>`

`<Addend> / <Negate Exp>`

Negatable Expressions

A *negatable expression* is a numeric value with or without a unary negative.

`<Negate Exp>` consists of one of these:

- `<Numeric Value>`

`<Numeric Value>`

- `<IdValue>`

Numeric Values

A *numeric value* can be any of these:

- A numeric constant
- A function call returning a numeric value
- A numeric sub-expression enclosed in parentheses

<Numeric Value> consists of one of these:

```
IntegerLiteral
RealLiteral
IF (<Boolean Exp>, <Numeric Exp>, <Numeric Exp>)
IF (<Boolean Exp>, <Numeric Exp>, <Typeless Exp>)
IF (<Boolean Exp>, <Typeless Exp>, <Numeric Exp>)
```

Numeric Built-in Functions:

```
DATEDIFF (<TimeInterval>, <Date Exp>, <Date Exp>)
DATEDIFF (<TimeInterval>, <Date Exp>, <Typeless Exp>)
DATEDIFF (<TimeInterval>, <Typeless Exp>, <Date Exp>)
DATEDIFF (<TimeInterval>, <Typeless Exp>, <Typeless Exp>)
DATEPART (<TimeInterval>, <Date Exp>)
DATEPART (<TimeInterval>, <Typeless Exp>)
LEN (<String Exp>)
LEN (<Typeless Exp>)
INSTR (<String Exp>, <String Exp>)
INSTR (<String Exp>, <Typeless Exp>)
INSTR (<Typeless Exp>, <String Exp>)
INSTR (<Typeless Exp>, <Typeless Exp>)
CEILING (<Numeric Exp>)
CEILING (<Typeless Exp>)
FLOOR (<Numeric Exp>)
FLOOR (<Typeless Exp>)
POWER (<Numeric Exp>, <Numeric Exp>)
POWER (<Numeric Exp>, <Typeless Exp>)
POWER (<Typeless Exp>, <Numeric Exp>)
POWER (<Typeless Exp>, <Typeless Exp>)
ROUND (<Numeric Exp>, <Numeric Exp>)
ROUND (<Numeric Exp>, <Typeless Exp>)
ROUND (<Typeless Exp>, <Numeric Exp>)
ROUND (<Typeless Exp>, <Typeless Exp>)
TRUNC (<Numeric Exp>, <Numeric Exp>)
TRUNC (<Numeric Exp>, <Typeless Exp>)
TRUNC (<Typeless Exp>, <Numeric Exp>)
```

```
TRUNC (<Typeless Exp>, <Typeless Exp>)
```

Event State Attributes:

```
EVENTREVISION ( )  
HANDLERSEQ ( )  
ACTIONSEQ ( )
```

Event Attributes:

```
RECIPIENTS (<EventActionRef>)  
RESPONDERS (<EventActionRef>)  
RESPONDERS (<EventActionRef>, <String Exp>)  
RESPONDERS (<EventActionRef>, <Typeless Exp>)
```

Framework Event Parameters:

```
RECORDCAP()  
ROWS (Id) ROWS ( )  
METHODPARMS ( )  
CAST (<Scalar Exp> AS NUMBER)
```

Sub-expression:

```
(<Numeric Exp>)
```

Date Rules

A *date expression* is a date value (because there are no natural arithmetic date operators).

<Date Exp> consists of this:

```
<Date Value>
```

A date value is a function call returning a date value, or a date sub-expression enclosed in parentheses.

<Date Value> consists of one of these:

```
DATE (<Numeric Exp>, <Numeric Exp>, <Numeric Exp>)  
DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>)  
DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>)
```

```

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Numeric Exp>, <Numeric Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Numeric Exp>, <Numeric Exp>, <Typeless Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Numeric Exp>, <Typeless Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Numeric Exp>, <Typeless Exp>, <Typeless Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Typeless Exp>, <Numeric Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Typeless Exp>, <Numeric Exp>, <Typeless Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Typeless Exp>, <Typeless Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,
<Typeless Exp>, <Typeless Exp>, <Typeless Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,
<Numeric Exp>, <Numeric Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,
<Numeric Exp>, <Numeric Exp>, <Typeless Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,
<Numeric Exp>, <Typeless Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,
<Numeric Exp>, <Typeless Exp>, <Typeless Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,
<Typeless Exp>, <Numeric Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,
<Typeless Exp>, <Typeless Exp>, <Numeric Exp>)
DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,
<Typeless Exp>, <Typeless Exp>, <Typeless Exp>)
CURDATETIME ()
DATEADD (<TimeInterval>, <Numeric Exp>, <Date Exp>)
DATEADD (<TimeInterval>, <Numeric Exp>, <Typeless Exp>)
DATEADD (<TimeInterval>, <Typeless Exp>, <Date Exp>)
DATEADD (<TimeInterval>, <Typeless Exp>, <Typeless Exp>)
BEGINDATE ()
CAST (<Scalar Exp> AS DATE)

```

Sub-expression:

```
(<Date Exp>)
```

Date Enumerations

`<TimeInterval>` consists of one of these:

- `<DatePartYear>`
- `<DatePartQuarter>`
- `<DatePartMonth>`
- `<DatePartDayOfYear>`
- `<DatePartDay>`
- `<DatePartWeek>`
- `<DatePartWeekDay>`
- `<DatePartHour>`
- `<DatePartMinute>`
- `<DatePartSecond>`
- `<DatePartMillisecond>`

`<DatePartYear>` consists of one of these:

- `year`
- `YY`
- `YYYY`

`<DatePartQuarter>` consists of one of these:

- `quarter`
- `qq`
- `q`

`<DatePartMonth>` consists of one of these:

- `month`
- `mm`
- `m`

`<DatePartDayOfYear>` consists of one of these:

- `dayofyear`
- `dy`
- `y`

`<DatePartDay>` consists of one of these:

- `day`
- `dd`

d

<DatePartWeek> consists of one of these:

week
wk
ww

<DatePartWeekDay> consists of one of these:

weekday
dw

<DatePartHour> consists of one of these:

hour
hh

<DatePartMinute> consists of one of these:

minute
mi
n

<DatePartSecond> consists of one of these:

second
ss
s

<DatePartMillisecond> consists of one of these:

millisecond
ms

Restricted Arguments

<EventActionRef> consists of this:

IntegerLiteral

Keyword Paren Lists

A *keyword paren list* is one or more "**KEYWORD**(*value*)" statements separated by whitespace.

<KeywordParenList> consists of one of these:

```
<KEYWORD (value) >  
<KEYWORD (value) > ... <KEYWORD (value) >
```

For a complete list and description of **KEYWORD**(*value*) statements as they relate to action types, see Event Action Parameters on page 147

Enumerations

These lists of keywords can be used with **ACTION**(), **VOTINGRULE**(), and **TASKSTATUS**() expressions.

ACTION()

<SaveAction> consists of one of these:

```
<SaveActionInsert>  
<SaveActionUpdate>  
<SaveActionDelete>
```

<SaveActionInsert> consists of this:

```
INSERT
```

<SaveActionUpdate> consists of this:

```
UPDATE
```

<SaveActionDelete> consists of this:

```
DELETE
```

VOTINGRULE()

<VotingRule> consists of one of these:

```
<VotingRuleMajority>  
<VotingRulePlurality>  
<VotingRuleConditionalPlurality>  
<VotingRuleMinimumCount>  
<VotingRuleMinimumPercentage>  
<VotingRuleEarliestResponse>
```

<VotingRulePreferredChoice>
<VotingRuleMinimumCountPreferredChoice>
<VotingRuleMinimumPercentagePreferredChoice>

<VotingRuleMajority> consists of this:

Majority

<VotingRulePlurality> consists of this:

Plurality

<VotingRuleConditionalPlurality> consists of this:

ConditionalPlurality

<VotingRuleMinimumCount> consists of this:

MinimumCount

<VotingRuleMinimumPercentage> consists of this:

MinimumPercentage

<VotingRuleEarliestResponse> consists of this:

EarliestResponse

<VotingRulePreferredChoice> consists of this:

PreferredChoice

<VotingRuleMinimumCountPreferredChoice> consists of this:

MinimumCountPreferredChoice

<VotingRuleMinimumPercentagePreferredChoice> consists of this:

MinimumPercentagePreferredChoice

TASKSTATUS()

<InitialTaskStatus> consists of one of these:

<TaskStatusReady>

<TaskStatusWaiting>

<TaskStatusReady> consists of this:

READY

<TaskStatusWaiting> consists of this:

WAITING

The sample stored procedures in this appendix provide sample code and function usage.

Passing Parameters to a Synchronous Event

```
SET @MyEventParmId = NEWID()
EXEC InsertEventInputParameterSp @MyEventParmId, 'Parameter1', @Variable1
EXEC InsertEventInputParameterSp @MyEventParmId, 'Parameter2', @Variable2
EXEC InsertEventInputParameterSp @MyEventParmId, 'OutParameter3', NULL, 1
EXEC FireEventSp 'EventName', @MyEventParmId
PRINT dbo.EventOutputParameterValue(@MyEventParmId, 'OutParameter3')
```

Calling a Synchronous Event within a Transaction and Handling Failure

First, determine the current site, after which you must name a configuration, by convention:

```
DECLARE @Site SiteType
SELECT @Site = site FROM parms
```

Then determine the current SessionId:

```
DECLARE @SessionId RowPointerType
SET @SessionId = dbo.SessionIdSp()
```

Finally, add the procedure code:

```
BEGIN TRANSACTION
UPDATE coitem
SET due_date = dbo.CalcDueDate(@Parm1, @Parm2)
WHERE coitem.co_num = @CoNum
```

```
AND coitem.co_line = @CoLine
AND coitem.co_release = @CoRelease

SET @MyEventParmId = NEWID()
EXEC InsertEventInputParameterSp @MyEventParmId, 'CoNum', @CoNum
EXEC InsertEventInputParameterSp @MyEventParmId, 'CoLine', @CoLine
EXEC InsertEventInputParameterSp @MyEventParmId, 'CoRelease', @CoRelease

DECLARE
@anyHandlersFailed [tinyint],
@result [nvarchar](4000),
@Infobar [nvarchar](4000)

EXEC @Severity = FireEventSp
@eventName = SetCoitemDueDate',
@configName = 'SyteLine',
@sessionID = @SessionID,
@eventTrxId = null,
@eventParmId = @MyEventParmID OUTPUT,
@transactional = 0,
@anyHandlersFailed = @anyHandlersFailed output,
@result = @result output,
@Infobar = @infobar output

IF @Severity > 0
BEGIN
EXEC RaiseError @Infobar, @Severity
ROLLBACK TRANSACTION
END
...
COMMIT TRANSACTION
```

Appendix E: Synchronization of Metadata

E

This appendix discusses the various concepts and strategies for synchronizing metadata belonging to different owners, and provides a number of detailed examples.

Overview and rationale

Application event system metadata can come from three primary sources:

- The Infor system framework
- Applications produced by Infor and/or Infor's business partners or other authorized vendor developers
- End-customer development

The ownership for the metadata from each of these sources is controlled by the Access As identifier. For more information on this identifier, see "About the Access As identifier" on page 35.

Any of these sources can upgrade and reissue their metadata at times independent of the others. The upgrade process needs to:

- Update any changed handlers that they own.
- Insert any new handlers they might have created since the last version.
- Maintain other owners' handlers and the relationships between them.

Therefore, a synchronization mechanism is needed, to make sure that changes by one metadata owner do not adversely affect the functioning of another owner's metadata.

This mechanism is provided in two components:

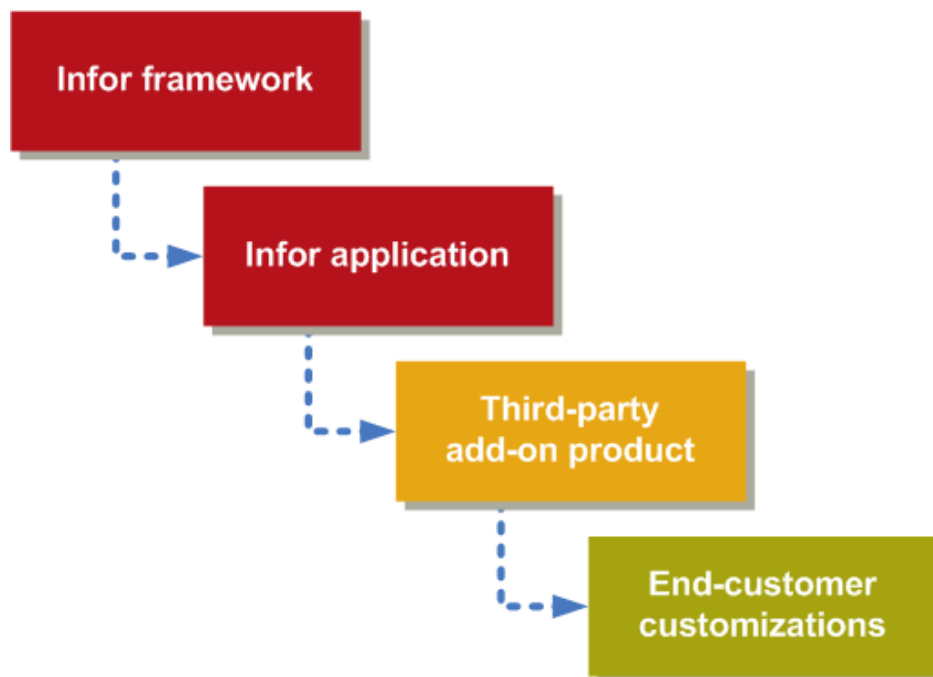
- The Access As identifier (see "About the Access As identifier" on page 35)
- The App Metadata Sync and App Metadata Transport utilities, which both provide the capability to synchronize event metadata belonging to different owners.

Using these utilities, metadata developers can export their events and event handlers and make them available for import by other metadata owners. They also use this utility to import their own or others' metadata into their system.

For more information about these utilities, see the online help for each utility. This help is available by selecting **Help > Customizing Forms** and then performing a search for "App Metadata Sync" or "App Metadata Transport".

The Inherent Hierarchy of Metadata

In the Application Event System, there is no programmatic ranking of Access As identifiers. There is, however, an *inherent* hierarchy, based on the normal production flow and use of the system software. As illustrated in this diagram, Infor's framework developers are the first to develop event system objects. Other Infor application developers then can add their own event system objects, as can authorized business partners and other vendor developers. Finally, end customers can make custom modifications and develop their own custom event system objects.



In reality, the system is designed so that *no* metadata owner can modify or delete the metadata of another owner. So, in that sense, they are all equal. However, this real-time production flow creates an inherent hierarchy that allows us to *think* of them as "higher-level" (that is, Infor) and "lower-level" (that is, end-customer) owners.

With that in mind, we can state these general rules:

- Lower-level owners can insert their handlers between two higher-level handlers for the same event.
- In many cases, lower-level owners can override higher-level handlers.
There is, however, an option for higher-level owners to disallow overrides for individual handlers.

-
- Higher-level handlers that are overridden remain in the metadata store but are marked as Inactive.

This means, among other things that, if the lower-level handler is later deleted, the higher-level handler is still available and can become active again.

Chronology rules allow downstream owners to integrate and control the sequence of their events and handlers with respect to those upstream.

For more information, see "Detailed Examples" on page 228.

Maintaining Handler IDs Through Metadata Updates

Each event handler is identified with a unique (and hidden) ID, which is referenced by the **Keep With** field on the **Event Handlers** form. This ID, rather than the actual **Handler Sequence** number, becomes the "fixed" reference point for that handler. This means that an event handler owner does not need to worry about maintaining the **Handler Sequence** numbers across releases: The system takes care of it automatically, by preserving the hidden ID number and reassigning **Handler Sequence** numbers as required.

After each insertion, update, or deletion of a handler, and during a merge performed by the App Metadata Sync utility or the App Metadata Transport utility, the system calculates new integers, if necessary, for display in the **Handler Sequence** field. The underlying ID, however, remains unchanged. When a handler is deactivated and another added in the same position, the new handler gets a new ID.

Protecting Running Events from Metadata Changes

Once an event handler begins executing, it is essential to prevent changes to its attributes and actions. Otherwise, unpredictable behavior could result, especially if actions are resequenced.

One way to do prevent these changes would be to make a copy of all active, non-obsolete handlers and their actions each time an event is triggered and control execution from this copy. However, that method would result in the persistence of a great number of identical copies, assuming that handlers are modified much less frequently than they are executed.

Since the system stores state data separately from metadata, it is sufficient to make a copy only when the metadata changes, and furthermore only when the corresponding event is triggered and a copy of the last metadata modifications has not yet been made.

In other words, handler metadata for an event can be created and edited as many times as necessary. The first time the event is triggered, a copy of the last saved metadata is made. This copy is called an *event revision*. The execution of the event's handlers is then controlled by the event revision and not by the original metadata (which happen to be identical at this point).

The event can be triggered as many times as necessary, all the time controlled by this event revision, as long as no intervening modifications have been made to the original metadata.

After one or more metadata edits have been saved, though, the next time the event is triggered, the system copies a new event revision from the last-saved metadata.

For more information about event revisions, see "Events and event handler revisions" on page 32.

Detailed Examples

This section provides three detailed examples of how sequencing and synchronization work in the Application Event System.

Using Specific Chronology

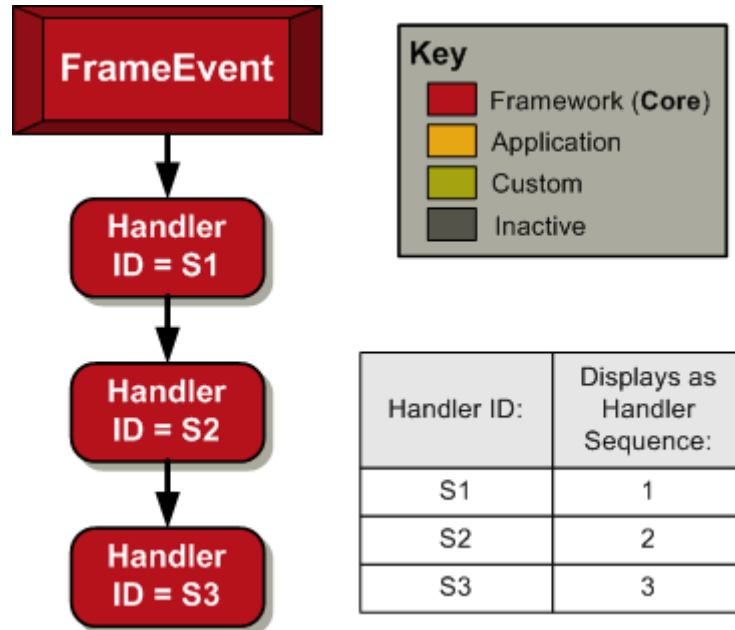
The primary way for a lower-level handler creator to resequence existing handlers (from higher-level owners) is to use what is known as *specific chronology*. That is, the handler's creator can attach the new handler to an existing handler and specify the order in which the two handlers are to execute with respect to each other.

The mechanism used to do this are the **Keep With** and **Chronology** fields on the **Event Handlers** form. These fields allow you to specify whether your handler should run before, after, or in place of the handler it is associated with, as in this example.

For more information about the **Keep With** and **Chronology** fields, see the online help for those fields.

Infor Creates a Framework Event with Three Handlers

The Infor framework team creates an event, `FrameEvent`, and creates three event handlers that execute in order when the event is generated.



This event and these handlers are all included with the application software when it ships.

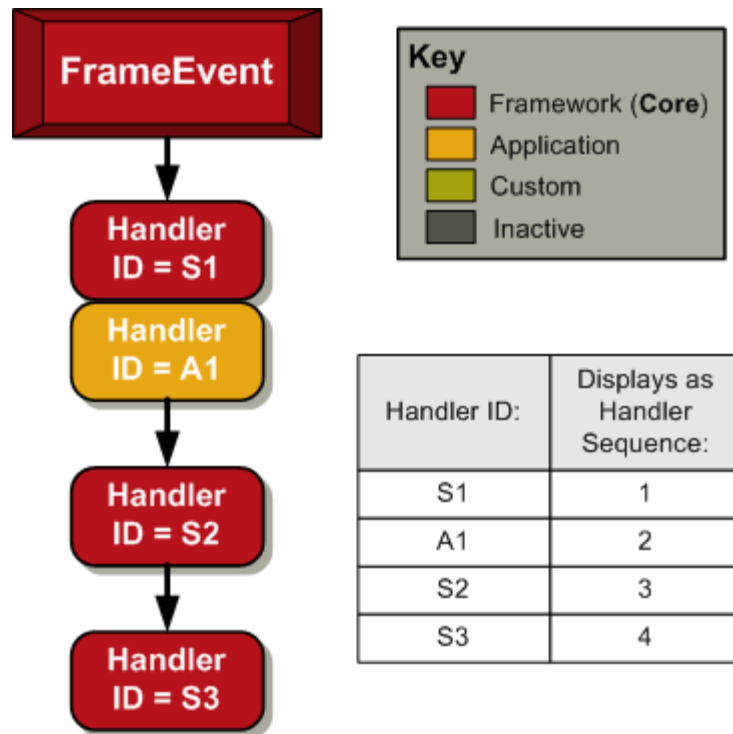
Note that, if a "lower-level" developer wants to add or modify handlers, these rules apply:

- Lower-level handler creators cannot change the sequence of higher-level handlers.
- Any handlers lower-level creators want to use that will affect the sequence in specific ways must be associated with a particular, already-existing handler, using the **Keep With** and **Chronology** fields.

A Business Partner Creates an Additional Handler

An Infor business partner decides to add a handler that will execute just after the first framework handler. The business partner creates the event handler and uses the **Keep With** and **Chronology** fields to keep their handler with ID **1** (S1) and to execute **After** that handler.

The sequence is now as follows:



Note that, when the new event handler is saved, the system automatically assigns new **Handler Sequence** numbers, so that you can tell which handler executes in what order.

The business partner uses the App Metadata Sync or App Metadata Transport utility to export the new metadata, along with the existing metadata, to a file that can be imported by the end-customer.

The business partner then sells the add-on product to the end-customer. The add-on product includes this file, along with any product code the business partner has developed.

The End-customer Creates Three More Handlers

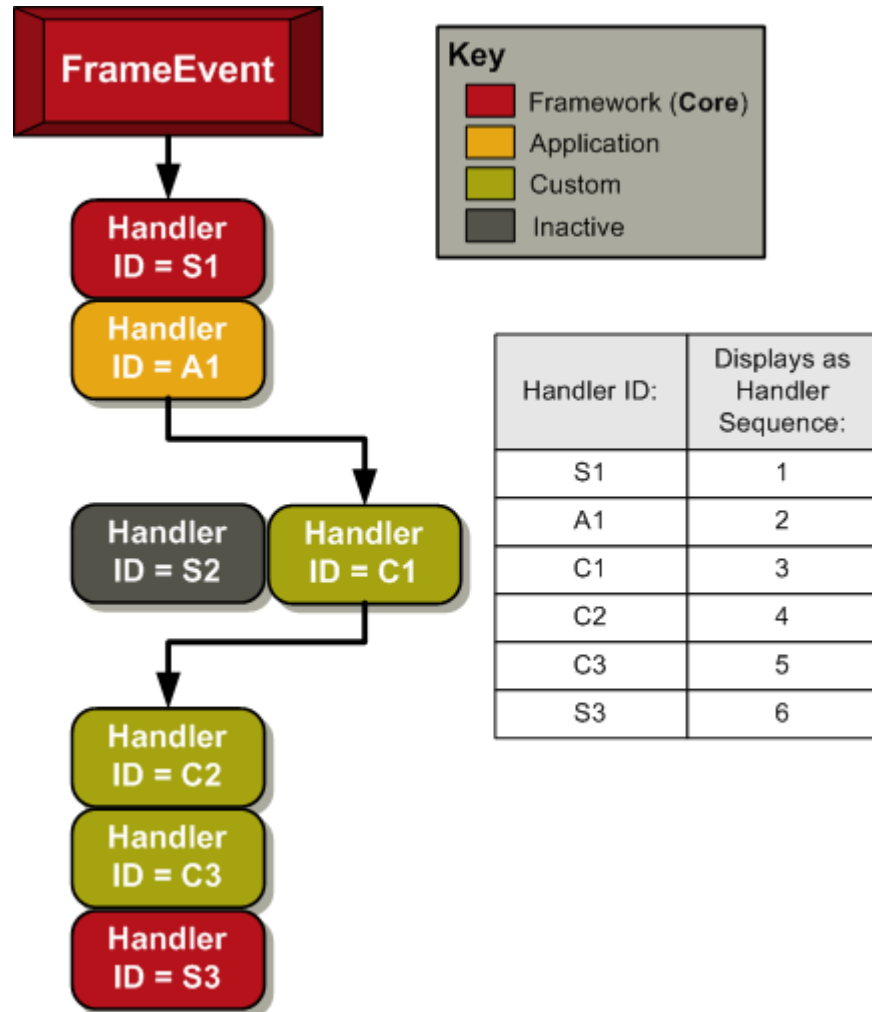
When the end-customer receives the add-on software from the third-party business partner, in addition to whatever other software installation procedures the customer must perform, the customer must also use the App Metadata Sync or App Metadata Transport utility to import the event metadata from the business partner.

The end-customer now decides to use a different handler in place of the second framework handler. The customer also wants two custom handlers to run just before the third framework handler. To accomplish this, the customer:

- Creates a handler and uses the **Keep With** field to assign this handler to ID 2 (S2). In the **Chronology** field, the customer selects **Instead**.
- Creates a second handler and uses the **Keep With** field to assign this handler to ID 3 (S3). In the **Chronology** field, the customer selects **Before**.

- Creates a third handler and uses the **Keep With** field to also assign this handler to ID 3 (S3). In the **Chronology** field, again the customer selects **Before**.

After saving their new handlers with the existing handlers, the sequence is now as follows:



Again, note that, when the customer saves the new handlers, the system automatically assigns and displays the correct **Handler Sequence** number on the **Event Handlers** form. The underlying Handler ID, however, remains unchanged.

Note that event handler S2 is now inactive and does not execute at all. It is still in the system, but the system ignores it in favor of the new customer handler.

Note also that the custom handlers C2 and C3 execute in that order unless you use the **Up / Down** buttons on the **Event Handler Sequence** form to alter the default order.

Using Non-specific Chronology

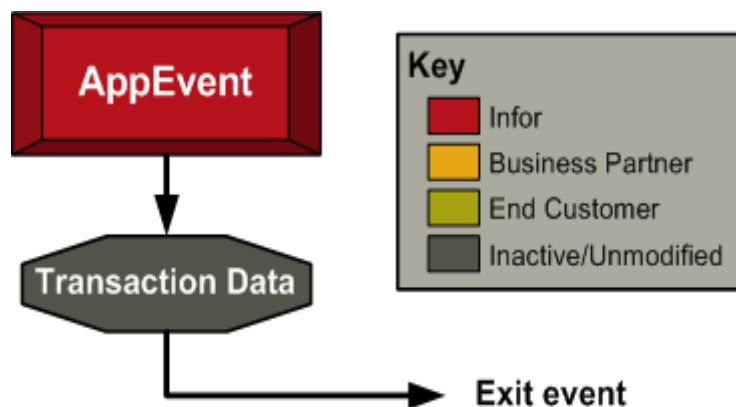
A second way for downstream developers to affect the sequence of handlers is with the use of non-specific chronology. Non-specific chronology allows developers to indicate that a particular handler should always execute either first or last.

This is done by selecting either the **First** or **Last** option from the **Chronology** drop-down field, as in this example.

Infor Creates an Event with No Handler

The Infor development team adds code to generate a new application event (AppEvent) before a certain transaction posting is performed.

The transaction data is passed into the event and received from the event, so that any downstream subscribers can test and modify the values before the posting is performed if they want. An event failure is trapped and aborts the posting, displaying the error message returned by the event. Infor adds no event handlers for the event at this time.

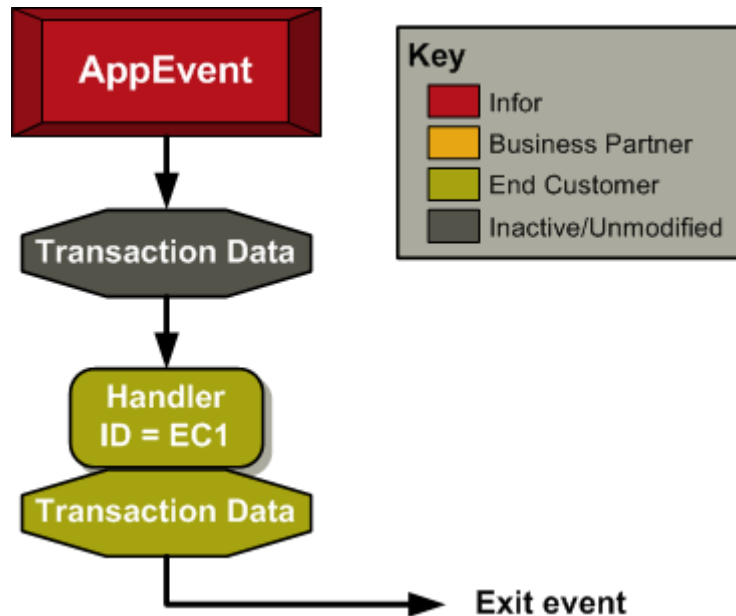


The product is then released. The event is now published and available to downstream developers who install this product.

An End-customer Creates a Handler

An end-customer installs the software and decides to create a handler (EC1) to validate that the posting data is within a certain range. If it detects an out-of-range condition, the handler fails the event and aborts the posting.

At this point, the sequence is this:



Because no upstream handlers exist yet (and the end-customer is the furthest downstream developer), a specific chronology cannot be specified at this time.

A Business Partner Creates an Add-on Product

In the meantime, an Infor business partner creates an add-on product that stores custom parameters which can be used to adjust the transaction data before posting. They add their own handler (BP1) to input, adjust, and output the data.

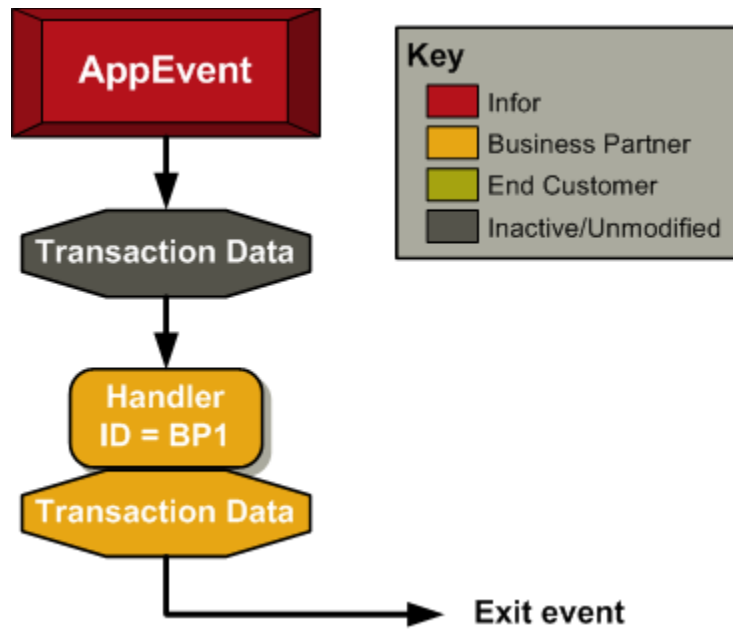
In the **Chronology** field, they select **First** to indicate that handler BP1 should run before:

- Any existing, downstream handlers *with no specified chronology* that might already exist for this published event

Note: A specific chronology (**Before**, **After**, **Instead**) takes precedence over a non-specific one (**First** and **Last**).

- Any future upstream or peer handlers

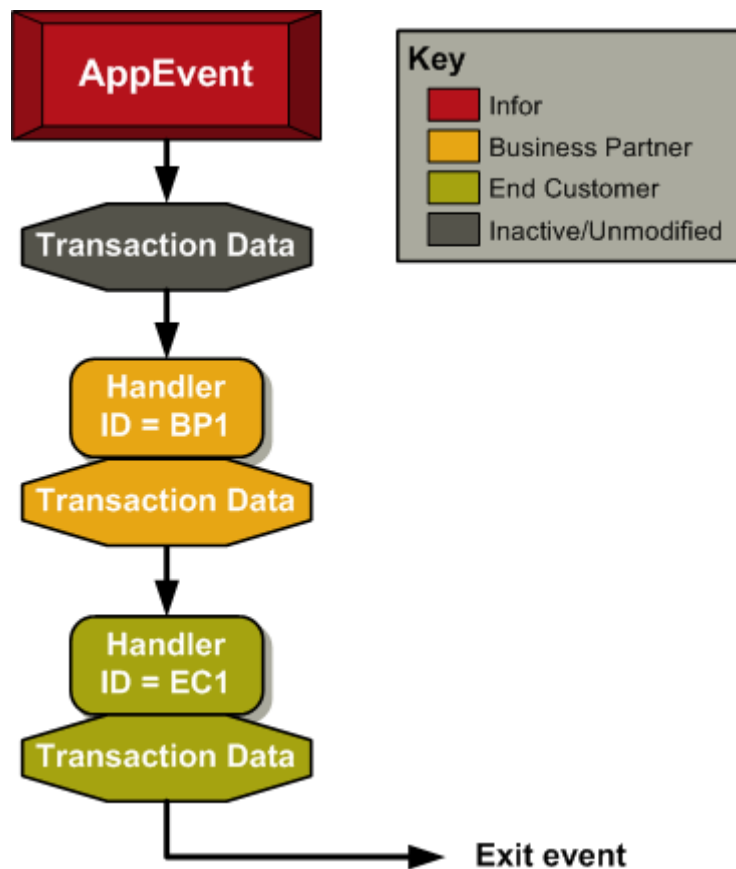
With this add-on product, at the business partner level, the sequence is now this:



The End-customer Installs the Business Partner's Add-on Product

Now the end-customer purchases and installs the add-on from the Infor business partner and uses the App Metadata Sync or App Metadata Transport utility to synchronize the metadata from the business partner with their own.

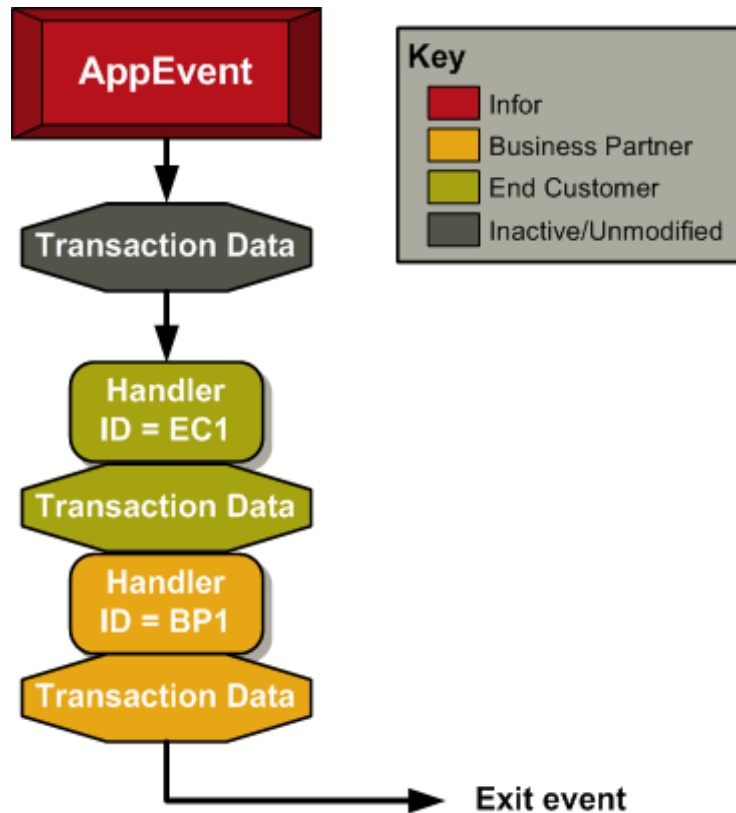
Because the business partner specified that their handler should always run first, the sequence is now this:



The End-customer Rearranges the Sequence

After installing the business partner's add-on product, the end-customer decides that their handler really should run *before* the business partner's. So the end-customer uses the **Keep With** and **Chronology** fields to associate handler EC1 with the business partner's handler (BP1) and to execute **Before** it.

After saving the event handler EC1, the sequence is this:

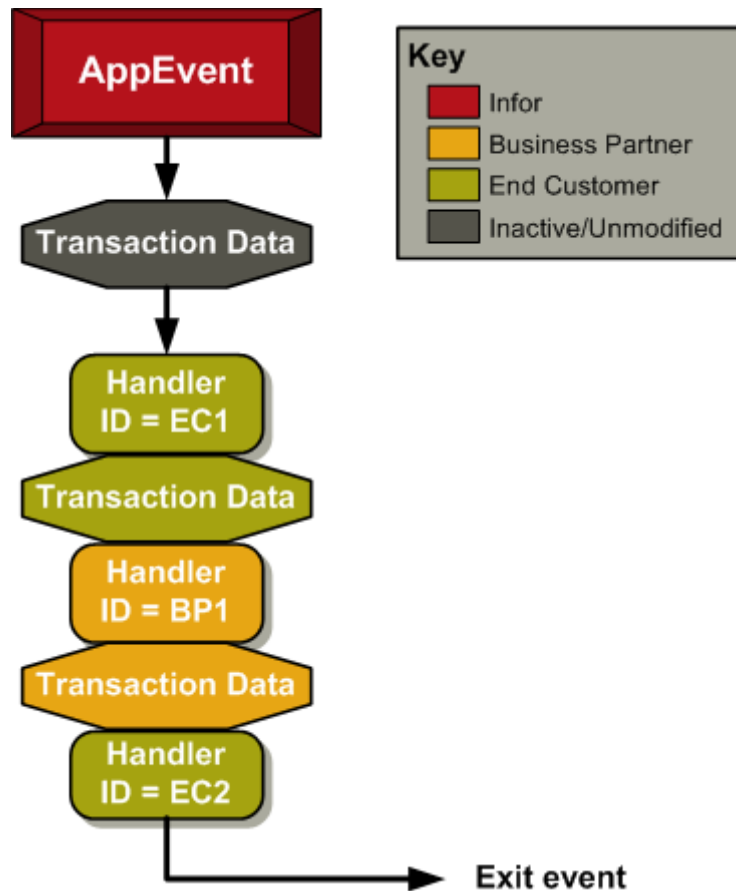


This is possible because a specific chronology designation (see "Using Specific Chronology" on page 228) takes precedence over a non-specific chronology.

The End-customer Decides to Add Another Handler

After this, the end-customer adds a new handler (EC2) to save the final, adjusted transaction data into a data warehouse. They use the **Keep With** and **Chronology** fields to associate the new handler with the business partner's handler, specifying that the new one execute **After** the business partner's (BP1).

After saving the new event handler, the sequence is this:

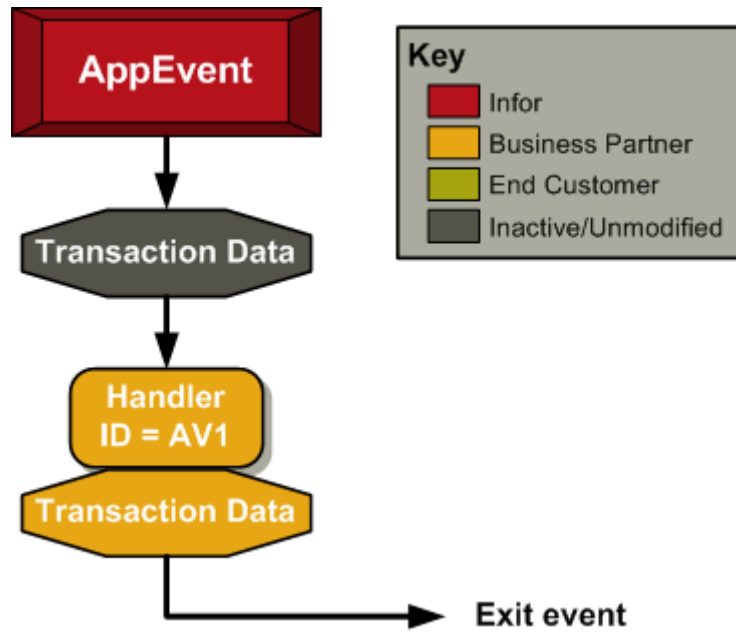


Another Business Partner Adds a Handler

Independently of all this, a second Infor business partner installs the standard Infor software and decides to add another event handler for their add-on product. This event handler (AV1) is to post additional information to the journal based on the transaction data that was specified.

Because this handler should run *after* all transaction data has been processed, the business partner specifies in the **Chronology** field that event handler AV1 should execute **Last**.

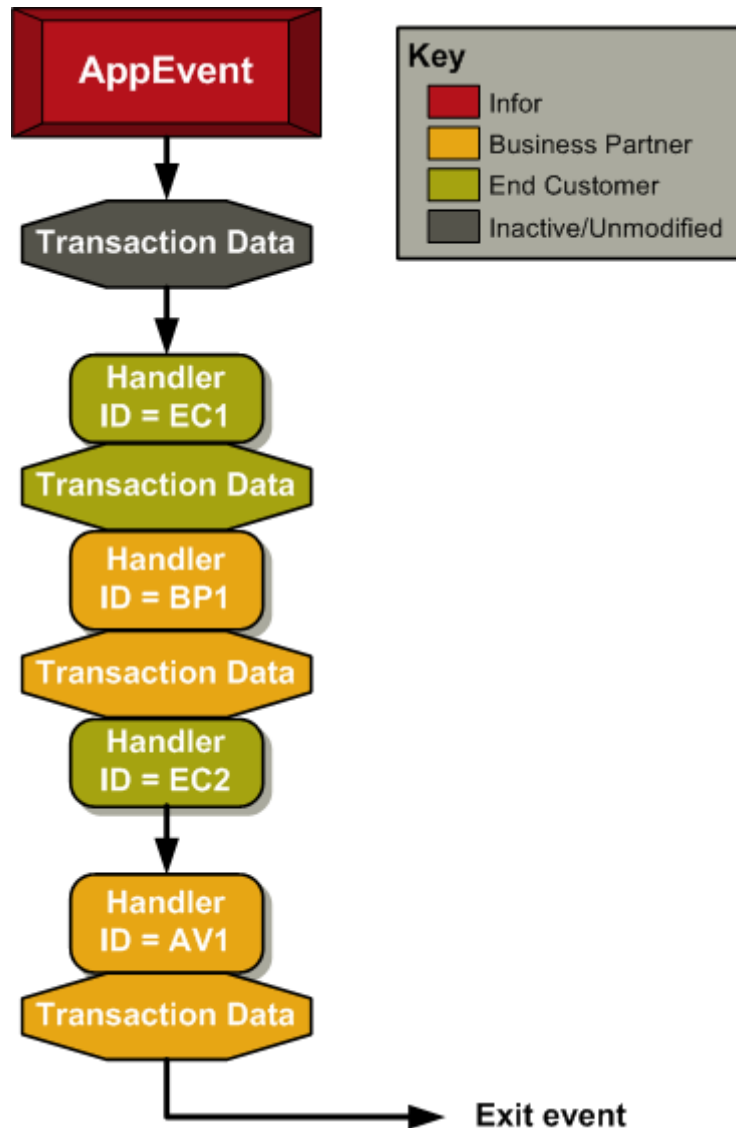
At this point, the flow looks similar to the first business partner's add-on flow:



However, the effect of designating this handler to run **Last** is quite different.

The End-customer Adds the Second Business Partner's Handler

Finally, the end-customer purchases and installs the second business partner's add-on product. After using the App Metadata Sync or App Metadata Transport utility to synchronize the new metadata with the existing metadata, the flow now proceeds as follows:



Note that, even without the **Last** designation, in this case, handler AV1 would have executed last simply because the handler BP1 is designated to execute **First**, and the other two handlers are both "attached" to BP1.

Performing Upgrades

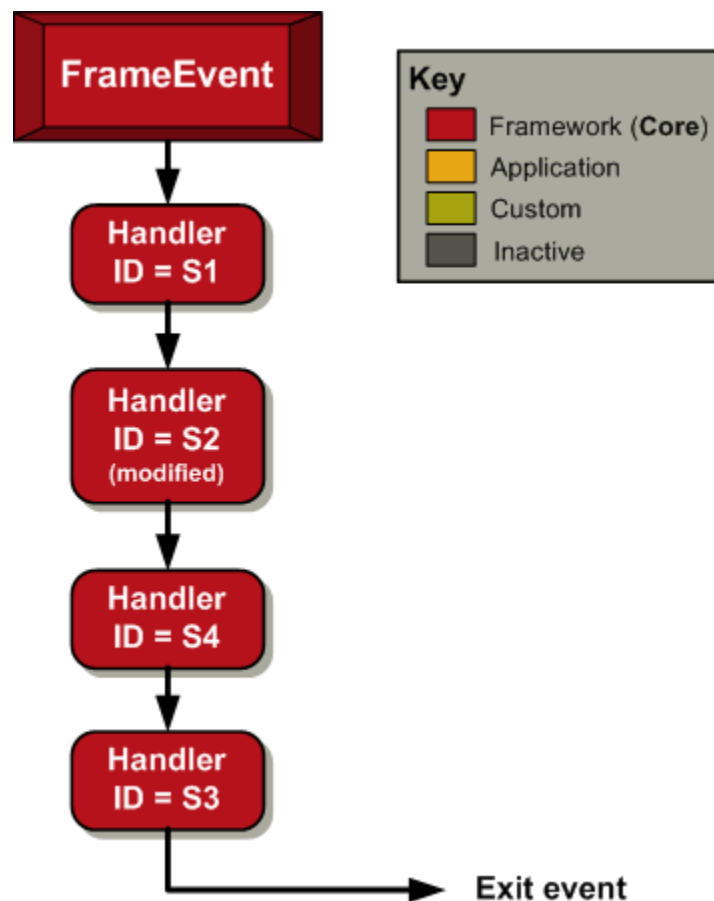
It is likely that at some point in the future, Infor will make changes to the existing events and handlers, and these changes will be included in an upgrade. This poses a few problems and questions. The upgrade process must:

- Update any changed Infor handlers and insert new ones while maintaining custom handlers belonging to business partners and end-customers.
- Keep custom handlers in the correct sequence with Infor's handlers (especially for overrides) beyond the point where Infor inserts a new handler.

To illustrate how this works, we return to the previous examples (in "Using Specific Chronology" on page 228 and "Using Non-specific Chronology" on page 232).

Continuing the Specific Chronology Example...

After the end-customer has installed the Infor software and the business partner add-on product and added its own custom handlers, Infor introduces a new version in which one of the original base handlers (S2) has been modified and a new one, S4, has been added between S2 and S3. The new base flow in the upgrade is illustrated in this diagram:

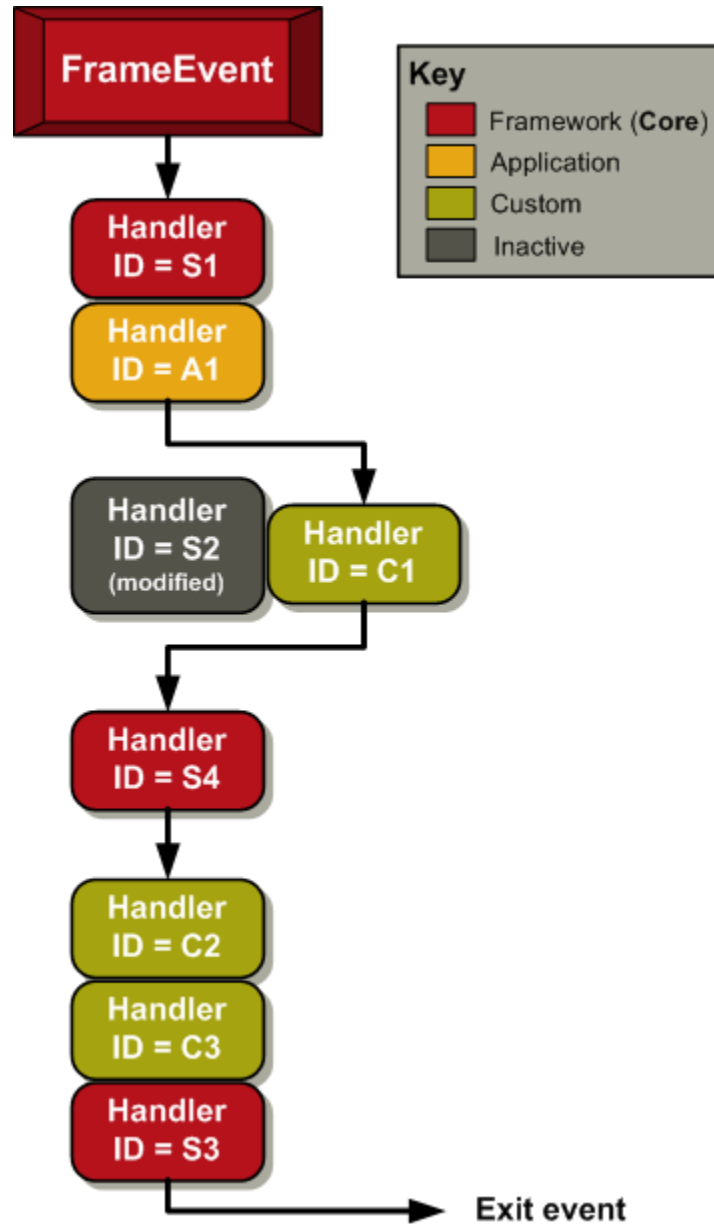


Note that, when this version is released, the numbers actually displayed in the **Handler Sequence** field of the **Event Handlers** form are as follows:

Handler ID	Handler Sequence number
S1	1
S2	2
S3	4
S4	3

Note that what was previously the third event handler in the flow (S3), now appears as **4** in the **Handler Sequence** field. This is because, when the Infor developer saves the new handler, the system automatically assigns the correct **Handler Sequence** number, without changing the underlying Handler ID number.

The end-customer now installs the new Infor upgrade and uses the App Metadata Sync or App Metadata Transport utility to import and synchronize the updated metadata for this event. After the integration is complete, the new end-customer sequence is this:



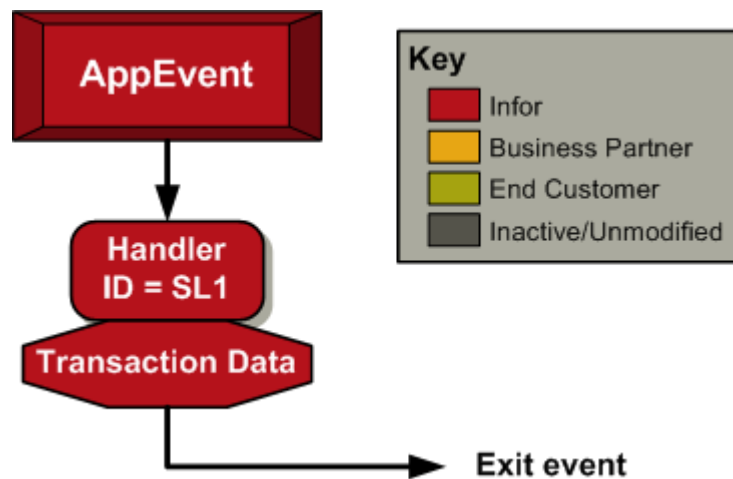
Note:

- Infor handler S2 has been updated. In this case, the update does not run, because of the end-customer handler C1, which replaces it. But the changes are there (stored as handler metadata) and available if the end-customer should ever decide to delete handler C1.
- The synchronization process inserts the new Infor handler between end-customer handlers C1 and C2. This is because end-customer handler C1 is "attached" to Infor handler S2; end-

customer handlers C2 and C3 are "attached" to Infor handler S3; and the new Infor handler has been inserted between S2 and S3.

Continuing the Non-specific Chronology Example...

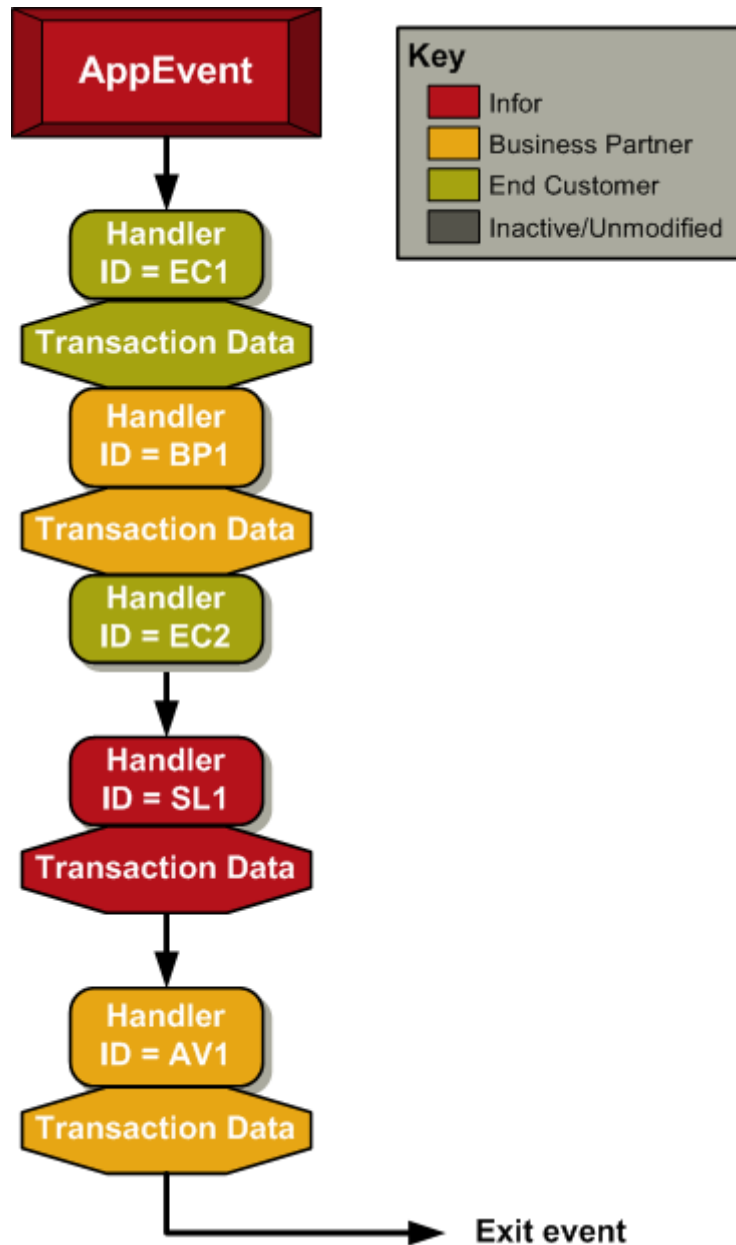
(Starting at the end of "Using Non-specific Chronology":) After the end-customer has installed the Infor software and both business partner add-on products, Infor introduces a new version in which a new handler to validate data formats has been added and designated to run **First**. The new base flow in the upgrade is illustrated in this diagram:



Note that this new upgrade will now cause the event to be generated and check data formats whenever the appropriate transaction is posted. This is different, because before the Infor base version of this event did nothing.

Now the end-customer installs the Infor upgrade and uses the App Metadata Sync or App Metadata Transport utility to import and synchronize the new handler metadata with the existing metadata for this event. Because the business partner handler BP1 is marked to execute **First**, that handler and all handlers "attached" to it (EC1 and EC2) are placed at the beginning. Because the business partner handler AV1 is marked to execute **Last**, that handler is placed at the end. Because the Infor handler SL1 is not marked to execute in any particular order, it is inserted between EC2 (which is attached to BP1) and AV1.

After the integration is complete, the new handler sequence is as follows:



Overriding Others' Handlers

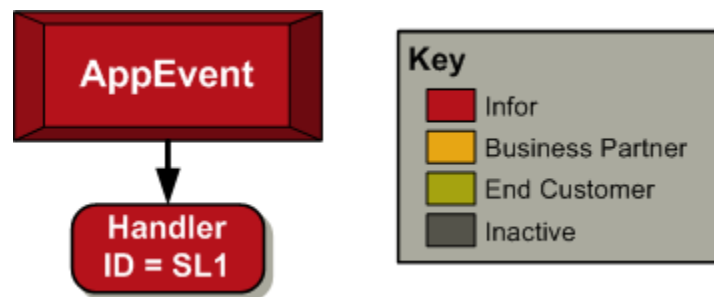
As demonstrated briefly in "Using Specific Chronology" on page 228, downstream event handler creators can create handlers that, in effect, replace an upstream handler. This is done primarily with

the use of the **Instead** option in the **Chronology** field on the **Event Handlers** form. This option allows multiple overriding handlers to coexist. In other words, this option is considered a *non-exclusive* override.

There is also a second option for situations where a handler's creator might want to override *all* other handlers, regardless of who owns them or when they were added to the sequence. This is done using the **Exclusively** (instead) option in the second **Keep With** field. This option is considered an *exclusive* override.

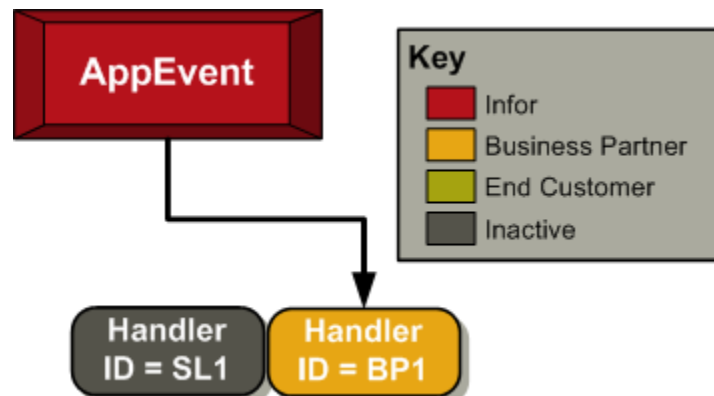
Non-exclusive Overrides

In this example, the Infor application development team creates an application event and adds an event handler, SL1, for it.



The application is released with this event metadata included as part of the standard product.

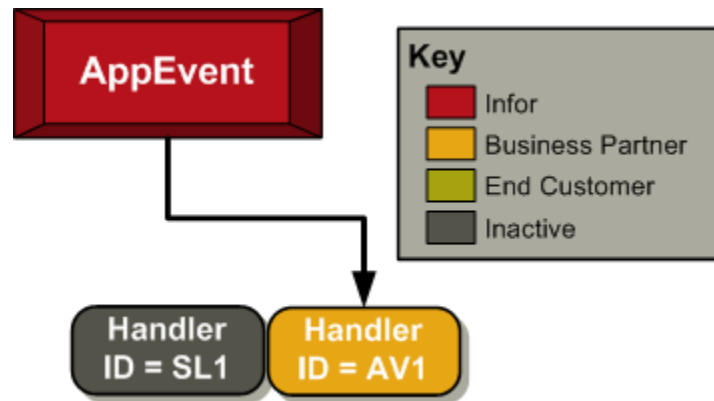
An Infor business partner installs the software with this event metadata. They decide, however, that they want to replace the standard Infor event handler with one of their own. So they create handler BP1 and select **Instead** in the **Chronology** field on the **Event Handlers** form. After saving their handler, the sequence is as follows:



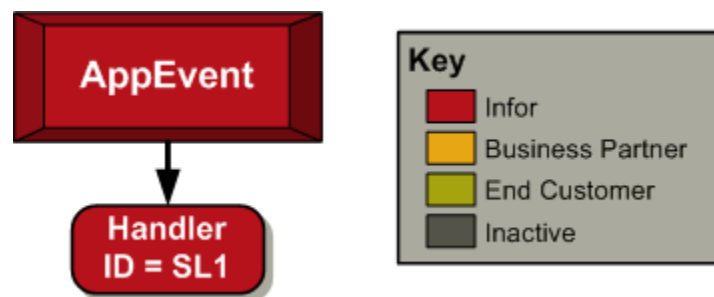
Notice that this effectively bypasses and deactivates the original Infor event handler, SL1.

The business partner then releases the add-on product with this new event handler in place.

In the meantime, a second Infor business partner does the same thing. For the sake of the example, the second business partner's handler is labeled AV1.



Now suppose that a customer purchases and installs the standard Infor product. At this point, the sequence is the same as what both business partners received originally:

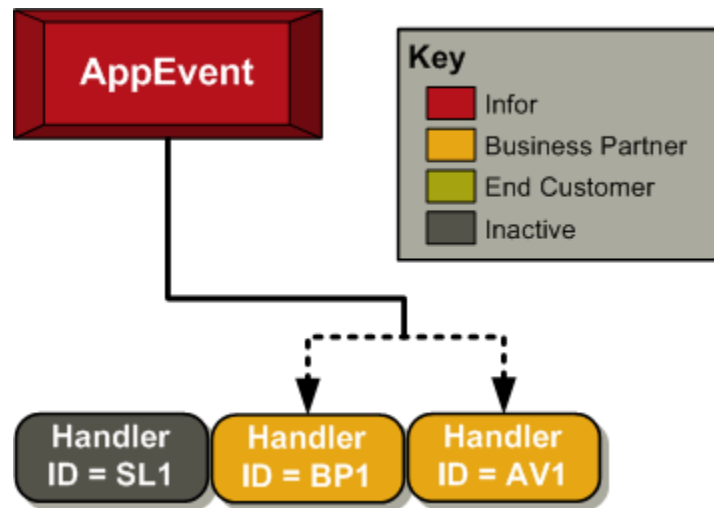


When the event is generated, the Infor event handler executes.

The customer then purchases and installs the add-on product from the first business partner. At this point, the sequence is the same as the first business partner's. The App Metadata Sync or App Metadata Transport utility recognizes the override of BP1 and automatically deactivates the Infor handler SL1.

But then, the customer also purchases the second business partner's add-on product and installs it. The App Metadata Sync or App Metadata Transport utility recognizes the new override of AV1 and

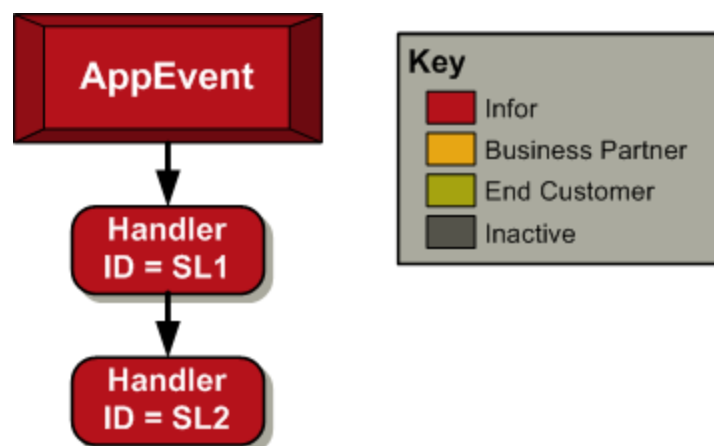
allows both overrides to coexist. When the event is generated, both handlers BP1 and AV1 execute. The sequence now looks something like this:



Note that, in this case, there is no reliable way to know the precise order in which event handlers BP1 and AV1 will execute, since they coexist at the same level in the sequence and execute independently of one another.

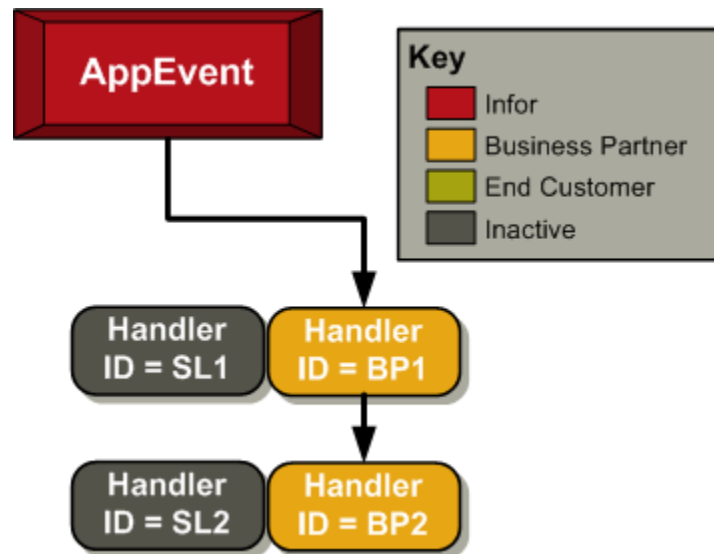
Exclusive Overrides

To continue the example, suppose that the Infor application development team now adds another event handler, SL2, for the same event. When Infor releases the upgrade, the sequence for this event now looks like this:



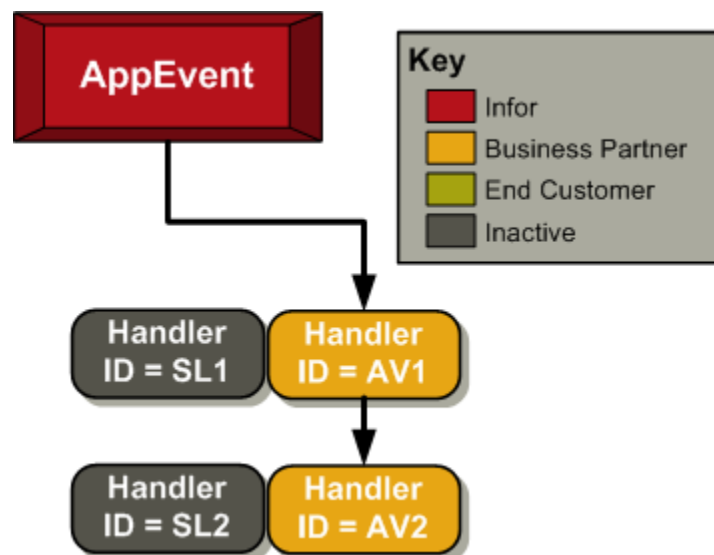
The first business partner installs the upgrade using the App Metadata Sync or App Metadata Transport utility. Once again, they decide they want to override the new Infor handler with one of their

own. So they create a new handler, BP2, and assign it to run **Instead** of SL2. When they release the new version of their add-on product, the sequence for this event now looks like this:



Note that, so far, this is typical *non-exclusive* override behavior.

The second business partner does the same thing as the first, with one exception: They specify that the new handler, AV2, is to execute **Exclusively Instead**, regardless of any other handlers that might exist at that point in the sequence. When they release their new version, the sequence for this event looks very similar to that of the first business partner:



However, the outcome is quite different.

Suppose the customer now installs the Infor software upgrade and tries to install the upgrades for both business partners as well. All goes well until the customer tries to install the new version of the second business partner's add-on product. When the customer tries to synchronize the event

metadata from the second business partner, the App Metadata Sync or App Metadata Transport utility generates an error, because the handler AV2 is set to execute *exclusively*, which puts it in conflict with the first business partner's handler BP2. The two cannot coexist.

To resolve the conflict, the customer would have to uninstall one of the business partner handlers and contact them for guidance.

In practice, this should be a very rare occurrence.

Disabling the Ability to Override

Owners of "higher-level" handlers can designate that their handlers cannot be overridden. Downstream developers cannot use a **Chronology** setting of **Instead** or **Exclusive Instead** with such a handler.

To designate an event handler as not subject to being overridden, on the **Event Handlers** form, use the **Overrideable** check box. When this check box is cleared, no lower-level handlers can override that handler. In this case, the **Instead** and **Exclusive Instead** options are disabled for any other handlers (downstream) that use the **Keep With** field to associate those handlers with this handler.

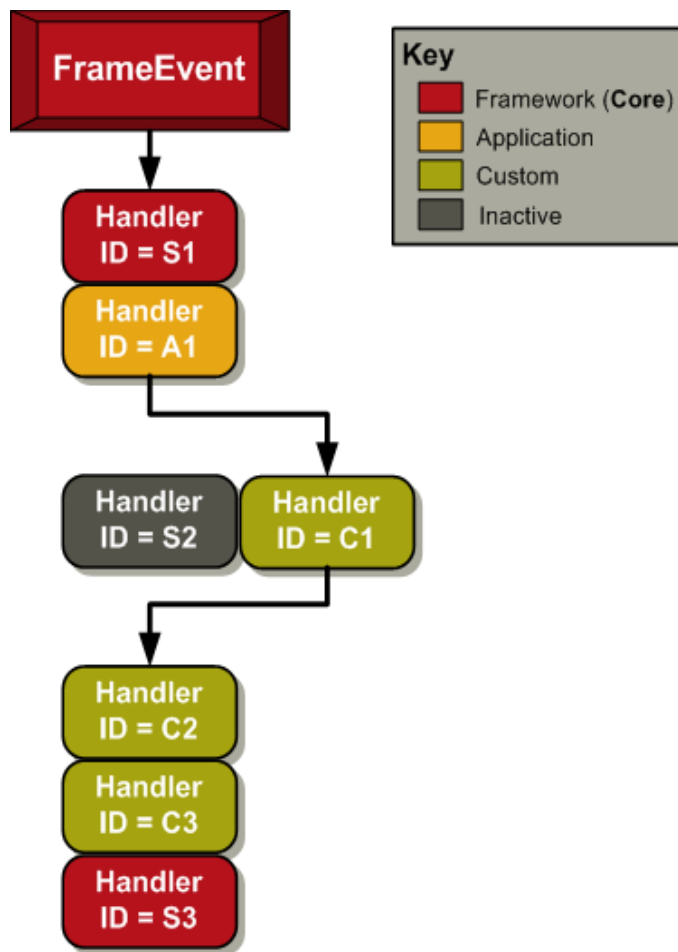
Dealing with Obsolete Handlers

If you are an Infor or business partner developer, you must not delete handler metadata that has been exported and made available to other (downstream) metadata developers. This is because those developers might have their own metadata associated with your metadata, by means of the **Keep With** feature, before your next release.

If your exported metadata removes a handler that is referenced by their handler's **Keep With** field, a broken link occurs during import. This forces the App Metadata Sync or App Metadata Transport utility to relegate the downstream developer's handler to "free floating" status, meaning its sequence is less controllable. This is especially true if you—or an upstream developer whose handlers are referenced by your handlers' **Keep With** field—later resequence your handlers.

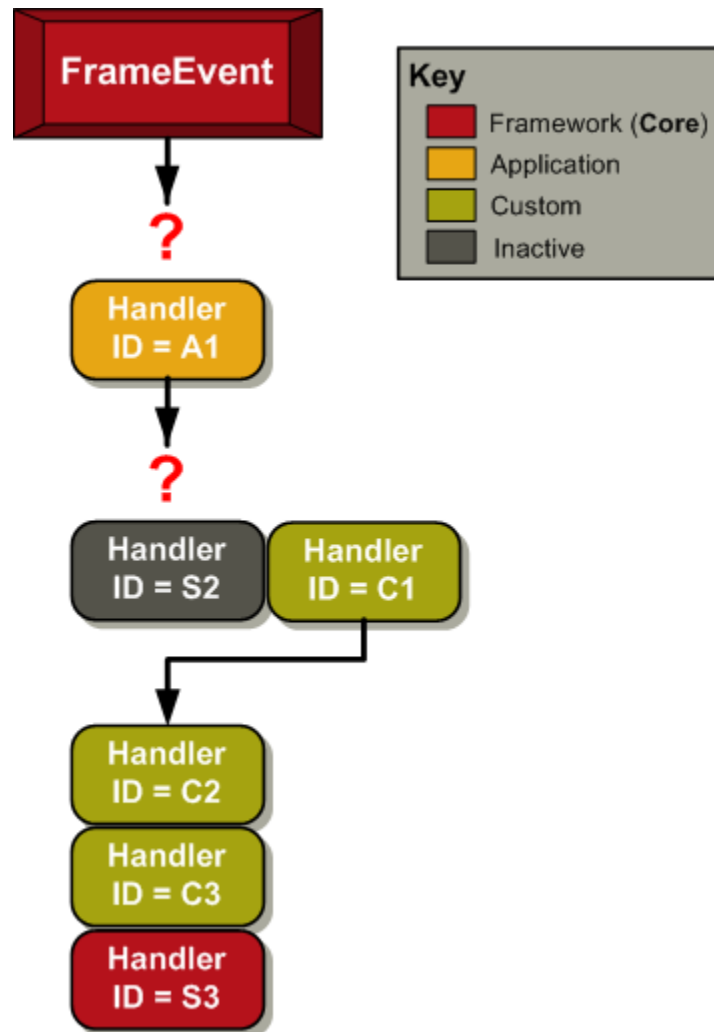
Therefore, when a handler is no longer needed, on the **Event Handlers** form you should mark it as *obsolete*, by selecting the check box labeled **Obsolete**. This effectively and permanently deactivates the handler but leaves it in the sequence, so that associations with it do not get destroyed.

For example, consider this sequence of handlers:

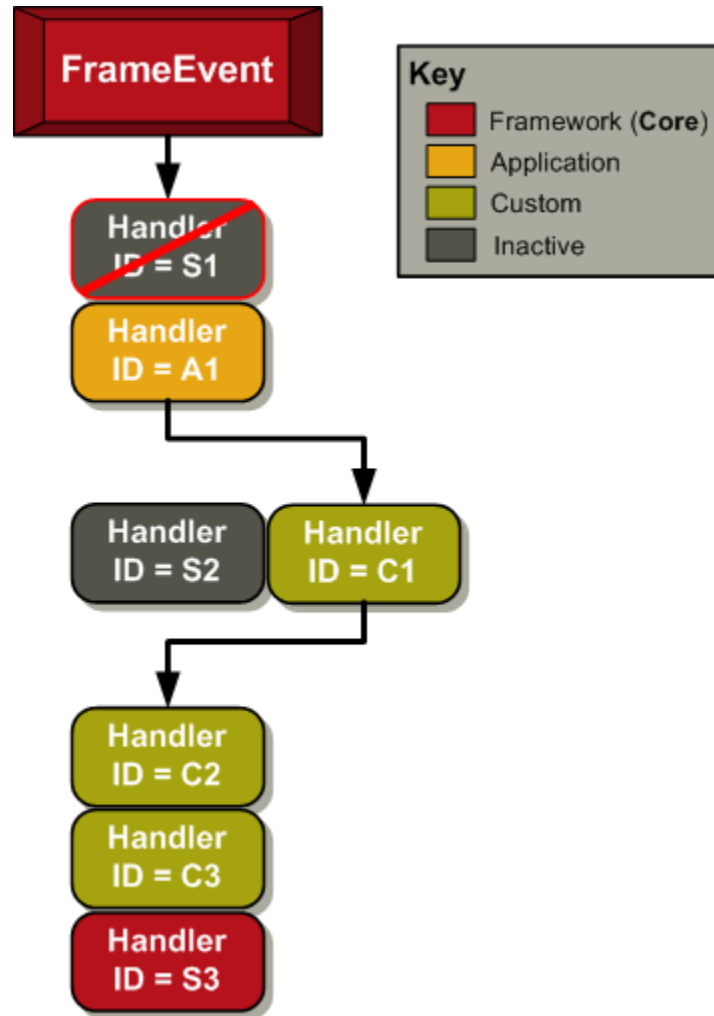


Suppose, after initially releasing this version of the metadata, Infor decides not to use the handler S1 anymore. Because a business partner has associated handler A1 with this handler, if Infor simply deletes handler S1, a broken link occurs the next time the business partner or a customer who uses the business partner's add-on product tries to install the upgraded metadata. Handler A1 is relegated

to a "free floating" status (as in this diagram), and unexpected consequences can result after subsequent resequencing or synchronizing operations.



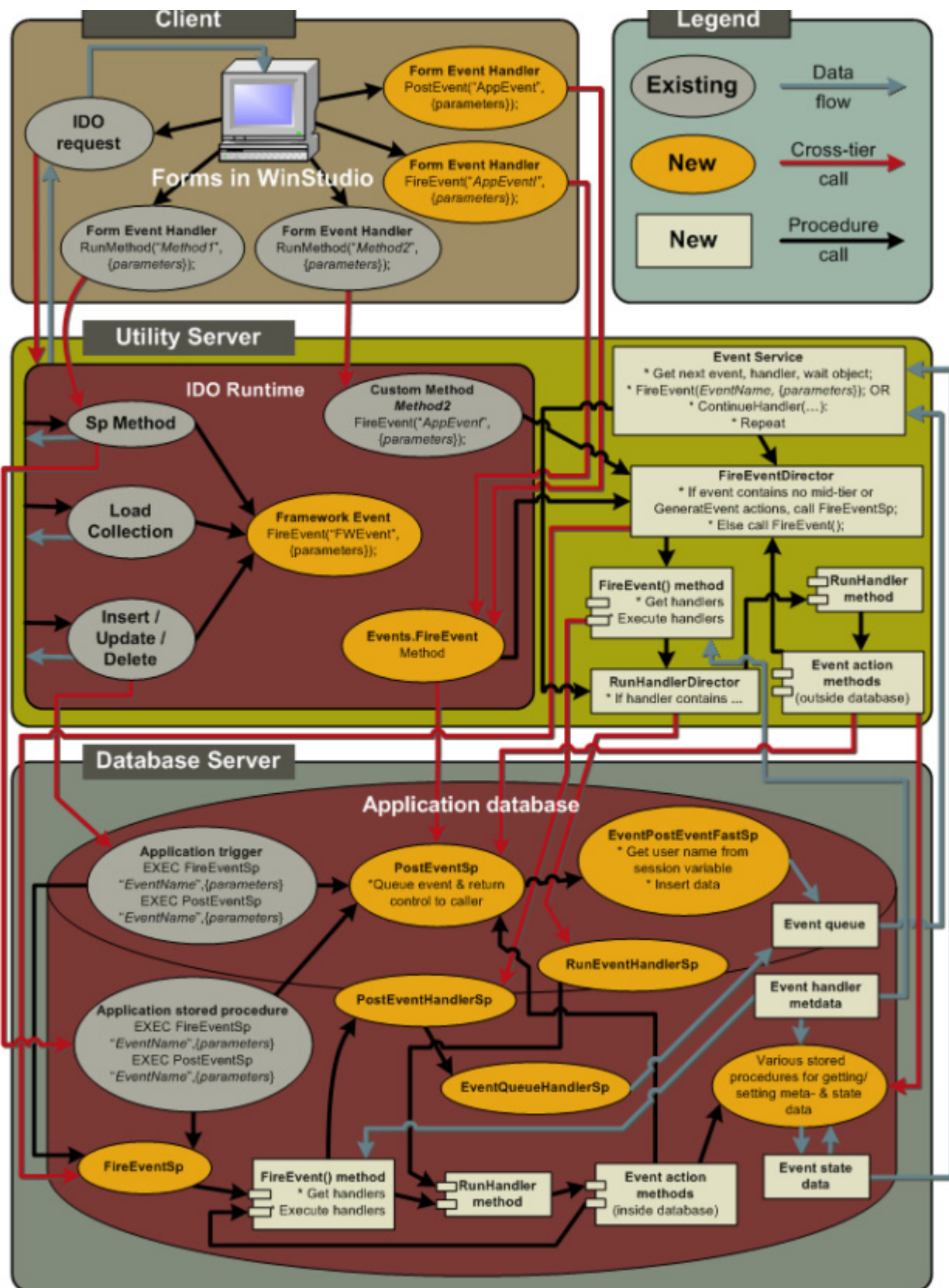
So, rather than delete handler S1, Infor would mark it as **Obsolete** before the next release. Then, when the update is installed, the reference to handler S1 is not broken, though the system ignores it when handling this event, as in this diagram:



Appendix F: Event Flow Options

A red square containing a white capital letter 'F'.

The flow diagram in this appendix illustrates many of the possible ways an event can be generated and the types of flows that can result. This diagram highlights the differences between functionality with and without the Application Event System.



Glossary

Access As

An identifier used to identify who created and owns a metadata object. This identifier is also used to control which metadata objects you can modify. You can modify only those metadata objects that are associated with the current **Access As** field value, as displayed on the **Access As** form.

For more information, see "About the Access As identifier" on page 3-35.

adjourning event

An event action that must wait for an external stimulus before it can continue. When the system encounters an adjourning event action, the event handler state is set to retest or to time out after a specified time. The event system then processes it at the next opportunity and resumes.

asynchronous event handler

An event handler designed to execute independently of other event handlers, and whose triggering process does not block while waiting for it to finish. These event handlers are sent to an event queue, from which they execute in FIFO order.

See *also* asynchronous event handler.

database tier

That part of the system framework that stores the actual data for:

- Rendering forms (the Forms database)
- Storing all customer business data (the Application database)

Programs can also run in this tier, where they can access data very quickly without having to traverse network paths.

For more information, see the "System Architecture" chapter in your *System Administration Guide*.

end-customer

The company that has bought and is using the Infor Mongoose software. They might also have bought one or more add-on products by Infor business partners to customize and enhance the performance of their system.

Explorer

A window in the application (similar to Windows Explorer) that displays folders containing form names, providing a means to find, organize, and open forms. Explorer is the default window when the application opens. To reopen Explorer if it is closed, select **View > Explorer** on the menu bar.

event

A uniquely named incident that can be triggered by:

- An action performed by somebody working in the system
- A particular condition that occurs while the system is running
- A certain value being exceeded in a database record

-
- Another event or one its handlers
 - Other, similar occurrences

A particular event can possibly be triggered by multiple situations or conditions, and you can determine how the system responds to each situation.

To be useful, an event must have one or more event handlers defined to respond to the event.

For more information, see "About events" on page 3-36.

event action

Metadata that specifies a unit of work to perform during the execution of an event handler. A single event handler can have multiple event actions. Depending on its action type, an event action can do such things as:

- Evaluate and compare expressions, using the results to select which event action of its event handler to perform next.
- Affect the event's visual state.
- Complete the event.
- Set event variables.
- Call methods.
- Perform other predefined tasks.

For more information, see "About event actions" on page 3-41.

event action state

A set of data that shows the current state of a running or finished event action. This data includes information about when the event action started running, its current status, the number of times it has run, and other information.

event action type

A designator that limits or directs what an event action can do. This designator essentially determines the unit of work that each event action performs.

event global constant

A named static value that event expressions can reference during processing of an event handler.

event handler

Metadata that defines a portion of work to be performed upon the firing of a particular event. Each event handler is uniquely identified in the system with the combination of an event name and a handler sequence number. Each event handler is comprised of one or more event actions and, optionally, an initial state.

For more information, see "About event handlers" on page 3-40.

event handler state

A set of data that shows the current state of a running or finished event handler. This data includes information about when the event handler started running, its current status, timeout settings, and other information.

event initial variable

See initial variable.

event input parameter

A named static value that is passed to an event upon its triggering. This value can be set to be available as an output after the event finishes executing.

Any number of uniquely named event input parameters can be collected before firing an event. Upon firing the event, each is converted to an event parameter.

event message

A message, initiated by the event system, sent from one system user to another, that in many respects simulates an email message. Event messages are created by event actions of a Notify or Prompt type, or by **Inbox** activities such as **Forward**, **Reply**, and **Reply All**, or from the Send Message form.

Event messages can appear in the **Sent Items** folder of the sender, and the **Inbox** of each recipient.

For more information, see "Event Messages" on page 5-61.

event output parameter

A named static value passed from an event upon its finish. Any number of uniquely named event output parameters can be associated with an event. Each output is created from an event parameter marked for output.

event parameter

A named storage area retrievable by an event action, that is associated with an event that has been generated and is processing. The system creates event parameters from event input parameters when the event is generated.

Event parameters can be set to be available to whatever process generated the event, after the event finishes. In this case, they can also be set by event actions and can result in the creation of event output parameters.

event queue

A FIFO list of events and event handlers to be processed asynchronously. Each entry has an associated user name, configuration name, and request date.

event state

1. A collection of data related to the current status of a running or finished event. This status is viewed using the **Event Status** form.
2. An optional text string that displays on the **Event Status** form when the event reaches certain milestones or finishes executing successfully. This text string is defined by the event handler's creator and associated with the **Achieve Milestone** and **Finish** action types as a parameter.

event trigger

A condition that causes an event to be generated with optional parameters.

For more information, see "About event triggers" on page 3-38.

event variable

A named storage area, the value of which can be set and retrieved by an event action associated with a running event handler. When associated with a synchronous event handler, an event variable can be designated as **Persistent**, in which case the value of the variable can be passed on to the next event handler.

FIFO

First In, First Out.

framework

The multi-tiered software structure that makes up the entire system. In this application, the framework consists of three basic tiers:

- The client tier, known as WinStudio.
- The middle tier
- The database tier

For more information, see the "System Architecture" chapter in your *System Administration Guide*.

framework event

An event which has been designed to be generated only in reference to certain framework occurrences.

initial variable

A named static value for an event variable associated with an event handler. This value provides the initial value of the event variable when the event handler begins executing.

Each event variable contains an authorization level that provides a default access within the scope of an event action that:

- Does not have a default access value defined on the **Variable Access** tab of the **Event Actions** form. In this case, default access value is determined on the **Event Variable Groups** form.
- Has a default access of **Default** on the **Variable Access** tab of the **Event Actions** form.

metadata

In the context of the Application Event System, refers to the practice of using uncompiled code and information about data formats that are interpreted during run time, rather than compiled code (also called "procedural code").

middle tier

The layer of software in the database system which provides the connections between the client tier (WinStudio) and the database tier. The middle tier has two primary functions:

- To provide access from the client tier to the database through IDOs (Intelligent Data Objects). The client tier never communicates directly with the database. In this respect, the middle tier can be thought of rather like a telephone: It allows two parties to talk to one another, but not face-to-face.
- To receive form-rendering requests from the client tier, retrieve the appropriate form-rendering data from the forms database, and return that data to the client so that the form displays correctly.

For more information about the middle tier, see the "System Architecture" chapter in your *System Administration Guide*.

synchronous event handler

An event handler designed to execute sequentially with other handlers, and whose triggering process blocks while waiting for it to finish (unless part of an event fired asynchronously). If any one event handler in the sequence fails, then the whole sequence fails.

See *also* asynchronous event handler.

transactional events or event handlers

One of a group of events or handlers that are included in a single transaction. Typically, all the event handlers and their actions must execute successfully before the transaction is considered finished and any results or data are committed.

If any of the events or handlers in a transaction do not finish successfully, the entire transaction fails and all data and variable values roll back (revert) to their original values.

WinStudio

The client software in the database system, sometimes referred to as the "presentation layer," through which users interact with information in the database. WinStudio displays forms and provides the interface through which users can find, add, edit, sort, and delete data.

For more information, see the "System Architecture" chapter in your *System Administration Guide*.

Index

A

Access As
 form 3-35
 identifier 3-35
Achieve Milestone action type B-140
ACTION function B-148
action types B-139
actions
 adjourning 2-25
 for events 3-41
 parameters B-147
 types B-139
ACTIONSEQ function B-172
ACTIONTYPENAMEfunction B-172
additional Infor ERP SL documentation -9
adjourning actions 2-25
adjournment 2-25
ANYHANDLERSFAILED function B-172
application event system
 advantages of 1-13
 design forms 3-47
 elements 3-35
 examples of use 1-14
 flow diagram F-253
 how it works 2-17
 overview 1-11
application-specific events 3-36
APPNAME function B-172
asynchronous
 event handlers 2-20
 events 2-20

 events in transactions 2-30

Attach action type B-140

Audit action type B-140

authorization group for event system forms 3-47,
4-57

B

BEGINDATE function B-172

BodOnReceive B-127, B-132

BODY function B-150

Boolean rules for expressions C-201

Branch action type B-140

C

Call Database Method action type B-141

Call IDO Method action type B-141

Call Web Service action type B-141

Caption, in Inbox form 5-63

CAST function B-172

CATEGORY function B-150

CC function B-150

CEILING function B-172

CHOICES function B-151

CLIENTSUBSTITUTE function B-173

COLLECTION function B-163

COMMIT function B-151

COMPANYNAME function B-173

complex expansions of functions B-193

CONDITION function B-152

ConditionalPlurality voting rule 5-67

CONFIGNAME function B-173

constants, event global 3-46

contacting Infor -10

controlling	event global constants 3-46
ownership of metadata 3-35	event handlers 3-40
sequence	event triggers 3-38
event actions 2-18, 3-52	events 3-37
event handlers 2-18, 3-51	DESC function B-152
core (framework) events B-126	Description parameter B-152
Core events 3-36	design forms 3-47
creating	Access As 3-35
event actions 3-41	authorization group for 3-47
event global constants 3-46	Event Actions 3-41
event handlers 3-40	Event Global Constants 3-46
event triggers 3-38	Event Handlers 3-40
events 3-37	Events 3-36
CURDATETIME function B-174	location 3-47
custom events and handlers 3-35– 3-55	designing custom events and handlers 3-35– 3-55
designing custom events 3-50	DEST function B-152, B-166
setting up 3-49	Destination parameter B-152
CUSTOMDELETE function B-174	disabling event handlers
customer-defined events 3-36	individually 2-20
CUSTOMINSERT function B-174	with Session Access As 2-19
CUSTOMLOAD function B-152	Dispatch IDO Request action type B-141
CUSTOMLOADPARMS function B-152	documentation, additional for Infor ERP SL -9
CUSTOMUPDATE function B-174	E
D	E function B-176
date expressions, rules C-212	EarliestResponse voting rule 5-68
DATE(ymd) function B-174	elements of the application event system 3-35
DATE(ymdhmsm) function B-174	ENTRYFORM function B-154
DATEADD function B-175	enumerations C-221
DATEDIFF function B-175	event action parameter forms 3-44
DATEPART function B-176	event actions 3-41
DBFUNCTION function B-176	controlling sequence 2-18
defining	defining 3-41
event actions 3-41	parameters B-147

- setting sequence order 3-52
- Event Actions form 3-41
- Event Global Constants form 3-46
- Event Handler Revisions form 4-59
- Event Handler Status form 4-58
- event handlers 3-40
 - controlling sequence 2-18
 - defining 3-40
 - designing custom 3-50
 - disabling individually 2-20
 - disabling with Session Access As 2-19
 - restricting which run 2-18
 - revisions 2-32
 - setting sequence order 3-51
 - setting up custom 3-49
 - status of
 - Failure 2-25
 - Success 2-25
 - success, failure, and retries 2-25
 - suspending 2-22
 - synchronicity (summary) B-125
 - synchronous and asynchronous 2-20
 - transactional 2-29
- Event Handlers form 3-40
- event messages 5-61– 5-70
 - indeterminate voting results 5-69
 - prompts and responses 5-65
 - related forms 5-61
 - voting rules 5-66
- event queue
 - Framework Event Service 2-31
 - processing order 2-31
- Event Queue form 4-58
- Event Revisions form 4-59
- Event Status form 4-57
- event triggers 3-38
 - defining 3-38
 - retesting 3-39
 - setting conditions for 3-39
- Event Triggers form 3-38
- Event Variable Groups form 3-45
- event variables 3-45
- EVENTNAME function B-154, B-176
- EVENTREVISION function B-176
- events 3-36
 - action types B-139
 - actions 3-41
 - asynchronous 2-20
 - defining 3-37
 - firing from tiers B-124
 - flow diagram 2-17
 - framework (core) B-126
 - global constants 3-46
 - handlers 3-40
 - handling 2-17
 - message-related forms 5-61
 - Inbox 5-61
 - Saved Messages 5-63
 - Send Message 5-64
 - naming 3-37
 - parameter forms 3-44
 - parameters 3-42
 - passing 3-44
 - setting values and variables 3-44
 - revisions 2-32
 - setting up custom 3-49

suspend-committing mode 2-23	keyword paren lists C-221
suspending 2-22	numeric-castable C-201
suspend-validating mode 2-22	operators B-194
synchronicity (summary) B-125	rules
synchronous 2-20	Boolean C-201
transactional 2-28	dates C-212
triggers 3-38	numeric C-210
defining 3-38	restricted arguments C-220
examples 2-17	strings C-206
retesting 3-39	rules for C-201
setting conditions for 3-39	scalar C-201
types 3-36	typeless, rules C-204
variables 3-45	F
where they can be generated from 2-18	Fail action type B-142
Events form 3-36	Failure status, event handler 2-25
EVENTSTATE function B-176	failures
EVENTTITLE function B-176	causes 2-25
Exclude Blank Access As setting	handling with stored procedures D-223
(Session Access As form) 2-19	ignoring 2-26
Execute IDO Request action type B-142	FE function B-176
expansions for functions	FGC function B-177
complex B-193	FILECONTENTS function B-177
nested B-194	FILTER function B-154, B-177
expressions	FILTERFORM function B-154
functions	FILTERMETHODPARM function B-177
pre-parser B-193	FILTERPROPERTY (FP) function B-178
standard B-171	FILTERSTRING function B-178
grammar C-197– C-222	Finish action type B-142
character sets C-199	FireGenericNotifySp A-120
enumerations C-221	firing events (from tiers) B-124
rules C-200	FLOOR function B-179
start symbol C-198	forms 3-37
terminals C-199	Access As 3-35

design	ACTIONTYPENAME B-172
authorization group for 3-47	ANYHANDLERSFAILED B-172
Event Actions 3-41	APPNAME B-172
Event Global Constants 3-46	BEGINDATE B-172
Event Handlers 3-40	BODY B-150
Event Triggers 3-38	CAST B-172
Event Variable Groups 3-45	CATEGORY B-150
Events 3-36	CC B-150
location 3-47	CEILING B-172
Workflow Event Handler Activation 3-48	CHOICES B-151
event action parameters 3-44	CLIENTSUBSTITUTE B-173
message-related 5-61	COLLECTION B-163
Inbox 5-61	COMMIT B-151
Saved Messages 5-63	COMPANYNAME B-173
Send Message 5-64	complex expansions B-193
status	CONDITION B-152
authorization group for 4-57	CONFIGNAME B-173
Event Handler Revisions 4-59	CURRDATETIME B-174
Event Handler Status 4-58	CUSTOMDELETE B-174
Event Queue 4-58	CUSTOMINSERT B-174
Event Revisions 4-59	CUSTOMLOAD B-152
Event Status 4-57	CUSTOMLOADPARMS B-152
location 4-57	CUSTOMUPDATE B-174
FP function B-179	DATE(ymd) B-174
Framework Event Service 2-31	DATE(ymdhmsm) B-174
administering 2-32	DATEADD B-175
processing order 2-31	DATEDIFF B-175
setting up 2-31	DATEPART B-176
framework events 3-36, B-126	DBFUNCTION B-176
FSV function B-179	DEST B-152, B-166
functions B-147	E B-176
ACTION B-148	ENTRYFORM B-154
ACTIONSEQ B-172	EVENTNAME B-154, B-176

EVENTREVISION B-176	KEY B-155
EVENTSTATE B-176	LEN B-181
EVENTTITLE B-176	LOADFLAGS B-181
FE B-176	LOWER B-181
FGC B-177	MESSAGE B-182
FILECONTENTS B-177	METHOD B-156, B-158, B-164, B-182
FILTER B-154, B-177	METHODPARM B-182
FILTERFORM B-154	METHODPARMS B-182
FILTERMETHODPARM B-177	MINIMUM B-156
FILTERPROPERTY (FP) B-178	MODIFIEDPAYLOADACCESS B-157
FILTERSTRING B-178	nested expansions B-194
FLOOR B-179	NEW B-157
for expressions	NEWGUID B-182
pre-parser B-193	NONRESPONDERLIST B-182
standard B-171	OLD B-157
FP B-179	OPTIMISTICLOCKING B-158
FSV B-179	ORDERBY B-158
FV B-179	ORIGINATOR B-183
GC B-179	P B-183
HANDLERIGNORESFAILURE B-179	PASSWORD B-160
HANDLERSUSPENDS B-179	PAYLOADACCESS B-160
HANDLERSYNCHRONOUS B-179	POSTQUERYACTIONS B-183
HANDLERTRANSACTIONAL B-180	POWER B-183
HASBEGUN B-180	PREFCHOICE B-161
HASFINISHED B-180	PROPERTIES B-161
IDO B-155, B-180	PROPERTY (P) B-184
IDOREQUEST B-155	PROPERTYMODIFIED B-184
IF B-180	PROPERTYNAMES B-184
IMAGE B-155	QUESTION B-161
INITIATOR B-180	QUORUM B-162
INSIDEDATABASE B-180	RECIPIENTLIST B-185
INSTR B-181	RECIPIENTS B-185
INTERVAL B-155	RECORDCAP B-162, B-185

REPLACE B-185	UNMODIFIEDPAYLOADACCESS B-167
RESPONDERLIST B-186	UPPER B-191
RESPONDERS B-187	URL B-168
RESULT B-163	USELOCALPASSWORD B-168
RETEST B-163	USERDESC B-191
ROUND B-188	USERNAME B-168, B-191
ROW B-163	V B-191
ROW SETPROPVALUES B-164	VARIABLENAME B-191
ROWS B-188	VARIABLEVALUE B-192
SAVEMESSAGE B-164	VOTINGDISPARITY B-192
SET B-158, B-159, B-162, B-163, B-166	VOTINGRESULT B-192
SETGLOBALVALUES B-164	VOTINGRULE B-170
SETMETHODPARMVALUES B-156	VOTINGTIE B-192
SETPARMVALUES B-160	WAITFORQUORUM B-170
SETPROPVALUES B-161	WORKINGDIR B-192
SETVALUE B-169	XML B-171
SETVARVALUES B-169	XSLT B-171
STATE B-164	functionss
SUBJECT B-164	DESC B-152
SUBSTITUTE B-189	FV function B-179
SUBSTRING B-190	G
SV B-190	GC function B-179
SYNCHRONOUS B-165	Generate Event action type B-142
TASKNAME B-165	GenericNotify B-130
TASKNUMBER B-165	GenericNotify event A-120
TASKPARMS B-165	GenericSendEmail B-130
TASKSTATUS B-165	global constants 3-46
TGC B-193	Goto action type B-143
TIMEOUT B-165	grammar for expressions C-197– C-222
TO B-166	character set C-199
TRANSACTIONAL B-166	enumerations C-221
TRUNC B-191	rules C-200
TV B-193	Boolean C-201

- dates C-212
- expressions C-201
- keyword paren lists C-221
- numeric C-210
- restricted arguments C-220
- string C-206
- typeless C-204
- variable, constant, and event parameter references C-200
- start symbol C-198
- terminals C-199

H

- HANDLERIGNORESFAILURE function B-179
- handlers 3-40
- Handlers, retrying 2-27
- HANDLERSEQ function
 - functions
 - HANDLERSEQ B-179
- HANDLERSUSPENDS function B-179
- HANDLERSYNCHRONOUS function B-179
- HANDLERTRANSACTIONAL function B-180
- handling events 2-17
- HASBEGUN function B-180
- HASFINISHED function B-180
- hierarchy of metadata (inherent) E-226

I

- IDO function B-155, B-180
- IdoOnInvoke B-128, B-132
- IdoOnItemDelete B-130, B-133
- IdoOnItemInsert B-130, B-133
- IdoOnItemUpdate B-130, B-134
- IdoOnLoadCollection B-128, B-134
- IdoOnPersistFailed B-128

- IdoOnUpdateCollection B-129, B-135
- IdoPostInvoke B-129, B-135
- IdoPostItemDelete B-129, B-135
- IdoPostItemInsert B-129, B-136
- IdoPostItemUpdate B-129, B-136
- IdoPostLoadCollection B-129, B-137
- IdoPostUpdateCollection B-129, B-137
- IDOREQUEST function B-155
- IF function B-180
- ignoring failures 2-26
- IMAGE function B-155
- Inbox
 - form 5-61
- indeterminate voting results for prompts 5-69
- Infor, contacting support -10
- Initial Action field 3-52
- initial states (variables) 3-45
- INITIATOR function B-180
- INSIDEDATABASE function B-180
- INSTR function B-181
- integrating metadata from several sources E-225–E-252
 - detailed examples E-228
 - using non-specific chronology E-232
 - using specific chronology E-228
- inherent hierarchy of metadata E-226
- maintaining handler IDs through updates E-227
- obsolete handlers E-249
- overriding others' handlers E-244
 - disabling E-249
 - exclusively E-247
 - non-exclusively E-245
- overview and rationale E-225

- performing upgrades E-240
- protecting events from changes E-227

Interpret contents, attributes 5-63

INTERVAL function B-155

K

Keep With fields 3-51

KEY function B-155

keywords

- paren lists C-221

L

LEN function B-181

Load IDO Collection action type B-143

Load IDO Row action type B-143

LOADFLAGS function B-181

location

- design forms 3-47

- status forms 4-57

LOWER function B-181

M

Majority voting rule 5-67

MESSAGE function B-182

message-related forms 5-61

- Inbox 5-61

- Saved Messages 5-63

- Send Message 5-64

messages (event) 5-61– 5-70

metadata

- inherent hierarchy E-226

- synchronizing E-225– E-252

METHOD function B-156, B-158, B-164, B-182

METHODPARM function B-182

METHODPARMS function B-182

MINIMUM function B-156

MinimumCount voting rule 5-68

MinimumCountPreferredChoice voting rule 5-69

MinimumPercentage voting rule 5-68

MinimumPercentagePreferredChoice voting rule 5-69

MODIFIEDPAYLOADACCESS function B-157

N

naming events 3-37

nested expansions of functions B-194

NEW function B-157

NEWGUID function B-182

NONRESPONDERLIST function B-182

Notify action type B-143

numeric expressions, rules C-210

numeric-castable expressions C-201

O

obsolete handlers E-249

OLD function B-157

operators for expressions B-194

OPTIMISTICLOCKING function B-158

ORDERBY function B-158

ORIGINATOR function B-183

P

P function B-183

parameters

- Action B-148

- Body B-150

- Category B-150

- CC (carbon copy) list B-150

- Choices B-151

- Collection B-163

- Commit B-151

- Condition B-152

Custom load method B-152	QUORUM B-162
Custom load method parameters B-152	Record cap B-162
Description B-152	Response (SET) B-162
Destination B-152	Restest interval B-163
Entry form B-154	Result B-163
Event name B-154	Result set identifier B-163
Filter B-154	Result XML (SET) B-163
Filter form B-154	Row B-163
for event actions B-147	Row number and property/expression pairs B-164
IDO B-155	Save in Sent Items? B-164
IDO request B-155	Session variable/Expression pairs B-164
Image B-155	setting value and variables 3-44
Interval B-155	State B-164
Key value B-155	Stored procedure name B-164
Method B-156	Subject B-164
Method Parameters B-156	Synchronous B-165
Minimum count or percentage B-156	syntax for 3-42
New value B-157	Task name B-165
Old value B-157	Task number B-165
Operation (METHOD) B-158	Task parameters B-165
Optimistic lock B-158	Task Status B-165
Order By clause B-158	Timeout B-165
Output B-158	Timeout destination B-166
Output parameters B-159	Title (SET) B-166
Parameter/Expression pairs B-160	To recipient list B-166
passing 3-44, D-223	Transaction B-166
Password B-160	URL B-168
Payload access B-157, B-160, B-167	USELOCALPASSWORD B-168
Preferred choice B-161	USERNAME B-168
Properties list B-161	Value (SETVALUE) B-169
Property/Expression pairs (SETPROPVAL-UES) B-161	Variable/Expression pairs B-169
Question B-161	Voting rule B-170

Wait for quorum B-170	reference tables B-123– B-196
XML B-171	REPLACE function B-185
XSLT B-171	RESPONDERLIST function B-186
paren lists for keywords C-221	RESPONDERS function B-187
PASSWORD function B-160	restricted arguments for expressions C-220
PAYLOADACCESS function B-160	restricting which handlers run 2-18
performing upgrades, integrating metadata E-240	RESULT function B-163
Plurality voting rule 5-67	resumption of adjourned events 2-25
POSTQUERYACTIONS function B-183	RETEST function B-163
POWER function B-183	Retest Interval parameter B-163
PREFCHOICE function B-161	retesting event triggers 3-39
PreferredChoice voting rule 5-68	Retrying event handlers 2-27
procedural code 1-14, -260	retrying event handlers 2-25
ProcessNewDataMaintenance B-131	revisions, events and event handlers 2-32
Prompt action type B-144	rolling back transactions 2-30
prompts and responses 5-65	ROUND function B-188
indeterminate voting results 5-69	ROW function B-163
voting rules 5-66	ROW SETPROPVALUES function B-164
PROPERTIES function B-161	ROWS function B-188
PROPERTY (P) function B-184	rules
PROPERTYMODIFIED function B-184	expression grammar
PROPERTYNAMES function B-184	constant references C-200
Q	dates C-212
QUESTION function B-161	event parameter references C-200
queued events	numeric C-210
Framework Event Service 2-31	restricted arguments C-220
processing order 2-31	strings C-206
QUORUM function B-162	typeless expressions C-204
Quorums 5-70	variable references C-200
R	voting, prompts 5-66
RECIPIENTLIST function B-185	Run Background Task action type B-144
RECIPIENTS function B-185	S
RECORDCAP function B-162, B-185	samples

scenarios A-71– A-121	handlers 3-51
stored procedures D-223	variable and parameter values 3-44
Saved Messages form 5-63	SETVALUE function B-169
SAVEMESSAGE function B-164	SETVARVALUES function B-169
scalar C-201	Sleep action type B-145
scalar expression expressions C-201	STATE function B-164
Scalar tuples C-205	status
scenarios, samples A-71– A-121	event handlers
Send BOD action type B-145	Failure 2-25
Send Email action type B-145	Success 2-25
Send Message form 5-64	forms
sequence	authorization group for 4-57
event actions 2-18, 3-52	Event Handler Revisions 4-59
event handlers 2-18, 3-51	Event Handler Status 4-58
Service Configuration Manager utility 2-31	Event Queue 4-58
Session Access As form 2-19	Event Revisions 4-59
SessionOnLogin B-127, B-137	Event Status 4-57
SessionOnLoginFail B-127	location 4-57
SessionOnLoginFailed B-138	tracking 4-57– 4-59
SessionOnLogout B-127, B-139	stored procedures
SessionOnVarChanged B-127, B-139	calling synchronous events and handling failure D-223
SessionPostVarChanged B-127, B-139	passing parameters to a synchronous event D-223
Set Attributes action type B-145	samples D-223
SET function B-158, B-159, B-162, B-163, B-166	strings, rules C-206
Set Values action type B-145	SUBJECT function B-164
SETGLOBVALUES function B-164	SUBSTITUTE function B-189
SETMETHODPARAMVALUES function B-156	SUBSTRING function B-190
SETPARMVALUES function B-160	Success event handler status 2-25
SETPROPVALUES function B-161	support
setting	contacting Infor -10
conditions for event triggers 3-39	suspend-committing mode 2-23
sequence order	suspend-validating mode 2-22
actions 3-52	

suspension 2-22– 2-24

- event handlers 2-22
- events 2-22
- process for non-suspended events 2-24
- process for suspended events 2-22

SV function B-190

synchronicity 2-20– 2-21

- event handlers (summary) B-125
- events (summary) B-125

synchronization of metadata E-225– E-252

- inherent hierarchy of metadata E-226
- overview and rationale E-225

synchronous

- calling events from stored procedures D-223
- event handlers 2-20
- events 2-20
- events in transactions 2-28

SYNCHRONOUS function B-165

syntax for event parameters 3-42

T

TaskListCheck B-131

TASKNAME function B-165

TASKNUMBER function B-165

TASKPARMS function B-165

TASKSTATUS function B-165

TGC function B-193

ties in prompt votes 5-69

Timeout destination parameter B-166

TIMEOUT function B-165

Title parameter (SET) B-166

TO function B-166

tracking status of events 4-57– 4-59

transactional

event handlers 2-29

events 2-28

TRANSACTIONAL function B-166

transactions 2-28– 2-30

- rolling back 2-30
- with asynchronous events 2-30
- with synchronous events 2-28

Transform XML action type B-146

translatable captions 5-63

triggers for events 3-38

- defining 3-38
- examples 2-17
- retesting 3-39
- setting conditions for 3-39

TRUNC function B-191

TV function B-193

types of events 3-36

U

UNMODIFIEDPAYLOADACCESS function B-167

Update Collection action type B-146

upgrades, integrating metadata through E-240

UPPER function B-191

URL function B-168

USELOCALPASSWORD function B-168

USERDESC function B-191

USERNAME function B-168, B-191

V

V function B-191

Value (SETVALUE) parameter B-169

VARIABLENAME function B-191

variables

- for events 3-45

setting parameter values 3-44	VOTINGRESULT function B-192
VARIABLEVALUE function B-192	VOTINGRULE function B-170
voting results indeterminate for prompts 5-69	VOTINGTIE function B-192
voting rules for prompts 5-66	W
ConditionalPlurality 5-67	Wait action type B-146
EarliestResponse 5-68	Wait for Quorum 5-70
Majority 5-67	WAITFORQUORUM function B-170
MinimumCount 5-68	where events can be named 3-37
MinimumCountPreferredChoice 5-69	Workflow Event Handler Activation form 3-48
MinimumPercentage 5-68	Workflow event handlers 1-16
MinimumPercentagePreferredChoice 5-69	WORKINGDIR function B-192
Plurality 5-67	X
PreferredChoice 5-68	XML function B-171
VOTINGDISPARITY function B-192	XSLT function B-171
